

The Zebra Programming Language

A Modern Language with Safety and Performance

Zebra Community

April 30, 2026

Contents

1	Part 1: Foundations	1
2	01: Getting Started	3
2.1	The Big Picture	3
2.2	What is Zebra?	4
2.3	Installation	4
2.3.1	macOS and Linux	4
2.3.2	Windows	4
2.3.3	From Source	5
2.4	Your First Program	5
2.4.1	What just happened?	6
2.4.2	If you're new to programming	6
2.4.3	If you know Python	6
2.4.4	Verbosity dial — annotations are optional when the type is obvious	7
2.5	The Toolchain	8
2.6	Common Commands	8
2.7	Your Second Program (A Little Interesting)	9
2.7.1	What's new?	9
2.8	Understanding Errors	10
2.9	Project Structure (Thinking Ahead)	10
2.10	Using Other Files: use and exposing	11
2.10.1	Selective Imports with exposing	11
2.11	Exercises	12
2.11.1	Exercise 1: Customize the Greeting	12
2.11.2	Exercise 2: Combine Multiple Variables	12
2.12	Next Steps	13
2.13	Key Takeaways	13
2.14	Troubleshooting	13
2.15	The Big Picture	14
2.16	Basic Types	14
2.16.1	Integers	14
2.16.2	Floating Point	16
2.16.3	Booleans	17
2.16.4	Strings	18

2.17	Declaring Variables	18
2.17.1	Type Inference (the default)	18
2.17.2	Explicit Types (when they aid the reader)	19
2.17.3	Naming Conventions	19
2.18	Comparison and Equality	20
2.19	Type Conversions	20
2.19.1	Implicit Conversion (When Safe)	20
2.19.2	Explicit Conversion	21
2.20	Nullable Types (Introduction)	21
2.20.1	If you're new to programming	22
2.21	Type Mismatch Errors	22
2.22	Real World: Processing User Data	23
2.23	Common Mistakes	24
2.24	Exercises	25
2.24.1	Exercise 1: Type Conversion	25
2.24.2	Exercise 2: Comparisons	25
2.24.3	Exercise 3: User Profile	26
2.25	Next Steps	27
2.26	Key Takeaways	27
2.27	The Big Picture	27
2.28	Lists	28
2.28.1	Creating Lists	28
2.28.2	List Operations	30
2.28.3	Iteration Patterns	31
2.28.4	If you're new to programming	31
2.28.5	If you know Python	32
2.29	HashMaps	32
2.29.1	Creating and Using HashMaps	32
2.29.2	HashMap Operations	34
2.29.3	If you know Python	34
2.30	Deduplication with HashMap	35
2.31	Real World: Data Processing	36
2.32	Common Patterns	37
2.32.1	Filter and Transform	37
2.33	Common Mistakes	38
2.34	Exercises	39
2.34.1	Exercise 1: List Operations	39
2.34.2	Exercise 2: HashMap Lookup	40
2.34.3	Exercise 3: Unique Words	40
2.35	Next Steps	41
2.36	Key Takeaways	41
2.37	The Big Picture	42
2.38	Basic Functions	42
2.38.1	Defining Functions	42
2.38.2	Functions With Multiple Parameters	43

2.38.3	Optional Logic (No Return Value)	43
2.39	Scope	44
2.39.1	Local Scope	45
2.39.2	Shadowing (Careful!)	45
2.39.3	If you're new to programming	46
2.40	Closures and Capture	46
2.40.1	Capture Syntax	47
2.41	Real World: Reusable Utilities	48
2.42	Common Patterns	49
2.42.1	Early Return	49
2.42.2	Default Arguments	49
2.42.3	Passing Functions As Arguments	50
2.43	Common Mistakes	51
2.44	Exercises	52
2.44.1	Exercise 1: Simple Function	52
2.44.2	Exercise 2: Closure	52
2.44.3	Exercise 3: Filtering	53
2.45	Next Steps	54
2.46	Key Takeaways	54
2.47	The Big Picture	54
2.48	If / Else	55
2.48.1	Basic If	55
2.48.2	If / Else	55
2.48.3	Multiple Conditions	56
2.49	Match (Pattern Matching)	56
2.49.1	Simple Match	57
2.49.2	Match With Types	57
2.49.3	If you know Python	58
2.50	Loops	59
2.50.1	For Loop (Iteration)	59
2.50.2	While Loop (Condition-Based)	60
2.50.3	Break and Continue	60
2.51	Guards	61
2.52	Real World: Data Validation	62
2.53	Common Patterns	63
2.53.1	Early Return	63
2.53.2	Nested Conditions	64
2.53.3	Else-If Chain	64
2.54	Common Mistakes	65
2.55	Exercises	66
2.55.1	Exercise 1: Grade Calculator	66
2.55.2	Exercise 2: Find Maximum	67
2.55.3	Exercise 3: Filter With Control Flow	68
2.56	Next Steps	69
2.57	Key Takeaways	69

2.58	The Big Picture	70
2.59	String Basics	70
2.59.1	String Literals	70
2.59.2	String Properties	71
2.59.3	String Interpolation	72
2.60	String Methods	73
2.60.1	Searching	73
2.60.2	Splitting and Joining	73
2.60.3	Trimming and Padding	74
2.60.4	Replacing	75
2.61	Unicode and Internationalization	77
2.61.1	Unicode Basics	77
2.61.2	Character Iteration	77
2.62	Regular Expressions (Intro)	78
2.62.1	Basic Patterns	78
2.63	Real World: Text Processing	79
2.64	Common Patterns	81
2.64.1	Email Validation	81
2.64.2	URL Parsing	81
2.64.3	String Templating	81
2.65	Common Mistakes	82
2.66	Exercises	83
2.66.1	Exercise 1: String Reversal	83
2.66.2	Exercise 2: Email Validator	84
2.66.3	Exercise 3: CSV Parsing	85
2.67	Next Steps	85
2.68	Key Takeaways	86
3	Part 2: Objects and Interfaces	87
3.1	The Big Picture	87
3.2	Defining Classes	89
3.2.1	Simple Class	89
3.2.2	Fields and Initialization	90
3.2.3	Instance Methods	90
3.2.4	Static Methods (Class Methods)	91
3.2.5	If you're new to programming	92
3.3	Real World: User Management	93
3.4	Common Patterns	95
3.4.1	Value Object	95
3.4.2	Service Class	95
3.4.3	Builder Pattern	95
3.5	Common Mistakes	96
3.6	Exercises	97
3.6.1	Exercise 1: Bank Account	97
3.6.2	Exercise 2: Product Catalog	98

3.7	Next Steps	99
3.8	Key Takeaways	100
3.9	The Big Picture	100
3.10	Defining Interfaces	101
3.10.1	Simple Interface	101
3.10.2	Multiple Methods	102
3.11	Polymorphism in Action	103
3.11.1	Using Different Implementations	103
3.11.2	Collections of Interface Types	104
3.12	Real World: Logging System	105
3.13	Common Patterns	107
3.13.1	Strategy Pattern	107
3.13.2	Adapter Pattern	108
3.14	If you're new to programming	109
3.14.1	If you know Python	109
3.15	Common Mistakes	110
3.16	Exercises	112
3.16.1	Exercise 1: Shape Interface	112
3.16.2	Exercise 2: Database Interface	113
3.16.3	Exercise 3: Document Processor	114
3.17	Next Steps	115
3.18	Key Takeaways	116
4	09: Composition and Mixins	117
4.1	Why no inheritance?	117
4.2	Mixins — sharing methods across classes	118
4.3	Composition — has-a, not is-a	120
4.4	Mixin vs. composition — when to use each	121
4.5	Multiple mixins — building richer classes	122
4.6	Polymorphism — reach for interfaces (Chapter 08)	123
4.7	Real world: a document with capabilities	124
4.8	If you're new to programming	126
4.9	Common mistakes	126
4.10	Exercises	129
4.10.1	Exercise 1: Vehicle behaviours	129
4.10.2	Exercise 2: Employees and managers — without inheritance	130
4.10.3	Exercise 3: Shapes implementing a common interface	132
4.11	Next Steps	134
4.12	Key Takeaways	134
5	10: Properties and Computed Values	137
5.1	The Big Picture	137
5.2	Getters	138
5.2.1	Simple Getter	138
5.2.2	Derived Properties	138

5.3	Setters (Validation)	139
5.3.1	Controlling Field Assignment	139
5.3.2	Setter with Side Effects	141
5.4	Computed Properties	142
5.5	Lazy Initialization	143
5.6	Real World: Temperature Converter	145
5.7	Real World: Configuration	146
5.8	Common Patterns	147
5.8.1	Read-Only Properties	147
5.8.2	Write-Only Properties	148
5.9	If you're new to programming	148
5.10	Common Mistakes	149
5.11	Exercises	151
5.11.1	Exercise 1: Bank Account Properties	151
5.11.2	Exercise 2: Circle Properties	152
5.11.3	Exercise 3: User Profile with Validation	153
5.12	Next Steps	155
5.13	Key Takeaways	155
6	Part 3: Advanced Features	157
6.1	The Big Picture	157
6.2	Nullable Types	158
6.2.1	Marking Optionality	158
6.3	Nil Checking	158
6.3.1	Safe Access Pattern	158
6.3.2	Type Narrowing	159
6.4	<code>if x as n</code> — Unwrap-and-Bind	161
6.5	The <code>to!</code> Operator (Unwrap)	162
6.6	Unwrap with Fallback	163
6.7	Real World: Database Queries	163
6.8	Common Patterns	165
6.8.1	Optional Chaining	165
6.8.2	Guard Clauses	165
6.9	If you're new to programming	166
6.10	Common Mistakes	166
6.11	Exercises	167
6.11.1	Exercise 1: Find User Email	167
6.11.2	Exercise 2: Safe Division	168
6.11.3	Exercise 3: Nullable Chain	169
6.12	Next Steps	171
6.13	Key Takeaways	171
6.14	The Big Picture	172
6.15	Functions That Can Fail: <code>throws</code>	172
6.16	<code>raise</code> — Signaling Errors	173
6.17	<code>try / catch</code> — Handling Errors	174

6.17.1	Block Form	174
6.17.2	Catch with Binding	174
6.17.3	Catch with Type	175
6.18	<code>catch</code> Expression — Inline Fallbacks	175
6.19	Error Propagation	176
6.20	Real World: API Client	177
6.21	Exercises	178
6.21.1	Exercise 1: Safe Division	178
6.21.2	Exercise 2: Chained Error Handling	179
6.22	Next Steps	180
6.23	Key Takeaways	180
6.24	The Big Picture	181
6.25	Generic Classes	182
6.26	Generic Methods	184
6.27	Generic Collections	185
6.28	Type Constraints	186
6.29	Real World: Generic Cache	188
6.30	Common Mistakes	190
6.30.1	Mistake 1: Forgetting to Instantiate Generic Parameters	190
6.30.2	Mistake 2: Mixing Types in a Generic Container	190
6.30.3	Mistake 3: Using Constraints Incorrectly	190
6.30.4	Mistake 4: Type Erasure at Runtime	191
6.31	Exercises	191
6.31.1	Exercise 1: Generic Stack	191
6.31.2	Exercise 2: Generic Filter Function	192
6.31.3	Exercise 3: Generic Wrapper with Validation	193
6.32	Key Takeaways	194
6.33	Next Steps	195
7	14: Contracts and Assertions	197
7.1	The Big Picture	197
7.2	Preconditions: <code>require</code>	198
7.3	Postconditions: <code>ensure</code>	199
7.3.1	The <code>result</code> keyword	200
7.3.2	The <code>old</code> snapshot	201
7.4	Invariants: <code>invariant</code>	201
7.5	Assertions: <code>assert</code>	203
7.6	Real-world example: a sorted list	204
7.7	<code>--turbo</code> : stripping contracts for production	206
7.8	If you're new to programming	206
7.9	Common Mistakes	206
7.10	Exercises	208
7.10.1	Exercise 1: Bank account with contracts	208
7.10.2	Exercise 2: Validated array indexer	209
7.10.3	Exercise 3: Stack with size invariant	211

7.11	Key Takeaways	212
7.12	Next Steps	213
7.13	The Big Picture	213
7.14	Basic Pipeline Syntax	214
7.15	Chaining Multiple Operations	215
7.16	Pipelines with Collections	216
7.17	Custom Functions in Pipelines	217
7.18	Real World: Data Transformation Pipeline	218
7.19	Function Composition Pattern	220
7.20	Pipelines with Result Types	221
7.21	Common Mistakes	222
7.21.1	Mistake 1: Forgetting the Dot Placeholder	222
7.21.2	Mistake 2: Breaking the Chain at the Wrong Place	222
7.21.3	Mistake 3: Piping to Functions with Multiple Parameters	223
7.21.4	Mistake 4: Over-Piping (Readability)	223
7.22	Exercises	224
7.22.1	Exercise 1: Text Processing Pipeline	224
7.22.2	Exercise 2: Number Transformation Pipeline	225
7.22.3	Exercise 3: List Filtering Pipeline	226
7.23	Key Takeaways	227
7.24	Next Steps	228
8	Part 4: Practical Projects	229
8.1	Project Overview	229
8.2	Step 1: Command-Line Argument Parsing	231
8.3	Step 2: File Reading and Basic Counting	232
8.4	Step 3: Pattern Matching	236
8.5	Step 4: Main Application Logic	238
8.6	Part 2: Adding Features	241
8.7	Part 3: Full Implementation	242
8.8	Testing Your Project	242
8.9	Exercises & Extensions	243
8.9.1	Exercise 1: Add Line Numbering Output	243
8.9.2	Exercise 2: Find Longest Line	243
8.9.3	Exercise 3: Statistics Summary	244
8.9.4	Exercise 4: Case-Insensitive Search	244
8.9.5	Challenge: Support Multiple Files	245
8.10	Key Concepts Reinforced	246
8.11	Architecture Decisions	247
8.12	What You've Built	247
8.13	Performance Notes	247
8.14	Project 2: HTTP Server (16-18 hours)	248
8.14.1	Step 1: HTTP Request/Response Types	248
8.14.2	Step 2: Request Routing	252
8.14.3	Step 3: Server Implementation	255

8.14.4	Exercises	257
8.14.5	Testing the Server	257
8.15	Project 3: Text Data Analysis (12-15 hours)	258
8.15.1	Step 1: Frequency Analysis	258
8.15.2	Step 2: N-gram Extraction	261
8.15.3	Step 3: Similarity Analysis	264
8.15.4	Step 4: Main Analysis Application	266
8.15.5	Exercises	269
8.15.6	Testing	270
8.16	Project Comparison Summary	270
8.17	Capstone Challenge: Integrated System	270
8.18	Project Comparison	271
8.19	Progressive Difficulty	272
8.20	Capstone Challenges	272
8.21	Expected Outcomes	272
9	Part 5: Ecosystem and Reference	273
10	Part 5: Ecosystem (Chapters 19-22)	275
10.1	Chapter 19: Standard Library Tour	275
10.1.1	Overview	275
10.1.2	Core Modules	275
10.1.3	Real World	276
10.2	Chapter 20: File I/O and System Access	276
10.2.1	File Operations	276
10.2.2	Line-by-Line Reading	277
10.2.3	System Access	277
10.2.4	Key Topics	277
10.3	Chapter 21: Regular Expressions (Deep Dive)	278
10.3.1	Regex Pattern Syntax	278
10.3.2	Common Patterns	278
10.3.3	Operations	279
10.3.4	Real-World Use Cases	279
10.4	Chapter 22: FFI and Interop	280
10.4.1	Calling C Code	280
10.4.2	Calling Zig Code	280
10.4.3	Type Marshaling	280
10.4.4	Safety Considerations	281
10.4.5	Real-World Applications	281
10.5	Appendices	281
10.5.1	Appendix A: Grammar Reference	281
10.5.2	Appendix B: Built-in Functions and Stdlib	282
10.5.3	Appendix C: Troubleshooting	283
10.6	Quick Reference	283
10.7	Where to Go Next	284

Chapter 1

Part 1: Foundations

Chapter 2

01: Getting Started

Audience: All

Time: 30 minutes

Prerequisites: None

You'll learn: Install Zebra, run your first program, understand the toolchain

2.1 The Big Picture

Zebra is a **modern, statically-typed language** that prioritizes **safety, clarity, and correctness**. It combines ideas from Python, C#, Objective-C, and Eiffel into a pragmatic language for building robust systems.

This book teaches you to write Zebra code fluently. By the end, you'll understand: - How Zebra's type system catches bugs at compile time - Why nil tracking and error handling are first-class citizens - How to write clean, expressive code that reads almost like pseudocode

Let's get your environment set up and run your first program.

2.2 What is Zebra?

Short version: A statically-typed language that feels dynamic, with compile-time safety guarantees that prevent entire categories of bugs.

Key properties: - **Compiled** — Zebra code compiles to Zig (which then compiles to machine code) - **Strongly typed** — Types are checked at compile time, preventing type mismatches - **Nil-safe** — You explicitly mark values that can be `nil`, preventing null pointer errors - **Error-aware** — Errors are values you handle, not exceptions you ignore - **Expressive** — Clear, readable syntax that emphasizes intent over cleverness

Who uses it: People building systems where reliability matters (CLI tools, servers, data processing, embedded systems).

2.3 Installation

2.3.1 macOS and Linux

```
# Install Zig first (Zebra compiles to Zig)
# https://ziglang.org/download/
# Then install Zebra
# https://github.com/cobra-language/zebra/releases

# Verify installation
zebra --version
```

2.3.2 Windows

```
# Download from: https://github.com/cobra-language/zebra/releases
# Extract to a directory
# Add to PATH
```

```
# Verify:
zebra --version
```

2.3.3 From Source

```
git clone https://github.com/cobra-language/zebra.git
cd zebra/zig-compiler
zig build
# Binary at: zig-out/bin/zebra
```

2.4 Your First Program

Create a file named `hello.zbr`:

```
# file: hello.zbr
# teaches: hello world, print statement
# chapter: 01-Getting-Started

def main()
    print "Hello, Zebra!"
```

Run it:

```
zebra hello.zbr
```

Output:

Hello, Zebra!

2.4.1 What just happened?

1. **def main()** — Defined the entry point function. Zebra runs **main** automatically when the program starts.
2. **print "..."** — Printed text to the console.
3. **zebra hello.zbr** — Compiled and ran the program.

That's the whole program. No class wrapper, no boilerplate.

2.4.2 If you're new to programming

A **function** is a reusable block of code. **main** is special — it runs automatically when you start the program.

A **statement** is an instruction. **print "Hello, Zebra!"** is a statement that outputs text.

Larger programs often group related code into **classes** (Chapter 07). Small scripts don't need them.

2.4.3 If you know Python

```
# Python
print("Hello, World!")

# Zebra
print "Hello, Zebra!"
```

Where Python's entry point is implicit (top-level statements run at import), Zebra's is explicit (you define **main**). The clarity scales: as programs grow you can group code into classes and modules without reshaping the entry point.

2.4.4 Verbosity dial — annotations are optional when the type is obvious

Zebra has full type inference. You can write either:

```
def main()
    var name: str = "World"      # explicit – annotation aids the reader
    var age: int = 30
    var pi: float = 3.14159
    print "${name} is ${age}"
```

or, equivalently:

```
def main()
    var name = "World"          # inferred from the literal
    var age = 30
    var pi = 3.14159
    print "${name} is ${age}"
```

Both compile to the same Zig. Use **annotations when they aid the reader** (function signatures, public class fields, hairy generic returns) and **drop them when the right-hand side already makes the type obvious** (literals, named-constructor calls, expressions whose return type is plain). The book mostly uses the inferred form going forward; you'll see explicit annotations where they earn their keep.

You can also still use the class-based entry point if you prefer it (older code does):

```
class Main
    static
        def main
            print "Hello, Zebra!"
```

Both forms work. Bare `def main()` is the recommended default for scripts and small programs.

2.5 The Toolchain

Zebra's workflow is:

hello.zbr (Zebra code)

↓

zebra compiler

↓

hello.zig (Zig code)

↓

zig compiler

↓

hello.exe (Executable)

↓

Run: ./hello

Why two steps? Zebra targets Zig because Zig is: - Low-level (good for systems programming) - Safe (catches undefined behavior at compile time) - Explicit (no hidden magic) - Fast (compiles to efficient machine code)

As a Zebra programmer, you mostly ignore the Zig step. You write `.zbr`, run `zebra <file>`, and get a fast executable.

2.6 Common Commands

```
# Compile and run
zebra hello.zbr
```

```
# Compile only (leaves hello.exe)
zebra -c hello.zbr

# Run with debug output
zebra -v=2 hello.zbr

# Keep intermediate Zig code for inspection
zebra -kif hello.zbr
```

2.7 Your Second Program (A Little Interesting)

```
# file: greet.zbr
# teaches: variables, string interpolation
# chapter: 01-Getting-Started

def main()
    var name = "World"
    print "Hello, ${name}!"
```

Output:

Hello, World!

2.7.1 What's new?

- **var name = "World"** — Declared a variable named `name` and initialized it with `"World"`. The type `str` is inferred from the literal; you could also write `var name: str = "World"` if you wanted to make the type explicit.
 - **"Hello, \${name}!"** — String interpolation: `${name}` is replaced with the value of `name`.
-

2.8 Understanding Errors

When you make a mistake, Zebra tells you clearly:

```
def main()
    print "Hello " + 5 # ? Can't add string + number
```

Error:

```
greet.zbr:5:24: error: arithmetic requires numeric type, got 'str'
```

```
    print "Hello " + 5
                  ^
```

What it means: - **File and line:** `greet.zbr:5:24` — error at line 5, column 24 - **Error type:** “arithmetic requires numeric type” — you tried math with incompatible types - **The problem:** `+` expects numbers, but got `"Hello "` (a string)

Fix: Use string concatenation:

```
print "Hello ".concat(5.toString())
```

Or use string interpolation:

```
print "Hello ${5}"
```

2.9 Project Structure (Thinking Ahead)

As programs grow, organize them:

```
my-project/
├─ main.zbr      # Entry point
├─ utils.zbr     # Helper functions
├─ data.zbr      # Data structures
└─ lib/
    └─ db.zbr    # Database utilities
```

Later chapters will show how to use multiple files. For now, keep everything in one `.zbr` file.

2.10 Using Other Files: **use** and **exposing**

When your program spans multiple files, Zebra’s **use** statement imports another module:

```
# file: math_utils.zbr
def square(n: int): int
    return n * n
```

```
# file: main.zbr
use math_utils exposing square

def main()
    var answer = square(5)
    print answer # 25
```

Naming note: `result` is a reserved keyword in Zebra (it binds the return value inside an **ensure** block — see Chapter 14). Pick another name like `answer` or `total` for ordinary locals.

2.10.1 Selective Imports with **exposing**

`use math_utils` brings the module into scope as `math_utils.square(5)`. Adding **exposing** `square` lifts the name in directly so you can call `square(5)` without the prefix. You can expose multiple names at once:

```
use ast exposing Stmt, Expr, TypeRef, DeclVar
```

This keeps your code clean when you use many names from a module. You’ll see `use...exposing` extensively in later chapters.

2.11 Exercises

2.11.1 Exercise 1: Customize the Greeting

Modify `hello.zbr` to: 1. Declare a variable for your name 2. Print a greeting using that variable

Solution

```
def main()
  var my_name = "Alice"
  print "Hello! My name is ${my_name}."
```

Output:

Hello! My name is Alice.

Why this works: We declared `my_name` (inferred as `str`), then used it in interpolation.

2.11.2 Exercise 2: Combine Multiple Variables

Create a program that: 1. Declares three variables: `first_name`, `last_name`, and `age` 2. Prints a sentence combining all three

Solution

```
def main()
  var first_name = "Bob"
  var last_name = "Smith"
  var age = 30
  print "My name is ${first_name} ${last_name} and I'm ${age} years old."
```

Output:

My name is Bob Smith and I'm 30 years old.

Why this works: We declared three variables — two strings and one integer, all inferred from their literals — and used all three in a single string interpolation.

2.12 Next Steps

- → **02-Values-and-Types** — Dive into Zebra’s type system
 - → **03-Collections** — Work with Lists and HashMaps
 - **Project-1-CLI-Tool** — Build something real
-

2.13 Key Takeaways

- **Zebra is compiled** — Code becomes fast executables
 - **Entry point is explicit** — Every program defines `main` (top-level `def main()`, or `Main.main` inside a class for larger programs)
 - **Type annotations are optional** — Use them where they aid the reader; lean on inference when the right-hand side is obvious
 - **Strings use interpolation** — “`${variable}`” is easier to read than concatenation
 - **Errors are clear** — The compiler helps you fix problems
 - **Simple programs are simple** — Print and variables are enough for Hello World
-

2.14 Troubleshooting

“**zebra: command not found**” → Zebra isn’t in your PATH. Check the installation instructions for your OS.

“**error: file not found: hello.zbr**” → Make sure you’re in the directory with `hello.zbr`.
Try: `ls hello.zbr`

“**error: arithmetic requires numeric type**” → You mixed types (like `string + number`).
Use `toString()` or string interpolation instead.

Program compiles but doesn't run → The executable was created but you need to run it. Try: `./hello` (Unix/Mac) or `hello.exe` (Windows)

Ready? Head to **02-Values-and-Types** and learn the full type system.

02: Values and Types

Audience: All

Time: 90 minutes

Prerequisites: 01-Getting-Started

You'll learn: Zebra's type system, declaring variables, type inference, value vs. reference semantics

2.15 The Big Picture

Every value in Zebra has a **type**. The type tells you: - What kind of data it holds - What operations you can perform on it - How much memory it uses - Whether it can be **nil**

Zebra's type system is **your best friend**—it catches mistakes at compile time instead of in production.

2.16 Basic Types

2.16.1 Integers

```
# file: 02_integers.zbr
# teaches: integer types, arithmetic
# chapter: 02-Values-and-Types
```

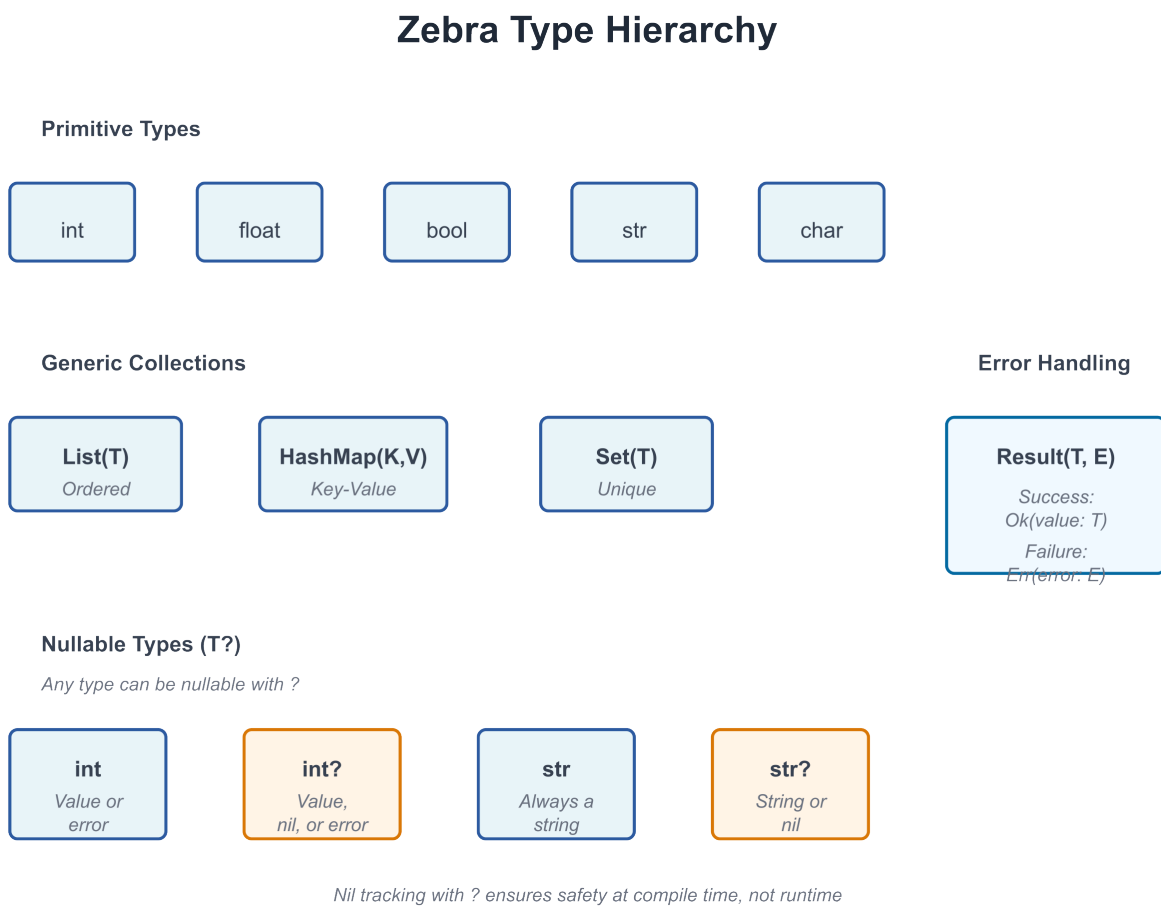


Figure 2.1: Type Hierarchy Diagram

```
def main()
    var age = 25

    var population = 8_000_000_000    # Underscores for readability
    var tiny: int8 = 100              # explicit narrow type
    var huge: int64 = 9_223_372_036_854_775_807

    print age
    print population
    print tiny
    print huge
```

Integer types: - `int` — Default integer (usually 64-bit) - `int8`, `int16`, `int32`, `int64` — Specific sizes - `uint`, `uint8`, etc. — Unsigned (non-negative) integers

Arithmetic:

```
var x = 10
var y = 3
print x + y    # 13
print x - y    # 7
print x * y    # 30
print x / y    # 3 (integer division)
print x % y    # 1 (remainder)
```

2.16.2 Floating Point

```
# file: 02_floats.zbr
# teaches: float types, precision
# chapter: 02-Values-and-Types

def main()
    var pi = 3.14159
```

```
var precise: float64 = 3.141592653589793

print pi
print precise

# Arithmetic
var result = pi * 2.0
print result
```

Float types: - `float` — Default floating point (usually 64-bit) - `float32`, `float64` — Specific precision

2.16.3 Booleans

```
# file: 02_bools.zbr
# teaches: boolean values, logic
# chapter: 02-Values-and-Types

def main()
    var is_ready = true
    var is_finished = false

    print is_ready
    print is_finished

    # Logic
    print true and false    # false
    print true or false     # true
    print not true          # false
```

2.16.4 Strings

```
# file: 02_strings.zbr
# teaches: string type, string operations
# chapter: 02-Values-and-Types

def main()
  var greeting = "Hello"
  var name = "World"

  print greeting
  print greeting.len      # 5
  print greeting.upper()  # HELLO
  print name.lower()      # world

  # Concatenation
  var message = greeting.concat(" ").concat(name)
  print message           # Hello World
```

String methods (we'll cover these fully in Chapter 06): - `.len` — Length - `.upper()` — Uppercase - `.lower()` — Lowercase - `.concat()` — Concatenate - `.contains()` — Check for substring - `.split()` — Split by delimiter

2.17 Declaring Variables

2.17.1 Type Inference (the default)

Zebra infers the type from the right-hand side. Most variables don't need an annotation:

```
var age = 25           # Inferred as int
var name = "Alice"     # Inferred as str
var active = true      # Inferred as bool
var pi = 3.14          # Inferred as float
```

2.17.2 Explicit Types (when they aid the reader)

You can always pin the type explicitly. Reach for this when: - the right-hand side is a function call whose return type isn't obvious to a reader - you want a different concrete width (`int8`, `float32`) than inference would pick - the variable starts as `nil` or `undefined` and inference can't narrow it

```
var age: int = 25           # redundant; inference would do
var payment_amount: float = compute_total() # earns its keep – return type isn't visible
var counter: int8 = 0       # narrower than the default int
var maybeUser: User? = nil  # required: nil alone has no type
```

The book uses inference by default and adds annotations only where they earn their keep.

2.17.3 Naming Conventions

Use `snake_case` for variables:

```
var user_id = 42          # ☑ Good
var UserID = 42           # ☒ Avoid (that's for classes)
var user_id_number = 42   # ☑ Clear but verbose
var uid = 42              # ☒ Too abbreviated
```

2.18 Comparison and Equality

```
# file: 02_comparisons.zbr
# teaches: comparison operators
# chapter: 02-Values-and-Types

def main()
  var x = 10
  var y = 20

  print x == y    # false (equal)
  print x != y    # true (not equal)
  print x < y     # true (less than)
  print x > y     # false (greater than)
  print x <= y    # true (less or equal)
  print x >= y    # false (greater or equal)

  var name = "Alice"
  print name == "Alice"  # true
  print name == "Bob"    # false
```

2.19 Type Conversions

2.19.1 Implicit Conversion (When Safe)

```
var small: int8 = 100
var big: int = small  # ☞ Fine: int8 → int is always safe
```

2.19.2 Explicit Conversion

```
# file: 02_conversions.zbr
# teaches: type conversion
# chapter: 02-Values-and-Types

def main()
  var x = 42

  var s = x.toString()    # int → str
  print s

  var pi = 3.14
  var i = pi.toInt()      # float → int (loses decimal)
  print i                 # 3

  var flag = true
  var b = flag.toString() # bool → str
  print b                 # true
```

2.20 Nullable Types (Introduction)

Sometimes a value might not exist. Zebra uses `?` to mark this:

```
# file: 02_nullable.zbr
# teaches: nullable types introduction
# chapter: 02-Values-and-Types

def main()
  var name: str? = "Alice"    # Can hold str or nil
  var empty: str? = nil       # Explicitly nil
```

```
# You must check before using
if name != nil
    print name           # Safe to use

if empty != nil
    print empty
else
    print "Name is empty"
```

Key point: `str?` means “string or nil”. We’ll explore nil tracking fully in Chapter 11.

2.20.1 If you’re new to programming

A **nullable type** is like an optional value. Sometimes you have it, sometimes you don’t. Instead of errors when it’s missing, Zebra lets you check first.

2.21 Type Mismatch Errors

Zebra prevents mixing types unsafely:

```
var x: int = "hello" # Error: can't assign str to int
var y: str = 42      # Error: can't assign int to str
var z: int = 3.14    # Error: can't assign float to int
```

Why? Type safety catches bugs before they happen. You can’t accidentally treat a string as a number.

```
# file: 02_user_data.zbr
# teaches: realistic variable use
# chapter: 02-Values-and-Types

class User
  var id: int
  var name: str
  var email: str
  var age: int
  var is_active: bool

def main()
  # Create a user
  var user = User()
  user.id = 1
  user.name = "Alice"
  user.email = "alice@example.com"
  user.age = 30
  user.is_active = true

  # Display
  print "User: ${user.name}"
  print "Email: ${user.email}"
  print "Age: ${user.age}"
  print "Active: ${user.is_active}"
```

2.23 Common Mistakes

Mistake: Skipping a type annotation when the right-hand side is opaque

```
var result = compute() # What's the type?
```

Why it matters: Inference is fine when the literal makes the type obvious (`var x = 25`). When the RHS is a function call whose return type isn't visible at the call site, an annotation pays for itself in readability.

Better:

```
var result: Int = compute() # Clear: it's an Int
```

Mistake: Mixing types in arithmetic

```
var x = 10
var y = 3.14
print x + y # ❌ Can't add Int + Double
```

Why: Different types might need different handling.

Better:

```
var x = 10.0
var y = 3.14
print x + y # ✅ Both Double
```

Mistake: Using a nullable type without checking

```
var name: String? = nil
print name.upper() # ❌ Error: nil doesn't have .upper()
```

Better:

```
var name: String? = nil
if name != nil
    print name.upper() # Safe
```

2.24 Exercises

2.24.1 Exercise 1: Type Conversion

Write a program that: 1. Creates an integer variable 2. Creates a float variable 3. Converts both to strings 4. Prints them

Solution

```
def main()
    var count = 42
    var price = 19.99

    var count_str = count.toString()
    var price_str = price.toString()

    print "Count: ${count_str}"
    print "Price: ${price_str}"
```

Output:

Count: 42

Price: 19.99

2.24.2 Exercise 2: Comparisons

Write a program that compares two numbers and prints which is larger:

Solution

```
def main()
    var x = 100
    var y = 75
```

```
if x > y
    print "${x} is larger than ${y}"
else
    print "${y} is larger than ${x}"
```

Output:

100 is larger than 75

2.24.3 Exercise 3: User Profile

Create a data structure to hold user information (name, age, email) and print a profile:

Solution

```
class Person
    var name: str
    var age: int
    var email: str

def main()
    var person = Person()
    person.name = "Carol"
    person.age = 28
    person.email = "carol@example.com"

    print "Name: ${person.name}"
    print "Age: ${person.age}"
    print "Email: ${person.email}"
```

2.25 Next Steps

- → **03-Collections** — Lists and HashMaps
 - → **04-Functions** — Write reusable code
 - → **11-Nil-Tracking** — Deep dive into nullable types
-

2.26 Key Takeaways

- **Every value has a type** — Integers, floats, booleans, strings, and more
 - **Types are checked at compile time** — Catches bugs early
 - **Use type inference when obvious, explicit types when clarity matters**
 - **Conversions are explicit** — No silent type changes
 - **Nullable types use ?** — `str?` can be string or nil
 - **Always check nil before using** — `if value != nil { ... }`
-

Next: Head to **03-Collections** to learn Lists and HashMaps.

03: Collections

Audience: All

Time: 120 minutes

Prerequisites: 01-Getting-Started, 02-Values-and-Types

You'll learn: Lists, HashMaps, Sets, iteration, indexing, collection methods

2.27 The Big Picture

Collections let you group values. Instead of declaring 100 separate variables for 100 names, you use one `List(str)` that holds all 100.

Zebra provides: - **List(T)** — Ordered, resizable sequences (like Python's list) - **HashMap(K, V)** — Key-value pairs (like Python's dict) - **StrSet** — Unique strings (a specialized set; the only flavor of **Set** Zebra ships today)

Collections Comparison in Zebra

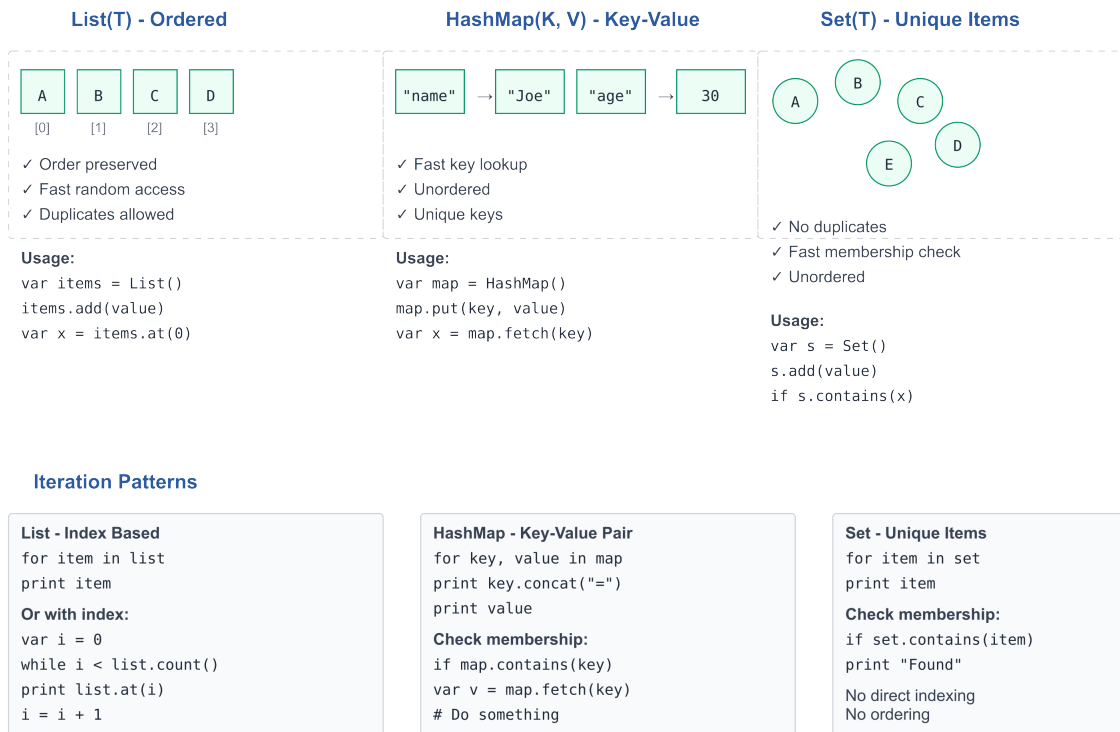


Figure 2.2: Collections Comparison

2.28 Lists

A **List** holds multiple values of the same type in order.

2.28.1 Creating Lists

```
# file: 03_lists.zbr
# teaches: list creation and access
```

```
# chapter: 03-Collections

def main()

  # List literal – the cleanest way to make a populated list.
  # The element type is inferred from the first element.
  var fruits = ["apple", "banana", "cherry"]

  # An empty list with an explicit annotation – and `.add()` to grow:
  var empty: List(str) = []
  empty.add("date")
  print empty.count()          # 1

  # Access by index – assign through a typed local before printing
  # so the formatter picks {s} instead of the byte-array fallback.
  var first: str = fruits.at(0)
  var second: str = fruits.at(1)
  print first    # apple
  print second   # banana

  # Check size
  print fruits.count() # 3

  # Iterate – assign through a typed local for {s} formatting (BUG-090 workaround)
  for fruit in fruits
    var f: str = fruit
    print f
```

The constructor form `var fruits = List(str)()` is still valid — useful when you want an empty list to grow without specifying an element type up front (the element type is inferred from the first `.add()`). For populated lists, `[...]` is shorter

and reads better.

Print formatting note: when the formatter can't see the element type — for example, a fresh for-loop binding from a `List(str)`, or a class field of type `str` — `print` falls back to byte-array formatting (`{ 97, 112, ... }`). Assigning through a typed local (`var f: str = fruit`) gives the formatter the `[]const u8` it expects. This is a known gap (BUG-089/090); examples in this chapter include the workaround so they actually print as you'd expect.

2.28.2 List Operations

```
# file: 03_list_ops.zbr
# teaches: list manipulation
# chapter: 03-Collections

def main()
    var nums = [10, 20, 30]           # list literal – type inferred

    # Check existence
    print nums.contains(20)           # true

    # Find index
    print nums.indexOf(20)            # 1

    # Remove by index (List.remove takes an index, not a value)
    nums.remove(1)
    print nums.count()                # 2

    # Clear
    nums.clear()
    print nums.count()                # 0
```

2.28.3 Iteration Patterns

```
# file: 03_iteration.zbr
# teaches: different iteration styles
# chapter: 03-Collections

def main()
    var items = List(str)()
    items.add("first")
    items.add("second")
    items.add("third")

    # Simple iteration (typed local works around print's {s} fallback)
    for item in items
        var s: str = item
        print s

    # Iteration with index – interpolation into a string handles the type fine
    var i = 0
    while i < items.count()
        print "${i}: ${items.at(i)}"
        i = i + 1
```

2.28.4 If you're new to programming

A **List** is like a numbered shelf. You can:

- Add items: `list.add(item)` - Take items: `list.remove(item)`
- Check what's there: `list.at(0)` gets the first item - Count items: `list.count()`

2.28.5 If you know Python

```
# Python
fruits = ["apple", "banana"]
fruits.append("cherry")
for fruit in fruits:
    print(fruit)

# Zebra
var fruits = List(str)()
fruits.add("apple")
fruits.add("banana")
fruits.add("cherry")
for fruit in fruits
    print fruit
```

The main difference: Zebra requires the element type (`List(str)`) at the construction site; once declared, the type flows through inference.

2.29 HashMaps

A `HashMap` stores key-value pairs. Fast lookup by key.

2.29.1 Creating and Using HashMaps

```
# file: 03_hashmaps.zbr
# teaches: hashmap creation and access
# chapter: 03-Collections

def main()
```

```
# Create empty HashMap
var ages = HashMap(str, int)()

# Add key-value pairs
ages.put("Alice", 30)
ages.put("Bob", 25)
ages.put("Carol", 28)

# Retrieve by key – get returns int? (nil if missing)
if ages.get("Alice") as a
    print a                # 30

# Check if key exists
print ages.contains("Alice")    # true

# Iterate over key/value pairs
for name, age in ages
    print "${name}: ${age}"
```

Two notes on the API:

- `.put()/.fetch()` are older `HashMap` method names that still work; `.set()/.get()` are the canonical forms used by QUICKSTART. `.fetch()` returns the value directly and panics on missing keys; `.get()` returns an optional — the safer pattern.
- `HashMap` key/value iteration (`for k, v in m`) is the canonical form, but a known compiler gap in the current selfhost (BUG-094, filed separately) emits a spurious `_ = name`; discard that Zig rejects. As a workaround until that's fixed: iterate the keys via a list you maintain alongside the map, or read values back via `.get(known_key)` for the small number of cases where you need values during a loop.

2.29.2 HashMap Operations

```
# file: 03_hashmap_ops.zbr
# teaches: hashmap manipulation
# chapter: 03-Collections

def main()
  var config = HashMap(str, str)()
  config.put("host", "localhost")
  config.put("port", "8080")
  config.put("debug", "true")

  # Count entries
  print config.count()           # 3

  # Remove entry
  config.remove("debug")
  print config.count()           # 2

  # Look up safely – get returns str? (nil if missing)
  if config.get("host") as host
    print host                   # localhost

  # Iterate over keys and values (see BUG-094 note above on the kv-loop gap)
  for key, value in config
    print "${key} = ${value}"
```

2.29.3 If you know Python

```
# Python
ages = {"Alice": 30, "Bob": 25}
print(ages["Alice"])
for name, age in ages.items():
    print(name, age)

# Zebra
var ages = HashMap(str, int)()
ages.put("Alice", 30)
ages.put("Bob", 25)
if ages.get("Alice") as a
    print a
for name, age in ages
    print "${name} ${age}"
```

2.30 Deduplication with HashMap

Need unique values? Use a `HashMap` where keys track membership:

```
# file: 03_dedup.zbr
# teaches: using HashMap for uniqueness
# chapter: 03-Collections

def main()
    var seen = HashMap(int, bool)()
    var unique = List(int)()

    var ids = [1, 2, 3, 2]           # last element is a duplicate
```

```
for id in ids
    if not seen.contains(id)
        seen.put(id, true)
        unique.add(id)

print unique.count()    # 3

# Check membership
print seen.contains(2)  # true
```

2.31 Real World: Data Processing

```
# file: 03_real_world.zbr
# teaches: collections in realistic scenarios
# chapter: 03-Collections

class Student
    var name: str
    var gpa: float

def main()
    # List of students
    var students = List(Student)()

    var alice = Student()
    alice.name = "Alice"
    alice.gpa = 3.9
    students.add(alice)
```

```
var bob = Student()
bob.name = "Bob"
bob.gpa = 3.5
students.add(bob)

# Calculate average GPA
var total = 0.0
for student in students
    total = total + student.gpa
var average = total / students.count()
print "Average GPA: ${average}"

# Find student by name
var target_name = "Alice"
for student in students
    if student.name == target_name
        print "Found: ${student.name} (${student.gpa})"
```

2.32 Common Patterns

2.32.1 Filter and Transform

```
# file: 03_patterns.zbr
# teaches: collection patterns
# chapter: 03-Collections

def main()
    var numbers = [1, 2, 3, 4, 5]
```

```
# Filter: keep only even numbers
var evens: List<int> = []
for num in numbers
    if num % 2 == 0
        evens.add(num)

print "Evens: "
for e in evens
    print e

# Count matching items
var count_gt_3 = 0
for num in numbers
    if num > 3
        count_gt_3 = count_gt_3 + 1
print "Numbers > 3: ${count_gt_3}"
```

2.33 Common Mistakes

Mistake: Forgetting type parameters

```
var items = List() # What type? List(what)?
```

Better:

```
var items = List<str>() # Clear: list of strings
```

Mistake: Iterating and modifying

```
for item in items
    items.remove(item) # ⚠ Unsafe: modifying while iterating
```

Better:

```
var to_remove = List(str)()
for item in items
    if should_remove(item)
        to_remove.add(item)
for item in to_remove
    items.remove(items.indexOf(item))
```

Mistake: Using wrong key type for HashMap

```
var map = HashMap(str, int)()
map.put(1, 100) # ⚠ Key should be str, not int
```

Better:

```
var map = HashMap(str, int)()
map.put("count", 100) # ⚠ Key is str
```

2.34 Exercises

2.34.1 Exercise 1: List Operations

Create a list of numbers and find the sum:

Solution

```
def main()
    var nums = [10, 20, 30, 40]

    var sum = 0
```

```
for num in nums
    sum = sum + num

print "Sum: ${sum}" # 100
```

2.34.2 Exercise 2: HashMap Lookup

Create a phone book and look up a number:

Solution

```
def main()
    var phone_book = HashMap(str, str)()
    phone_book.put("Alice", "555-1234")
    phone_book.put("Bob", "555-5678")
    phone_book.put("Carol", "555-9999")

    var name = "Bob"
    if phone_book.get(name) as number
        print "${name}'s number: ${number}"
```

2.34.3 Exercise 3: Unique Words

Count unique words in a sentence (using HashMap for deduplication):

Solution

```
def main()
    var text = "the quick brown fox jumps over the lazy dog"
    var words: List(str) = text.split(" ") # typed annotation auto-collects the iterator

    var seen = HashMap(str, bool)()
    for word in words
```

```
seen.put(word, true)

print "Total words: ${words.count()}"
print "Unique words: ${seen.count()}"
```

2.35 Next Steps

- → **04-Functions** — Reuse collection-processing code
 - → **05-Control-Flow** — Pattern matching on collections
 - **Project-3-Data-Analysis** — Real collection processing
-

2.36 Key Takeaways

- **List(T)** holds ordered items, access by index
 - **HashMap(K,V)** holds key-value pairs, fast lookup
 - **Deduplication** — use **HashMap** keys for unique-value tracking
 - **Iteration** with `for item in collection` is the main pattern
 - **Type parameters** are required: **List(str)** not just **List**
 - **Modifying while iterating** is unsafe; collect changes first
-

Next: Head to **04-Functions** to write reusable code.

04: Functions and Scope

Audience: All

Time: 120 minutes

Prerequisites: 01-02-03

You'll learn: Define functions, parameters, return values, scope, closures, capture blocks

2.37 The Big Picture

Functions let you: - **Reuse code** — Write once, call many times - **Organize logic** — Group related operations - **Name ideas** — `process_payment()` is clearer than inline logic - **Test parts** — Functions are easier to test in isolation

2.38 Basic Functions

2.38.1 Defining Functions

```
# file: 04_functions.zbr
# teaches: function definition and calling
# chapter: 04-Functions-and-Scope

def add(a: int, b: int): int
    return a + b

def multiply(x: int, y: int): int
    return x * y

def main()
    var result1 = add(5, 3)
    print result1                # 8

    var result2 = multiply(4, 7)
    print result2                # 28
```

Breakdown: - `def add(...)` — Define a function named `add` at the module top level. No surrounding class is needed when the function is just a reusable operation. - `(a: int, b: int)` — Parameters: `a` and `b`, both integers. - `: int` (after the closing paren) — Return type is integer. - `return a + b` — The value to return.

When you do want to group related functions on a class — for namespacing, or because they share class state — use `static def name(...)` inside `class Name`. We'll show that pattern when it earns its keep (Chapter 07).

2.38.2 Functions With Multiple Parameters

```
# file: 04_multi_params.zbr
# teaches: multiple parameters
# chapter: 04-Functions-and-Scope

def pad(text: str, width: int, fill: str): str
    # (Implementation would go here)
    return text

def main()
    var padded = pad("hi", 10, "*")
    print padded
```

2.38.3 Optional Logic (No Return Value)

```
# file: 04_void.zbr
# teaches: functions that don't return values
# chapter: 04-Functions-and-Scope

def log(message: str)
    print "[LOG] ${message}"
```

```
def main()
    log("Application started")
    log("Processing data")
    log("Done")
```

When a function doesn't return a value, omit the : `<type>` after the parameter list.

2.39 Scope

Scope is where a variable can be accessed.

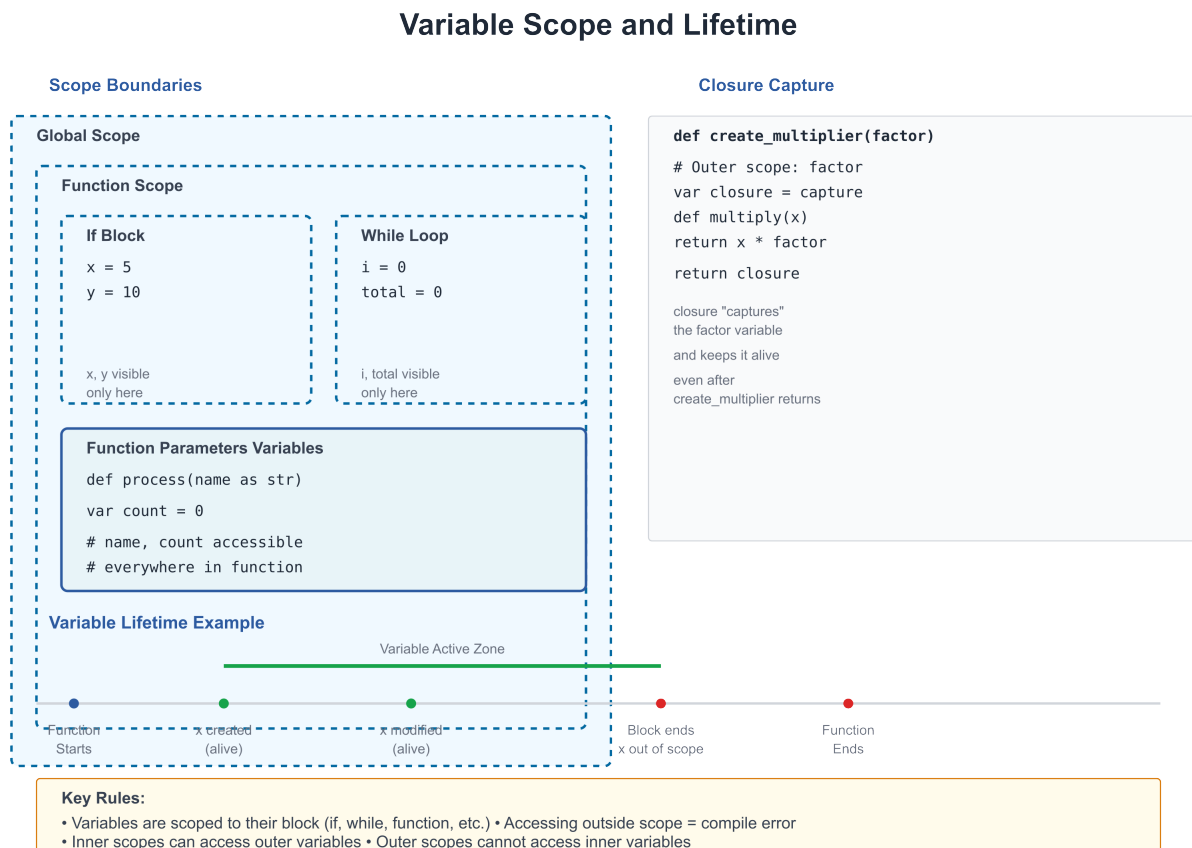


Figure 2.3: Scope and Lifetime

2.39.1 Local Scope

Variables declared inside a function are local (only accessible there):

```
# file: 04_scope.zbr
# teaches: variable scope
# chapter: 04-Functions-and-Scope

class Calculator
  static
    var global_val: int = 100 # Shared (class-level)

    def compute()
      var local_val = 50      # Local to this function
      print local_val         # ⓘ Can access
      print global_val        # ⓘ Can access class-level

    def other()
      # print local_val       # ⓘ Can't access, it's not here
      pass

def main()
  Calculator.compute()
```

2.39.2 Shadowing (Careful!)

```
class Example
  static
    var count: int = 10      # Class-level

    def show()
```

```
var count = 5          # Local (shadows class variable)
print count            # 5 (local shadows class)

def another():
    print count        # 10 (uses class-level)
```

Shadowing can be confusing. Rename to clarify intent.

2.39.3 If you're new to programming

Scope determines where you can use a variable. - **Local:** Only inside the function where it's declared - **Class-level:** Accessible from all functions in the class

2.40 Closures and Capture

Functions can create inner functions that capture outer variables:

```
# file: 04_closures.zbr
# teaches: closures and variable capture
# chapter: 04-Functions-and-Scope

def make_multiplier(factor: int): def(int)
    return def(x: int)
        capture
            var factor: int = factor
            return x * factor

def main()
    var double = make_multiplier(2)
```

```
var triple = make_multiplier(3)

print double(5)           # 10
print triple(5)           # 15
```

What's happening: 1. `make_multiplier(2)` creates a function that multiplies by 2 2. `double = ...` stores that function in a variable 3. `double(5)` calls it: $5 * 2 = 10$

The `capture` block explicitly captures `factor` from the outer scope.

2.40.1 Capture Syntax

```
# file: 04_capture.zbr
# teaches: capture blocks
# chapter: 04-Functions-and-Scope

def make_counter(): def(): int
    var count: int = 0
    return def(): int
        capture
            var count: int = count
            count = count + 1
            return count

def main()
    var counter = make_counter()
    print counter()           # 1
    print counter()           # 2
    print counter()           # 3
```

2.41 Real World: Reusable Utilities

```
# file: 04_utilities.zbr
# teaches: practical function use
# chapter: 04-Functions-and-Scope

def repeat_str(text: str, times: int): str
    var result = ""
    var i = 0
    while i < times
        result = result.concat(text)
        i = i + 1
    return result

def pad_left(text: str, width: int): str
    var padding_needed = width - @as(int, text.len)
    if padding_needed <= 0
        return text
    var padding = repeat_str(" ", padding_needed)
    return padding.concat(text)

def main()
    var padded = pad_left("hello", 10)
    print "[${padded}]"           # [      hello]
```

Aside: the `@as(int, text.len)` cast works around BUG-093 (`s.len` is `usize`, not `int`, so it can't be subtracted from an `int` literal directly). Once that bug is fixed the cast becomes optional.

2.42 Common Patterns

2.42.1 Early Return

```
# file: 04_patterns.zbr
# teaches: early return pattern
# chapter: 04-Functions-and-Scope

def validate_email(email: str): bool
    if email.len == 0
        return false
    if not email.contains("@")
        return false
    if not email.contains(".")
        return false
    return true
```

2.42.2 Default Arguments

Zebra supports parameter defaults directly — you don't need an overload:

```
def log(message: str, prefix: str = "[INFO]")
    print "${prefix} ${message}"

def main()
    log("Application started")           # [INFO] Application started
    log("Disk failing", "[WARN]")        # [WARN] Disk failing
```

You can also pass arguments by name when that aids the reader:

```
log(prefix: "[ERROR]", message: "Out of memory")
```

2.42.3 Passing Functions As Arguments

```
# file: 04_higher_order.zbr
# teaches: functions as arguments
# chapter: 04-Functions-and-Scope

def map_int(items: List(int), transform: def(int): int): List(int)
  var result = List(int)()
  for item in items
    result.add(transform(item))
  return result

def main()
  var nums = List(int)()
  nums.add(1)
  nums.add(2)
  nums.add(3)

  var double_it = def(x: int): int = x * 2
  var doubled = map_int(nums, double_it)

  for d in doubled
    print d                # 2, 4, 6
```

2.43 Common Mistakes

Mistake: Returning wrong type

```
def get_age(): int
    return "25" # ❌ Returning str, not int
```

Better:

```
def get_age(): int
    return 25 # ✅ Returning int
```

Mistake: Using variable outside its scope

```
def process()
    var data = fetch_data()

def use()
    print data # ❌ data doesn't exist here
```

Better:

```
def process(): str
    var data = fetch_data()
    return data

def use()
    var data = process()
    print data # ✅ data is here
```

Mistake: Forgetting to return

```
def compute(): int
    var result = 5 + 3
    # Forgot return!
```

Better:

```
def compute(): int
    var result = 5 + 3
    return result
```

2.44 Exercises

2.44.1 Exercise 1: Simple Function

Write a function that calculates the factorial of a number:

Solution

```
def factorial(n: int): int
    if n <= 1
        return 1
    return n * factorial(n - 1)

def main()
    print factorial(5)    # 120
```

2.44.2 Exercise 2: Closure

Create a function that returns an “adder” function:

Solution

```
def make_adder(base: int): def(int): int
    return def(x: int): int
        capture
            var base: int = base
        return x + base
```

```
def main()
    var add_10 = make_adder(10)
    print add_10(5)           # 15
    print add_10(20)         # 30
```

2.44.3 Exercise 3: Filtering

Write a function that filters a list:

Solution

```
def even_numbers(nums: List(int)): List(int)
    var evens = List(int)()
    for num in nums
        if num % 2 == 0
            evens.add(num)
    return evens

def main()
    var nums = List(int)()
    nums.add(1)
    nums.add(2)
    nums.add(3)
    nums.add(4)
    nums.add(5)

    var result = even_numbers(nums)
    for r in result
        print r
```

2.45 Next Steps

- → **05-Control-Flow** — if/else, loops, pattern matching
 - → **15-Pipelines** — Function composition
 - **Project-1-CLI-Tool** — Use functions extensively
-

2.46 Key Takeaways

- **Functions are reusable blocks** of code with clear inputs and outputs
 - **Scope** determines where variables can be accessed
 - **Closures** capture outer variables for use in inner functions
 - **Early return** makes logic clearer
 - **Functions as arguments** enable powerful abstractions
 - **Name functions well** — `process_payment()` is better than `do_thing()`
-

Next: Head to **05-Control-Flow** to learn if/else, loops, and pattern matching.

05: Control Flow

Audience: All

Time: 90 minutes

Prerequisites: 01-04

You'll learn: if/else, match, for loops, while loops, guards, break/continue

2.47 The Big Picture

Control flow directs which code runs based on conditions. Instead of executing every line in order, you: - **Skip code** if a condition isn't met (**if**) - **Repeat code** until done (**for**, **while**) - **Branch** based on many options (**match**)

2.48 If / Else

2.48.1 Basic If

```
# file: 05_if.zbr
# teaches: conditional execution
# chapter: 05-Control-Flow

def main()
    var age = 18

    if age >= 18
        print "You can vote"
```

2.48.2 If / Else

```
# file: 05_if_else.zbr
# teaches: if-else branching
# chapter: 05-Control-Flow

def main()
    var score = 75

    if score >= 90
        print "A"
    else if score >= 80
        print "B"
    else if score >= 70
```

```
    print "C"
else
    print "F"
```

Breakdown: - `if condition` — Execute if true - `else if condition` — Try the next condition (Zebra spells the chain `else if`, two words) - `else` — Default case (if all above are false)

2.48.3 Multiple Conditions

```
# file: 05_conditions.zbr
# teaches: boolean logic
# chapter: 05-Control-Flow

def main()
    var age = 25
    var has_license = true

    if age >= 18 and has_license
        print "Can drive"

    if age < 16 or not has_license
        print "Cannot drive"
```

Operators: - `and` — Both must be true - `or` — At least one must be true - `not` — Flip the value

2.49 Match (Pattern Matching)

Match is more powerful than `if/else`. It handles many cases elegantly.

2.49.1 Simple Match

```
# file: 05_match.zbr
# teaches: pattern matching
# chapter: 05-Control-Flow

def main()
  var color = "red"

  branch color
    on "red"
      print "Stop"
    on "yellow"
      print "Caution"
    on "green"
      print "Go"
    else
      print "Unknown"
```

2.49.2 Match With Types

Zebra’s idiomatic way to dispatch on “what kind of thing is this” is a **union** plus **branch**.

Each variant of the union carries its own data:

```
# file: 05_match_type.zbr
# teaches: type-based matching with unions
# chapter: 05-Control-Flow

union Pet
  dog: str      # name
  cat: str      # name
```

```
fish: str      # tank-size

def main()

  var pet: Pet = Pet.dog("Rex")

  branch pet
    on Pet.dog as name
      print "${name} says: Woof!"
    on Pet.cat as name
      print "${name} says: Meow!"
    on Pet.fish as size
      print "tank: ${size}"
    else
      pass
```

Zebra has no class inheritance, so dispatching on a class hierarchy isn't the natural pattern here. See **Chapter 09 (Composition and Mixins)** for why, and **Chapter 07b (Structs and Unions)** for more on tagged unions.

2.49.3 If you know Python

```
# Python
match color:
  case "red":
    print("Stop")
  case "green":
    print("Go")

# Zebra
branch color
```

```
on "red"  
    print "Stop"  
on "green"  
    print "Go"
```

2.50 Loops

2.50.1 For Loop (Iteration)

```
# file: 05_for.zbr  
# teaches: for loop iteration  
# chapter: 05-Control-Flow  
  
def main()  
    var fruits = List(str)()  
    fruits.add("apple")  
    fruits.add("banana")  
    fruits.add("cherry")  
  
    for fruit in fruits  
        var f: str = fruit      # typed local for {s} formatting  
        print f  
  
    # Numeric range – n.to(end) is exclusive on `end`.  
    var i = 0  
    while i < fruits.count()  
        print "Item ${i}: ${fruits.at(i)}"  
        i = i + 1
```

2.50.2 While Loop (Condition-Based)

```
# file: 05_while.zbr
# teaches: while loop
# chapter: 05-Control-Flow

def main()
    var count = 0

    while count < 5
        print "Count: ${count}"
        count = count + 1

    print "Done!"
```

2.50.3 Break and Continue

```
# file: 05_break_continue.zbr
# teaches: loop control
# chapter: 05-Control-Flow

def main()
    # Break: exit loop early
    var i = 0
    while true
        if i == 5
            break
        print i
        i = i + 1
```


The `guard cond, return` form reads as “the call must satisfy `cond`; otherwise return now”. It’s equivalent to `if not cond: return` but states the precondition positively. Block form (`guard cond ... else`) is also available — see QUICKSTART §13.

2.52 Real World: Data Validation

```
# file: 05_validation.zbr
# teaches: practical control flow
# chapter: 05-Control-Flow

def validate_email(email: str): bool
    # Check not empty
    if email.len == 0
        return false

    # Check has @
    if not email.contains("@")
        return false

    # Check has domain
    var parts: List(str) = email.split("@")
    if parts.count() != 2
        return false

    var domain: str = parts.at(1)
    if not domain.contains(".")
        return false
```

```
    return true

def main()
    var emails = List(str)()
    emails.add("alice@example.com")
    emails.add("invalid.email")
    emails.add("bob@domain.co")

    for email in emails
        var e: str = email
        if validate_email(e)
            print "Valid: ${e}"
        else
            print "Invalid: ${e}"
```

2.53 Common Patterns

2.53.1 Early Return

```
def process_order(order_id: int): bool
    var order = fetch_order(order_id)

    if order == nil
        return false

    if order.total == 0
        return false
```

```
if not order.has_items
    return false

# Do the real processing
charge_card(order)
ship_items(order)
return true
```

2.53.2 Nested Conditions

```
for user in users
    if user.is_active
        for order in user.orders
            if order.status == "pending"
                process_order(order)
```

2.53.3 Else-If Chain

```
if age < 13
    category = "child"
else if age < 18
    category = "teen"
else if age < 65
    category = "adult"
else
    category = "senior"
```

2.54 Common Mistakes

Mistake: Forgetting the condition

```
if age >= 18 # Missing condition or wrong structure
    print "Adult"
    print "Extra" # Both print regardless
```

Better:

```
if age >= 18
    print "Adult"
else
    print "Minor"
```

Mistake: Unreachable code

```
if x > 0
    return true
else
    return false
return false # ☹ Never reached
```

Better:

```
if x > 0
    return true
return false
```

Mistake: Mutating collection while iterating

```
for fruit in fruits
    if fruit == "apple"
        fruits.remove(0) # ☹ Unsafe – modifying during iteration
```

Better:

```
var to_remove = List<Int>() # collect indices
var idx = 0
for fruit in fruits
    if fruit == "apple"
        to_remove.add(idx)
        idx = idx + 1
# Remove in reverse so earlier indices stay valid.
var k = to_remove.count() - 1
while k >= 0
    fruits.remove(to_remove.at(k))
    k = k - 1
```

2.55 Exercises

2.55.1 Exercise 1: Grade Calculator

Write a function that converts numeric scores to letter grades:

Solution

```
def grade_to_letter(score: Int): String
    if score >= 90
        return "A"
    else if score >= 80
        return "B"
    else if score >= 70
        return "C"
    else if score >= 60
        return "D"
    else
```

```
        return "F"

def main()
    var scores = List<Int>()
    scores.add(95)
    scores.add(75)
    scores.add(88)
    scores.add(62)

    for score in scores
        var grade = grade_to_letter(score)
        print "${score} = ${grade}"
```

2.55.2 Exercise 2: Find Maximum

Iterate through a list and find the maximum value:

Solution

```
def find_max(nums: List<Int>): Int
    if nums.count() == 0
        return 0

    var max_val = nums.at(0)
    for num in nums
        if num > max_val
            max_val = num
    return max_val

def main()
    var nums = List<Int>()
```

```
nums.add(10)
nums.add(45)
nums.add(23)
nums.add(89)
nums.add(34)

var max = find_max(nums)
print "Max: ${max}" # 89
```

2.55.3 Exercise 3: Filter With Control Flow

Count elements matching a condition:

Solution

```
def count_evens(nums: List(int)): int
    var count = 0
    for num in nums
        if num % 2 == 0
            count = count + 1
    return count

def main()
    var nums = List(int)()
    nums.add(1)
    nums.add(2)
    nums.add(3)
    nums.add(4)
    nums.add(5)
    nums.add(6)
```

```
var even_count = count_evens(nums)
print "Even numbers: ${even_count}" # 3
```

2.56 Next Steps

- → **06-Strings-and-Unicode** — Text manipulation
 - → **05-Control-Flow** — Combine control flow with strings
 - **Project-1-CLI-Tool** — Use loops and conditions extensively
-

2.57 Key Takeaways

- **if/else** handles simple binary choices
 - **match** (branch) is better for many cases
 - **for** loops iterate over collections
 - **while** loops repeat based on conditions
 - **break** exits loops, **continue** skips iterations
 - **Early return** makes functions clearer
 - **Guard clauses** prevent “arrow code” (deeply nested if statements)
-

Next: Head to **06-Strings-and-Unicode** to work with text.

06: Strings and Unicode

Audience: All

Time: 90 minutes

Prerequisites: 01-05

You’ll learn: String operations, Unicode, formatting, pattern matching with regex

2.58 The Big Picture

Strings are how programs work with text. Zebra treats strings as: - **Unicode-aware** — Emoji, Chinese, Arabic, etc. all work correctly - **First-class** — Rich library of methods - **Immutable** — Can't change them after creation (create new ones instead) - **Efficient** — UTF-8 encoding optimizes storage

2.59 String Basics

2.59.1 String Literals

```
# file: 06_string_basics.zbr
# teaches: string creation
# chapter: 06-Strings-and-Unicode

def main()
    # Simple string
    var greeting = "Hello"
    print greeting

    # With quotes inside
    var quoted = "She said \"Hello\""
    print quoted

    # Multi-line (triple-quoted)
    var poem = """
    Roses are red
```

```
Violets are blue
"""
print poem

# Escape sequences
var path = "C:\\Users\\Name\\Documents"
var tab = "Name\tAge\tCity"
var newline = "Line1\nLine2"
```

2.59.2 String Properties

```
# file: 06_string_props.zbr
# teaches: string properties and methods
# chapter: 06-Strings-and-Unicode

def main()
    var text = "Hello, World!"

    # Byte length (Unicode-aware count is below – see codePointCount)
    print text.len                # 13

    # Character at index – text[i] returns a char, .toString() lifts it back to str
    var first_char = text[0]
    print first_char.toString()   # H

    # Substring/slice
    var part: str = text[0..5]
    print part                    # Hello

    # Case conversion
```

```
print text.upper()          # HELLO, WORLD!
print text.lower()          # hello, world!
```

2.59.3 String Interpolation

```
# file: 06_interpolation.zbr
# teaches: string interpolation
# chapter: 06-Strings-and-Unicode

def main()
    var name = "Alice"
    var age = 30

    # Simple interpolation
    print "Name: ${name}"          # Name: Alice

    # Expressions in interpolation
    print "Age next year: ${age + 1}" # Age next year: 31

    # Method calls
    var lower_name = name.lower()
    print "Lowercase: ${lower_name}" # Lowercase: alice

    # Format specifiers
    var price = 19.99
    print "Price: ${price:.2f}"     # Price: 19.99
```

2.60 String Methods

2.60.1 Searching

```
# file: 06_search.zbr
# teaches: searching in strings
# chapter: 06-Strings-and-Unicode

def main()
  var text = "Hello, World!"

  # Contains
  print text.contains("World")      # true
  print text.contains("xyz")        # false

  # Index – returns -1 when not found
  print text.indexOf("World")       # 7
  print text.indexOf("xyz")         # -1

  # Starts/ends with
  print text.startsWith("Hello")    # true
  print text.endsWith("!")          # true
```

2.60.2 Splitting and Joining

```
# file: 06_split_join.zbr
# teaches: splitting and joining strings
# chapter: 06-Strings-and-Unicode

def main()
  # Split – annotate the local with `: List(str)` to materialise the iterator
```

```
# into an actual list (otherwise it stays as a forward iterator only).
var csv = "apple,banana,cherry"
var fruits: List(str) = csv.split(",")
for fruit in fruits
    var f: str = fruit
    print f

# Join
var items = List(str)()
items.add("one")
items.add("two")
items.add("three")
var result = ", ".join(items)
print result                                # one, two, three
```

2.60.3 Trimming and Padding

```
# file: 06_trim_pad.zbr
# teaches: trimming and padding
# chapter: 06-Strings-and-Unicode

def main()
    var padded = "  hello  "

    # Trim whitespace
    print "|${padded.trim()}|"          # |hello|
    print "|${padded.trimLeft()}|"      # |hello  |
    print "|${padded.trimRight()}|"     # |  hello|

    # Padding
```

```
var short = "hi"

print short.padLeft(10, "*")      # *****hi
print short.padRight(10, "-")    # hi-----
print short.center(10, "*")      # ****hi****
```

2.60.4 Replacing

```
# file: 06_replace.zbr
# teaches: string replacement
# chapter: 06-Strings-and-Unicode

def main()
  var text = "cat and dog and bird"

  # Replace (first occurrence, or all)
  var once = text.replace("and", "or")    # Replaces once
  print once

  var all = text.replaceAll("and", "or")  # Replaces all
  print all

  # Case conversion
  var lower = "Hello World".lower()
  print lower                            # hello world
```

Unicode Representation in Zebra

ASCII (Single Byte per Character)

String: "Hello"

H	e	l	l	o
---	---	---	---	---

Bytes: 48 65 6C 6C 6F (5 bytes)

Code points: U+0048, U+0065, U+006C, U+006C, U+006F

Unicode (Multi-byte for Special Chars)

String: "Hello🌍"

H	e	l	l	o	🌍
---	---	---	---	---	---

Bytes: 48 65 6C 6C 6F F0 9F 8C 8D (9 bytes total)

Emoji is 4 bytes in UTF-8 (multi-byte character)

How Zebra Handles Unicode Strings

.len — Returns byte count (UTF-8 encoded)

```
var str = "Hello🌍"  
print str.len # Outputs: 9 (5 ASCII bytes + 4 UTF-8 bytes for emoji)
```

.charAt(index) — Gets byte at position

```
var str = "Hello"  
print str.charAt(0) # "H"  
print str.charAt(4) # "o"
```

.substring(start, end) — Extract portion

```
var str = "Hello World"  
print str.substring(0, 5) # "Hello"
```

✓ Zebra uses UTF-8 encoding ✓ String operations work with bytes ✓ Use .len for byte count, not character count

Figure 2.4: Unicode Representation

2.61 Unicode and Internationalization

2.61.1 Unicode Basics

```
# file: 06_unicode.zbr
# teaches: unicode support
# chapter: 06-Strings-and-Unicode

def main()
    # Emoji
    var emoji = "Hello 🇧🇪 🇩🇪 🇦🇹"
    print emoji
    print emoji.len                # Byte length
    print emoji.codePointCount()   # Character count

    # Chinese
    var chinese = "你好"           # Hello World in Chinese
    print chinese

    # Arabic (right-to-left)
    var arabic = "مرحبا" "العالم"   # Hello World
    print arabic

    # Mixed scripts
    var mixed = "Hello 🇧🇪 🇩🇪 🇦🇹"
    print mixed
```

2.61.2 Character Iteration

```
# file: 06_char_iter.zbr
# teaches: iterating over characters
```

```
# chapter: 06-Strings-and-Unicode

def main()
    var text = "Hello"

    # Iterate characters (yields `char`; .toString() lifts back to str for printing)
    for c in text.chars()
        print c.toString()

    # Byte iteration – yields each byte as a u8
    var data = "AB"
    for byte in data.bytes()
        print byte                # 65, 66 (ASCII values)
```

`.chars()` and `.bytes()` are different lenses on the same string. `.chars()` walks Unicode codepoints (so "A" yields one element); `.bytes()` walks the raw UTF-8 bytes (so "A" yields four). Use whichever matches what you're counting.

2.62 Regular Expressions (Intro)

Regular expressions let you search and validate text patterns.

2.62.1 Basic Patterns

```
# file: 06_regex_intro.zbr
# teaches: regular expressions introduction
# chapter: 06-Strings-and-Unicode

def main()
```

```
# Simple pattern
var email = "alice@example.com"
var pattern = Regex.compile("[a-z]+@[a-z]+\.[a-z]+")

print pattern.match(email)          # true

# Find matches
var text = "I have 2 apples and 3 oranges"
var digit_pattern = Regex.compile("\\d+")
if digit_pattern.find(text) as found
    print found                      # 2

# Replace
var clean = digit_pattern.replace(text, "X")
print clean                         # I have X apples and X oranges
```

2.63 Real World: Text Processing

```
# file: 06_text_processing.zbr
# teaches: practical text operations
# chapter: 06-Strings-and-Unicode

def parse_csv_line(line: str): List(str)
    var out: List(str) = line.split(",")
    return out

def normalize_whitespace(text: str): str
    var lines = List(str)()
```

```
    for line in text.split("\n")
        var trimmed = line.trim()
        if trimmed.len > 0
            lines.add(trimmed)
    return "\n".join(lines)

def extract_numbers(text: str): List(str)
    var results = List(str)()
    var pattern = Regex.compile("\\d+")
    for match in pattern.findAll(text)
        results.add(match)
    return results

def main()
    # Parse CSV
    var csv_line = "Alice,30,alice@example.com"
    var fields = parse_csv_line(csv_line)
    var name: str = fields.at(0)
    var age: str = fields.at(1)
    print "Name: ${name}"
    print "Age: ${age}"

    # Extract numbers
    var text = "I was born in 1990 and moved in 2005"
    var years = extract_numbers(text)
    for year in years
        var y: str = year
        print y
```

2.64 Common Patterns

2.64.1 Email Validation

```
def is_valid_email(email: str): bool
    if not email.contains("@")
        return false

    var parts: List(str) = email.split("@")
    if parts.count() != 2
        return false

    var domain: str = parts.at(1)
    if not domain.contains(".")
        return false

    return true
```

2.64.2 URL Parsing

```
def parse_url(url: str): HashMap(str, str)
    var result = HashMap(str, str)()
    var parts: List(str) = url.split("://")
    if parts.count() == 2
        var proto: str = parts.at(0)
        result.put("protocol", proto)

    return result
```

2.64.3 String Templating

```
def template(text: str, values: HashMap(str, str)): str
    var result = text
    for key, value in values
        var placeholder = "${${key}}"
```

```
    result = result.replace(placeholder, value)
return result
```

2.65 Common Mistakes

Mistake: Forgetting that strings are immutable

```
var text = "hello"
text[0] = 'H' # ❌ Can't modify
```

Better:

```
var text = "hello"
var capitalized = "H".concat(text[1..])
```

Mistake: Ignoring Unicode length

```
var emoji = "👉"
print emoji.len # ❌ Returns 4 (bytes), not 1
```

Better:

```
var emoji = "👉"
print emoji.codePointCount() # ❌ Returns 1 (characters)
```

Mistake: Inefficient concatenation in loops

```
var result = ""
for i in 1.to(1000)
    result = result + "${i}," # ❌ O(n2) complexity
```

Better:

```
var sb = StringBuilder()
for i in 1.to(1000)
```

```
sb.append("${i},")
var result = sb.build() # O(n) complexity
```

2.66 Exercises

2.66.1 Exercise 1: String Reversal

Write a function that reverses a string:

Solution

```
def reverse_str(text: str): str
    var chars = List(str)()
    for c in text.chars()
        chars.add(c.toString())

    var result = ""
    var i = chars.count() - 1
    while i >= 0
        result = result.concat(chars.at(i))
        i = i - 1
    return result

def main()
    var reversed = reverse_str("hello")
    print reversed # olleh
```

Zebra also has a built-in `text.reverse()` for the common case; the loop above is just to show character iteration.

2.66.2 Exercise 2: Email Validator

Write a simple email validator:

Solution

```
def is_valid_email(email: str): bool
    if email.len < 5
        return false
    if not email.contains("@")
        return false
    if not email.contains(".")
        return false
    var parts: List(str) = email.split("@")
    if parts.count() != 2
        return false
    var domain: str = parts.at(1)
    if domain.len < 3
        return false
    return true

def main()
    var emails = List(str)()
    emails.add("alice@example.com")
    emails.add("invalid")
    emails.add("bob@domain.co")

    for email in emails
        var e: str = email
        if is_valid_email(e)
            print "Valid: ${e}"
        else
```

```
print "Invalid: ${e}"
```

2.66.3 Exercise 3: CSV Parsing

Parse a CSV line and extract fields:

Solution

```
def parse_csv(line: str): List(str)
  var out: List(str) = line.split(",")
  return out

def parse_with_trim(line: str): List(str)
  var trimmed = List(str)()
  for field in line.split(",")
    trimmed.add(field.trim())
  return trimmed

def main()
  var csv = "Alice, 30, NYC"
  var fields = parse_with_trim(csv)
  for field in fields
    var f: str = field
    print "|${f}|"
```

2.67 Next Steps

- → **07-Classes** — Object-oriented programming
- → **18-Regular-Expressions** — Deep dive into regex
- **Project-1-CLI-Tool** — Text processing practical application

2.68 Key Takeaways

- **Strings are immutable** — Create new ones instead of modifying
 - **Interpolation is readable** — Use "`${var}`" over concatenation
 - **Methods are rich** — `.split()`, `.replace()`, `.trim()`, etc.
 - **Unicode works seamlessly** — emoji, Chinese, Arabic, etc.
 - **Regex enables pattern matching** — For validation and extraction
 - **StringBuilder is efficient** — Use for loop-based concatenation
-

Next: Head to **Part 2** for object-oriented programming with **07-Classes**.

Chapter 3

Part 2: Objects and Interfaces

07: Classes and Instances

Audience: All (with beginner sidebar)

Time: 120 minutes

Prerequisites: 01-06

You'll learn: Define classes, instantiate objects, instance methods, fields, initialization

3.1 The Big Picture

Classes are blueprints for objects. Instead of a loose collection of variables, a class groups related data and behavior together.

Real world: A car has properties (color, speed) and behaviors (accelerate, brake)

Code: A Car class has fields (color, speed) and methods (accelerate, brake)

Class Structure in Zebra

BankAccount Class Layout:

class BankAccount
Instance Fields: var account_id as int var owner as str var balance as float var is_open as bool = true
Instance Methods: def deposit(amount as float) def withdraw(amount as float) def check_balance as float def close_account()
Shared (Static) Members: shared var next_account_id = 1001 shared def create(owner as str) var account = BankAccount() account.account_id = next_id next_account_id = next_id + 1 return account Shared across all instances

Creating and Using Instances:

```

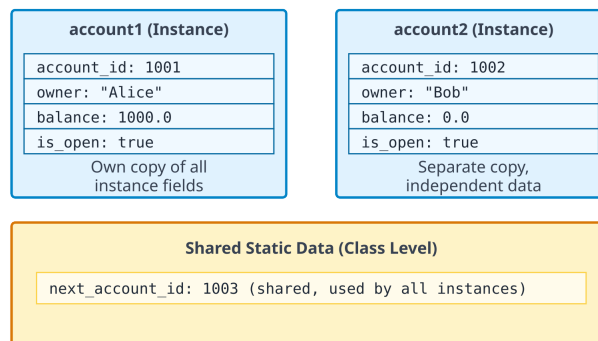
var account1 = BankAccount.create("Alice")
# account1 instance created
# Fields: account_id=1001, owner="Alice",
# balance=0, is_open=true

account1.deposit(1000) # Call instance method
var bal = account1.check_balance() # bal = 1000

var account2 = BankAccount.create("Bob")
# account2 separate instance, different fields

```

Memory Layout:



Instance fields: one per object • Shared fields: one for entire class

Figure 3.1: Class Structure

3.2 Defining Classes

3.2.1 Simple Class

```
# file: 07_class_basic.zbr
# teaches: class definition, cue init
# chapter: 07-Classes-and-Instances

class Person
    var name: str
    var age: int

    cue init(name: str, age: int)
        this.name = name
        this.age = age

    def greet()
        print "Hi, I'm ${this.name}"

def main()
    var person = Person("Alice", 30)
    person.greet() # Hi, I'm Alice
```

Breakdown: - `class Person` — Define a class named Person. - `var name: str` — Field (property) of the class. - `cue init(name, age)` — Constructor. `this.name = name` initialises the field from the argument of the same name (using `this.` to disambiguate). - `def greet()` — Method of the class. - `Person("Alice", 30)` — Create an instance, passing constructor args.

The class fields don't need defaults when `cue init` always sets them. The “field default” form (`var name: str = ""`) shines when you want to allow construction without specifying every field — see “Fields and Initialization” below.

3.2.2 Fields and Initialization

```
# file: 07_init.zbr
# teaches: field defaults vs. explicit init
# chapter: 07-Classes-and-Instances

class Rectangle
    var width: int = 0      # Defaults – class is constructible with `Rectangle()`
    var height: int = 0

    def area(): int
        return this.width * this.height

def main()
    var rect = Rectangle()
    rect.width = 10
    rect.height = 5
    print rect.area()  # 50
```

Field defaults make a class **bare-constructible** (`Rectangle()` works) and let callers fill in only the fields they care about. Adding cue `init(width: int, height: int)` would make construction strict and let you drop the defaults — both styles compose.

3.2.3 Instance Methods

```
# file: 07_methods.zbr
# teaches: instance methods
# chapter: 07-Classes-and-Instances

class Counter
```

```
var count: int = 0

def increment()
    this.count = this.count + 1

def decrement()
    this.count = this.count - 1

def reset()
    this.count = 0

def get_count(): int
    return this.count

def main()
    var counter = Counter()
    counter.increment()
    counter.increment()
    counter.increment()
    print counter.get_count() # 3
    counter.reset()
    print counter.get_count() # 0
```

3.2.4 Static Methods (Class Methods)

```
# file: 07_static.zbr
# teaches: static (class-level) methods
# chapter: 07-Classes-and-Instances

class MathUtil
```

```
static

    def abs(x: int): int
        if x < 0
            return 0 - x
        return x

    def max_of(a: int, b: int): int
        if a > b
            return a
        return b

def main()
    print MathUtil.abs(-5)      # 5
    print MathUtil.max_of(10, 20) # 20
```

When the methods are pure and don't share state, you have a choice: group them on a class for namespacing (as above) or write them as plain top-level **defs** in a module. Pick the form that reads better at the call site. Top-level **def** keeps the call as `abs(-5)`; the class form makes `MathUtil.abs(-5)` self-documenting at a distance.

3.2.5 If you're new to programming

A **class** is like a template. When you create an instance (with `Person()`), you're making a specific copy from that template.

Fields are the properties (like `name`, `age`)

Methods are the behaviors (like `greet()`)

Instance methods work on a specific object (`person.greet()`)

Static methods belong to the class itself (`MathUtil.abs()`)

3.3 Real World: User Management

```
# file: 07_user_system.zbr
# teaches: realistic class design
# chapter: 07-Classes-and-Instances

class User
  var username: str = ""
  var email: str = ""
  var created_at: str = ""
  var is_active: bool = true

  def is_valid(): bool
    return this.username.len > 0 and this.email.contains("@")

  def deactivate()
    this.is_active = false

  def display_profile()
    print "User: ${this.username}"
    print "Email: ${this.email}"
    print "Active: ${this.is_active}"

class UserManager
  static
    var users: List(User) = List(User)()

  def add_user(user: User): bool
```

```
        if not user.is_valid()
            return false

        UserManager.users.add(user)

        return true

def find_user(username: str): User?
    for user in UserManager.users
        if user.username == username
            return user
    return nil

def user_count(): int
    return UserManager.users.count()

def main()
    var user1 = User()
    user1.username = "alice"
    user1.email = "alice@example.com"

    if UserManager.add_user(user1)
        print "User added"

    if UserManager.find_user("alice") as found
        found.display_profile()
```

3.4 Common Patterns

3.4.1 Value Object

```
class Point
  var x: Int = 0
  var y: Int = 0

  def distance_from_origin(): Float
    return 0.0 # sqrt(x*x + y*y) once you have a sqrt to call
```

3.4.2 Service Class

```
class EmailService
  static
    def send(to: String, subject: String, body: String): Boolean
      # Implementation
      return true
```

3.4.3 Builder Pattern

```
class UserBuilder
  var username: String = ""
  var email: String = ""
  var age: Int = 0

  def with_username(name: String)
    this.username = name

  def with_email(addr: String)
    this.email = addr
```

```
def build(): User
    var user = User()
    user.username = this.username
    user.email = this.email
    return user
```

3.5 Common Mistakes

Mistake: Forgetting to initialize fields

```
class Person
    var name: str # No default value
var p = Person()
print p.name # ? Uninitialized!
```

Better:

```
class Person
    var name: str = "" # Has default
```

Mistake: Modifying shared fields unintentionally

```
class Counter
    static var count: int = 0

    def reset
        count = 0 # Affects ALL instances!
```

Better:

```
class Counter
    var count: int = 0 # Instance field
```

```
def reset
  count = 0 # Only this instance
```

3.6 Exercises

3.6.1 Exercise 1: Bank Account

Create a BankAccount class with deposit and withdraw methods:

Solution

```
class BankAccount
  var balance: float = 0.0
  var account_number: str = ""

  def deposit(amount: float)
    this.balance = this.balance + amount

  def withdraw(amount: float): bool
    if amount > this.balance
      return false
    this.balance = this.balance - amount
    return true

  def get_balance(): float
    return this.balance

def main()
  var account = BankAccount()
```

```
account.account_number = "1234567890"
account.deposit(1000.0)
print "Balance: ${account.get_balance()}"
account.withdraw(100.0)
print "Balance: ${account.get_balance()}"
```

3.6.2 Exercise 2: Product Catalog

Create a Product class and a simple store:

Solution

```
class Product
    var name: str = ""
    var price: float = 0.0
    var quantity: int = 0

    def total_value(): float
        return this.price * @as(f64, @floatFromInt(this.quantity))

class Store
    var products: List(Product) = List(Product)()

    def add_product(product: Product)
        this.products.add(product)

    def total_inventory_value(): float
        var total = 0.0
        for product in this.products
            total = total + product.total_value()
        return total
```

```
def main()
    var store = Store()

    var apple = Product()
    apple.name = "Apple"
    apple.price = 0.50
    apple.quantity = 100
    store.add_product(apple)

    var orange = Product()
    orange.name = "Orange"
    orange.price = 0.75
    orange.quantity = 80
    store.add_product(orange)

    print "Total value: ${store.total_inventory_value()}"
```

The `@as(f64, @floatFromInt(this.quantity))` is a temporary float-from-int cast — Zebra’s `int * float` doesn’t auto-promote (mixed-type arithmetic is rejected, by design). Once a typed `float(int)` builtin lands, that escape hatch becomes a clean conversion.

3.7 Next Steps

- → **08-Interfaces** — Contracts for classes
 - → **09-Inheritance** — Class hierarchies
 - **Project-2-HTTP-Server** — Use classes extensively
-

3.8 Key Takeaways

- **Classes group data and behavior** — Fields + methods
 - **Instances are copies** — Each `Person()` is independent
 - **Instance methods** work on specific objects
 - **Shared methods** belong to the class, not instances
 - **Initialize fields** — Don't leave them undefined
 - **Methods should have clear names** — `deposit()` not `d()`
-

Next: Head to **08-Interfaces** to define contracts between classes.

08: Interfaces and Protocols

Audience: All

Time: 120 minutes

Prerequisites: 07-Classes-and-Instances

You'll learn: Define interfaces, implement contracts, polymorphism, real-world patterns

3.9 The Big Picture

Interfaces define contracts—promises about what an object can do, without specifying how. Instead of depending on concrete classes, you depend on interfaces. This enables: - **Swappable implementations** — Change how something works without changing callers - **Polymorphism** — Treat different objects the same way - **Testing** — Mock implementations easily - **Flexibility** — Add new implementations without modifying existing code

3.10 Defining Interfaces

```
# file: 08_interface_basic.zbr
# teaches: interface definition
# chapter: 08-Interfaces-and-Protocols

interface Animal
    def speak(): str
    def move()

class Dog
    implements Animal
        def speak(): str
            return "Woof!"

        def move()
            print "Running on four legs"

class Bird
    implements Animal
        def speak(): str
            return "Tweet!"

        def move()
            print "Flying through the air"

def main()
    var dog: Animal = Dog()
```

```
print dog.speak()      # Woof!
dog.move()             # Running on four legs

var bird: Animal = Bird()
print bird.speak()     # Tweet!
bird.move()            # Flying through the air
```

Key points: - interface `Animal` — Define what any animal must do - implements `Animal` — Promise to implement all methods - Treat objects as their interface type (`var dog: Animal`)

3.10.2 Multiple Methods

```
# file: 08_interface_methods.zbr
# teaches: interface with multiple methods
# chapter: 08-Interfaces-and-Protocols

interface PaymentProcessor
  def process(amount: float): bool
  def refund(transaction_id: str): bool
  def get_status(transaction_id: str): str

class CreditCardProcessor
  implements PaymentProcessor
    def process(amount: float): bool
      print "Processing credit card: ${amount}"
      return true

    def refund(transaction_id: str): bool
      print "Refunding transaction: ${transaction_id}"
      return true
```

```
def get_status(transaction_id: str): str
    return "completed"

class PayPalProcessor
    implements PaymentProcessor

    def process(amount: float): bool
        print "Processing PayPal: ${amount}"
        return true

    def refund(transaction_id: str): bool
        print "PayPal refund: ${transaction_id}"
        return true

    def get_status(transaction_id: str): str
        return "pending"
```

3.11 Polymorphism in Action

3.11.1 Using Different Implementations

```
# file: 08_polymorphism.zbr
# teaches: polymorphic behavior
# chapter: 08-Interfaces-and-Protocols

def process_payment(processor: PaymentProcessor, amount: float)
    if processor.process(amount)
        print "Payment successful"
    else
```

```
        print "Payment failed"

def main()
    var cc_processor = CreditCardProcessor()
    var paypal_processor = PayPalProcessor()

    # Same code, different behavior
    process_payment(cc_processor, 99.99)
    process_payment(paypal_processor, 49.99)
```

3.11.2 Collections of Interface Types

```
# file: 08_collection_interface.zbr
# teaches: storing different implementations
# chapter: 08-Interfaces-and-Protocols

class Zoo
    var animals: List(Animal) = List(Animal())

    def add_animal(animal: Animal)
        this.animals.add(animal)

    def make_them_speak()
        for animal in this.animals
            print animal.speak()

    def exercise_all()
        for animal in this.animals
            animal.move()
```

```
def main()
    var zoo = Zoo()
    zoo.add_animal(Dog())
    zoo.add_animal(Bird())
    zoo.add_animal(Dog())

    zoo.make_them_speak()
    zoo.exercise_all()
```

3.12 Real World: Logging System

```
# file: 08_logger_system.zbr
# teaches: realistic interface use
# chapter: 08-Interfaces-and-Protocols

interface Logger
    def debug(message: str)
    def info(message: str)
    def warn(message: str)
    def error(message: str)

class ConsoleLogger
    implements Logger
    def debug(message: str)
        print "[DEBUG] ${message}"

    def info(message: str)
        print "[INFO] ${message}"
```

```
def warn(message: str)
  print "[WARN] ${message}"

def error(message: str)
  print "[ERROR] ${message}"

class FileLogger
  implements Logger
  def debug(message: str)
    # Write to file: [DEBUG] message

  def info(message: str)
    # Write to file: [INFO] message

  def warn(message: str)
    # Write to file: [WARN] message

  def error(message: str)
    # Write to file: [ERROR] message

class Application
  var logger: Logger

  def set_logger(l: Logger)
    this.logger = l

  def do_work()
    this.logger.info("Starting work")
    # Do work
```

```
        this.logger.info("Work complete")

def main()
    var app = Application()

    # Use console logger
    app.set_logger(ConsoleLogger())
    app.do_work()

    # Switch to file logger (same code, different output)
    app.set_logger(FileLogger())
    app.do_work()
```

3.13 Common Patterns

3.13.1 Strategy Pattern

```
interface SortStrategy
    def sort(items: List(int))

class AscendingSort
    implements SortStrategy
    def sort(items: List(int))
        # Sort ascending

class DescendingSort
    implements SortStrategy
    def sort(items: List(int))
```

```
        # Sort descending

class Sorter
    var strategy: SortStrategy

    def set_strategy(s: SortStrategy)
        this.strategy = s

    def sort(items: List(int))
        this.strategy.sort(items)
```

3.13.2 Adapter Pattern

```
interface NewSystem
    def process(data: str)

class OldSystem
    def old_process(input: str)
        # Old implementation
        pass

class OldSystemAdapter
    implements NewSystem
        var old_system: OldSystem = OldSystem()

    def process(data: str)
        # Adapt new interface to old system
        this.old_system.old_process(data)
```

3.14 If you're new to programming

An **interface** is like a promise or contract. When you say a class “implements” an interface, you promise that it has all the methods the interface requires.

This lets you write code that works with **any** object that implements that interface, without knowing exactly which class it is.

Polymorphism means “many forms”—the same code can work with different types of objects.

3.14.1 If you know Python

```
# Python (using duck typing)
class Dog:
    def speak(self):
        return "Woof!"

class Bird:
    def speak(self):
        return "Tweet!"

def get_sound(animal):
    return animal.speak()
```

```
# Zebra (using interfaces)
interface Animal
    def speak(): str

class Dog implements Animal
    def speak(): str
        return "Woof!"
```

```
class Bird implements Animal
    def speak(): str
        return "Tweet!"

def get_sound(animal: Animal): str
    return animal.speak()
```

Python relies on duck typing (“if it quacks like a duck”). Zebra makes the contract explicit with interfaces.

3.15 Common Mistakes

Mistake: Not implementing all interface methods

```
interface Animal
    def speak(): str
    def move()

class Dog
    implements Animal
        def speak(): str
            return "Woof!"
        # ❌ Missing: def move
```

Why: The compiler requires all methods. You’re breaking the contract.

Better:

```
class Dog
    implements Animal
        def speak(): str
```

```
        return "Woof!"

    def move()

        print "Running"
```

Mistake: Wrong method signature

```
interface PaymentProcessor

    def process(amount: float): bool

class CreditCard

    implements PaymentProcessor

    def process(amount: int): bool # ? int, not float

    return true
```

Better:

```
class CreditCard

    implements PaymentProcessor

    def process(amount: float): bool # ? Matches interface

    return true
```

Mistake: Forgetting to declare implementation

```
class Dog # ? Doesn't say implements Animal

    def speak(): str

    return "Woof!"
```

Better:

```
class Dog

    implements Animal # ? Explicit contract

    def speak(): str

    return "Woof!"
```

3.16 Exercises

3.16.1 Exercise 1: Shape Interface

Create an interface for shapes and multiple implementations:

Solution

```
interface Shape
    def area(): float
    def perimeter(): float

class Circle
    var radius: float = 0.0
    implements Shape
    def area(): float
        return 3.14 * this.radius * this.radius
    def perimeter(): float
        return 2.0 * 3.14 * this.radius

class Rectangle
    var width: float = 0.0
    var height: float = 0.0
    implements Shape
    def area(): float
        return this.width * this.height
    def perimeter(): float
        return 2.0 * (this.width + this.height)

def print_shape_info(shape: Shape)
    print "Area: ${shape.area()}"
    print "Perimeter: ${shape.perimeter()}"
```

```
def main()
  var circle = Circle()
  circle.radius = 5.0
  print_shape_info(circle)

  var rect = Rectangle()
  rect.width = 10.0
  rect.height = 5.0
  print_shape_info(rect)
```

3.16.2 Exercise 2: Database Interface

Create a database interface with multiple implementations:

Solution

```
interface Database
  def save(key: str, value: str): bool
  def load(key: str): str?
  def delete(key: str): bool

class MemoryDatabase
  var data: HashMap(str, str) = HashMap(str, str)()
  implements Database
    def save(key: str, value: str): bool
      this.data.put(key, value)
      return true
    def load(key: str): str?
      return this.data.get(key)
    def delete(key: str): bool
      this.data.remove(key)
```

```
        return true

def main()
    var db: Database = MemoryDatabase()

    db.save("user1", "Alice")
    db.save("user2", "Bob")

    if db.load("user1") as user
        print "Found: ${user}"

    db.delete("user1")
```

3.16.3 Exercise 3: Document Processor

Create an interface for document processors:

Solution

```
interface DocumentProcessor
    def process(content: str): str
    def validate(content: str): bool

class MarkdownProcessor
    implements DocumentProcessor
        def process(content: str): str
            # Convert markdown to HTML
            return "<html>${content}</html>"
        def validate(content: str): bool
            return content.len > 0

class JSONValidator
```

```
implements DocumentProcessor

  def process(content: str): str
    return content # Already valid JSON

  def validate(content: str): bool
    return content.contains("{") and content.contains("}")

def process_document(processor: DocumentProcessor, doc: str)
  if processor.validate(doc)
    var processed = processor.process(doc)
    print "Processed: ${processed}"
  else
    print "Invalid document"

def main()
  var md = MarkdownProcessor()
  process_document(md, "# Hello")

  var json = JSONValidator()
  process_document(json, "{}")
```

3.17 Next Steps

- → **09-Composition-and-Mixins** — Sharing behaviour without inheritance
 - → **14-Contracts** — More complex interfaces
 - **Project-2-HTTP-Server** — Interfaces for handlers and middleware
-

3.18 Key Takeaways

- **Interfaces define contracts** — What methods must exist, not how they work
- **Polymorphism** lets you write code once, work with many types
- **Swap implementations** without changing callers
- **Testing is easier** — Mock implementations implement the interface
- **Makes code flexible** — Add new implementations without modifying existing code
- **Forces clear design** — Interfaces make relationships explicit

Next: Head to **09-Composition-and-Mixins** for code reuse without classical inheritance.

Chapter 4

09: Composition and Mixins

Audience: All **Time:** 90 minutes **Prerequisites:** 07-Classes, 08-Interfaces **You'll learn:** Why Zebra rejects class inheritance, how to share behaviour with **mixins**, and how to combine smaller classes via **composition**

4.1 Why no inheritance?

Zebra deliberately omits class inheritance. There is no **extends**, no **super**, no parent class.

Decades of inheritance experience surfaced known problems: the fragile base-class problem (a change in the parent silently breaks subclasses), diamond inheritance ambiguity, surprising method-resolution order, and tight coupling between base and derived classes. Languages like Go and Rust have shown that large systems can be built without ever needing inheritance.

Zebra leans on **three reuse primitives**, each with a clear role:

Primitive	Purpose	Keyword
Interface	Polymorphism — “X plays the role of Drawable”	implements (Chapter 08)

Primitive	Purpose	Keyword
Mixin	Behaviour reuse — “X has the methods of Logger”	adds
Composition	“X has-a Logger”	normal field

This chapter covers **mixins** (**adds**) and **composition** (field-holding). Interfaces were covered in Chapter 08.

4.2 Mixins — sharing methods across classes

A **mixin** is a named bag of methods that a class can pull in. Declare it with **mixin**:

```
# file: 09_mixin_basic.zbr
# teaches: declaring and using mixins
# chapter: 09-Composition-and-Mixins

mixin Loggable
  def log(message: str)
    print "[log] ${message}"

mixin Cacheable
  var _cache: HashMap(str, str)

  cue init
    _cache = HashMap(str, str)()

  def cache_get(key: str): str?
    return _cache.get(key)
```

```
def cache_set(key: str, value: str)
  _cache.set(key, value)
```

Then use **adds** on the class:

```
class UserService adds Loggable, Cacheable
  cue init
    _cache = HashMap(str, str)()

  def lookup(id: str): str
    var hit = cache_get(id)
    if hit as found
      return found
    log("cache miss for ${id}")
    var fetched = "user-${id}"
    cache_set(id, fetched)
    return fetched

def main()
  var svc = UserService()
  print svc.lookup("42")    # logs "cache miss for 42", returns "user-42"
  print svc.lookup("42")    # silent cache hit, returns "user-42"
```

Key points:

- A mixin is declared with **mixin**, not **class**. It can't be instantiated on its own.
- **class C adds M1, M2, M3** includes M1's, M2's, and M3's methods (and fields) into C as if they were declared on C directly.
- All mixin methods are accessible on the class without any dotted prefix. **svc.log(...)** and **svc.cache_get(...)** look identical to native methods.
- No method-resolution magic: if two mixins define a method with the same name, that's a compile error — pick one or rename one.

4.3 Composition — has-a, not is-a

When a class needs the *services* of another class but isn't conceptually the same kind of thing, hold an instance as a field:

```
# file: 09_composition.zbr
# teaches: composition with helper classes
# chapter: 09-Composition-and-Mixins

class FileWriter
  var path: str = ""

  cue init(p: str)
    path = p

  def write(content: str)
    File.write(path, content)

class AuditedReport
  var writer: FileWriter
  var entries: List(str)

  cue init(p: str)
    this.writer = FileWriter(p)
    this.entries = List(str)()

  def add_entry(line: str)
    this.entries.add(line)
```

```
def publish()
    var content = this.entries.join("\n")
    this.writer.write(content)

def main()
    var r = AuditedReport("/tmp/audit.log")
    r.add_entry("user logged in")
    r.add_entry("user updated profile")
    r.publish()
```

`AuditedReport` *uses* `FileWriter`; it isn't a `FileWriter`. The same idea inheritance-heavy languages call “favouring composition over inheritance.” In Zebra it's just the natural pattern.

4.4 Mixin vs. composition — when to use each

Use a mixin when...	Use composition when...
The shared behaviour is part of the class's identity (“X is also Loggable”)	The class <i>uses</i> a helper but isn't the helper (“X has a <code>FileWriter</code> ”)
The mixin's state should be flat with the class's state	You want the helper to have its own lifecycle
The behaviour doesn't need its own configuration or replaceable identity	You want the helper independently testable, mockable, or swappable

A reasonable default order: reach for **interfaces** first (when you only need a contract), **composition** second (when you need a helper), **mixins** last (when the same exact methods need to be reused verbatim across multiple classes).

4.5 Multiple mixins — building richer classes

A class can combine several mixins:

```
# file: 09_multiple_mixins.zbr
# teaches: combining multiple mixins
# chapter: 09-Composition-and-Mixins

mixin Timestamped
  var created_at_ms: int = 0

  def stamp_now()
    this.created_at_ms = DateTime.now().epochMs

mixin Tagged
  var tags: List(str)

  cue init()
    this.tags = List(str)()

  def tag(t: str)
    this.tags.add(t)

class BlogPost adds Timestamped, Tagged
  var title: str = ""
  var body: str = ""

  cue init(t: str, b: str)
    this.title = t
    this.body = b
    this.tags = List(str)()
```

```
        this.stamp_now()

def main()
    var p = BlogPost("Hello", "First post body")
    p.tag("intro")
    p.tag("zebra")
    print "${p.title} - ${p.tags.count()} tags @ ${p.created_at_ms} ms"
```

The class picks up `created_at_ms`, `tags`, `stamp_now()`, and `tag(...)` as if they were declared directly on `BlogPost`.

4.6 Polymorphism — reach for interfaces (Chapter 08)

When you want a function that works on “anything that can do X,” use an interface — not a hierarchy:

```
# file: 09_polymorphism.zbr
# teaches: interface-based polymorphism (instead of inheritance)
# chapter: 09-Composition-and-Mixins

interface Sounding
    def sound(): str

class Dog implements Sounding
    var name: str = ""

    cue init(n: str)
        this.name = n

    def sound(): str
```

```
        return "${this.name}: Woof!"

class Cat implements Sounding
    var name: str = ""

    cue init(n: str)
        this.name = n

    def sound(): str
        return "${this.name}: Meow!"

def main()
    var animals: List(Sounding) = List(Sounding())
    animals.add(Dog("Rex"))
    animals.add(Cat("Whiskers"))

    for a in animals
        print a.sound()
```

Both `Dog` and `Cat` implements `Sounding`. Code that wants any sounding thing accepts a `Sounding`; the underlying class can be anything. This is the polymorphism inheritance offers — without coupling `Dog` or `Cat` to a shared base implementation.

4.7 Real world: a document with capabilities

A realistic example combining interface + mixin + field composition:

```
# file: 09_document_capabilities.zbr
# teaches: combining interface + mixin + composition
# chapter: 09-Composition-and-Mixins
```

```
interface Publishable
    def publish()

mixin Versioned
    var version: int = 0

    def bump()
        this.version = this.version + 1

class Storage
    var path: str = ""

    cue init(p: str)
        path = p

    def write(content: str)
        File.write(path, content)

class BlogPost implements Publishable adds Versioned
    var title: str = ""
    var body: str = ""
    var storage: Storage

    cue init(t: str, store_path: str)
        title = t
        storage = Storage(store_path)

    def publish()
        this.bump()                                # mixin method
```

```
var content = "${this.title} v${this.version}\n\n${this.body}"
this.storage.write(content)           # composition

def main()
  var post = BlogPost("Hello", "/tmp/hello.txt")
  post.body = "First post"
  post.publish()                       # writes v1
  post.body = "First post (revised)"
  post.publish()                       # writes v2
```

BlogPost plays the role of a **Publishable** (interface), gains version-tracking (mixin), and uses a **Storage** helper (field). Three orthogonal capabilities, none of them coupled into a fragile inheritance chain.

4.8 If you’re new to programming

Inheritance is a popular but easy-to-misuse feature. The mental shift Zebra asks of you is: instead of “Dog **is a** kind of Animal,” ask “what does Dog need to **do**?” and pull those abilities in piece by piece.

A useful default: start with no mixins. When two classes share a contract, declare an **interface** and have both **implements** it. When one class needs another class’s services, hold an instance as a field. When two classes need exactly the same methods, factor those methods into a **mixin** and have both **adds** it.

4.9 Common mistakes

Reaching for class hierarchies you can’t have

```
class Dog inherits Animal      # ❌ `inherits` doesn't exist in Zebra
  def bark
    print "${name} says: Woof!"
```

Use a mixin or an interface:

```
mixin Named
  var name: str = ""

class Dog adds Named
  def bark
    print "${name} says: Woof!"
```

Calling **super** to chain into a “parent”

```
def accelerate
  super.accelerate()          # ❌ no `super` in Zebra
```

Express the shared behaviour as a method on a mixin and call it directly:

```
mixin Acceleratable
  var speed: int = 0

  def base_accelerate
    speed = speed + 10

class Car adds Acceleratable
  def accelerate
    base_accelerate()
    print "Car cruising at ${speed} mph"
```

Two mixins declaring the same method name

```
mixin A
  def hello
    print "A"

mixin B
  def hello
    print "B"

class C adds A, B                # ❗ ambiguous: which hello()?
```

Rename one or pick the canonical mixin:

```
mixin A
  def hello_a
    print "A"

mixin B
  def hello_b
    print "B"

class C adds A, B
```

Modelling a “kind-of” relationship with a mixin

A **mixin** is for shared *methods*, not for “is-a” modelling. If you find yourself reaching for inheritance because Dog is a kind of Animal, the right answer is usually an **interface** for the shared contract, with each concrete class implementing it independently.

4.10 Exercises

4.10.1 Exercise 1: Vehicle behaviours

Build three vehicle types (`Car`, `Motorcycle`, `Truck`) that share starting/stopping behaviour via a mixin, but each has a distinct `describe()` method.

Solution

```
mixin Engine
    var running: bool = false

    def start
        running = true
        print "Engine started"

    def stop
        running = false
        print "Engine stopped"

class Car adds Engine
    var brand: str = ""
    var doors: int = 4

    cue init(b: str)
        brand = b

    def describe: str
        return "${brand} car with ${doors} doors"

class Motorcycle adds Engine
    var brand: str = ""
```

```
    cue init(b: str)
        brand = b

    def describe: str
        return "${brand} motorcycle"

class Truck adds Engine
    var brand: str = ""
    var payload_kg: int = 0

    cue init(b: str, p: int)
        brand = b
        payload_kg = p

    def describe: str
        return "${brand} truck (${payload_kg}kg payload)"

def main()
    var car = Car("Toyota")
    car.start()
    print car.describe()
    car.stop()
```

4.10.2 Exercise 2: Employees and managers — without inheritance

Model **Employee**, **Manager**, and **Director** without any “kind-of” relationship. Use a mixin for shared salary/name behaviour, and composition for the manager’s reporting structure.

Solution

```
mixin Compensated
  var name: str = ""
  var salary: float = 0.0

  def info: str
    return "${name} (${salary})"

class Employee adds Compensated
  cue init(n: str, s: float)
    name = n
    salary = s

class Manager adds Compensated
  var team: List(Employee)

  cue init(n: str, s: float)
    name = n
    salary = s
    team = List(Employee)()

  def add_report(e: Employee)
    team.add(e)

  def info: str
    return "${name} (${salary}, manages ${team.count()})"

class Director adds Compensated
  var managers: List(Manager)
  var budget: float = 0.0
```

```
cue init(n: str, s: float, b: float)

    name = n
    salary = s
    budget = b
    managers = List(Manager)()

def info: str
    return "${name} (${salary}, budget ${budget})"
```

`Manager` and `Director` aren't subclasses of `Employee` — they each adds `Compensated` independently. Where a manager needs to know its reports, it holds them by composition (`team: List(Employee)`).

4.10.3 Exercise 3: Shapes implementing a common interface

Use the `Shape` interface from Chapter 08 with three independent classes — no shared base, no inheritance.

Solution

```
interface Shape

    def area: float
    def perimeter: float

class Circle implements Shape
    var radius: float = 0.0

    cue init(r: float)
        radius = r

    def area: float
        return 3.14 * radius * radius
```

```
def perimeter: float
    return 2.0 * 3.14 * radius

class Rectangle implements Shape
    var width: float = 0.0
    var height: float = 0.0

    cue init(w: float, h: float)
        width = w
        height = h

    def area: float
        return width * height

    def perimeter: float
        return 2.0 * (width + height)

class Triangle implements Shape
    var base: float = 0.0
    var height: float = 0.0
    var hypotenuse: float = 0.0

    cue init(b: float, h: float, hy: float)
        base = b
        height = h
        hypotenuse = hy

    def area: float
        return 0.5 * base * height
```

```
    def perimeter: float
        return base + height + hypotenuse

def main()
    var shapes: List(Shape) = List(Shape())
    shapes.add(Circle(5.0))
    shapes.add(Rectangle(4.0, 6.0))
    shapes.add(Triangle(3.0, 4.0, 5.0))

    for s in shapes
        print "area=${s.area()} perim=${s.perimeter()}"
```

4.11 Next Steps

- → **10-Properties** — Field access patterns and getter idioms
- → **14-Contracts** — Enforce invariants across composed types
- **Project-1-CLI-Tool** — Use mixins for cross-cutting concerns (logging, configuration)

4.12 Key Takeaways

- **Zebra has no inheritance.** No `extends`, no `super`, no parent class. This is by design.
- **Three reuse primitives:** interfaces (`implements` — polymorphism), mixins (`adds` — behaviour reuse), composition (fields — has-a).
- **Reach for interfaces first**, composition second, mixins last.
- **Mixins are flat** — their methods become part of the class as if declared directly.

- **Conflicting mixin methods are a compile error** — no method-resolution magic, no diamond ambiguity.

Next: Head to **10-Properties** for getter idioms and computed-value patterns.

Chapter 5

10: Properties and Computed Values

Audience: All

Time: 90 minutes

Prerequisites: 07-Classes

You'll learn: Getters, setters, computed properties, lazy initialization, encapsulation

5.1 The Big Picture

Properties let you control how fields are accessed and modified. Instead of letting code directly access `person.age`, you can: - **Validate** on assignment (no negative ages) - **Calculate** on access (compute age from birth year) - **Cache** computed values - **Log** access for debugging

5.2 Getters

5.2.1 Simple Getter

```
# file: 10_getter.zbr
# teaches: computed properties
# chapter: 10-Properties-and-Computed-Values

class Person
    var birth_year: int = 2000

    def age: int
        return 2024 - birth_year

    def name_length: int
        var name = "Alice"
        return name.len

def main()
    var person = Person()
    person.birth_year = 1990
    print person.age()          # 34
    print person.name_length() # 5
```

5.2.2 Derived Properties

```
# file: 10_derived.zbr
# teaches: deriving values from fields
# chapter: 10-Properties-and-Computed-Values

class Rectangle
```

```
var width: int = 0
var height: int = 0

def area: int
    return width * height

def perimeter: int
    return 2 * (width + height)

def is_square: bool
    return width == height

def main()
    var rect = Rectangle()
    rect.width = 10
    rect.height = 10

    print "Area: ${rect.area()}"          # 100
    print "Perimeter: ${rect.perimeter()}" # 40
    print "Square: ${rect.is_square()}"   # true
```

5.3 Setters (Validation)

5.3.1 Controlling Field Assignment

```
# file: 10_setter_validation.zbr
# teaches: setters with validation
# chapter: 10-Properties-and-Computed-Values
```

```
class Account
    var balance: float = 0.0

    def deposit(amount: float): bool
        if amount <= 0.0
            return false
        balance = balance + amount
        return true

    def withdraw(amount: float): bool
        if amount <= 0.0
            return false
        if amount > balance
            return false
        balance = balance - amount
        return true

    def get_balance: float
        return balance

def main()
    var account = Account()
    account.deposit(100.0)
    print account.get_balance()    # 100

    account.withdraw(25.0)
    print account.get_balance()    # 75

    account.withdraw(100.0)    # Fails (not enough balance)
```

```
print account.get_balance()    # 75 (unchanged)
```

5.3.2 Setter with Side Effects

```
# file: 10_setter_effects.zbr
# teaches: setters with side effects
# chapter: 10-Properties-and-Computed-Values

class User
  var username: str = ""
  var email: str = ""
  var last_modified: str = ""

  def set_username(new_name: str): bool
    if new_name.len < 3
      return false
    username = new_name
    last_modified = "2024-01-01" # Update timestamp
    return true

  def set_email(new_email: str): bool
    if not new_email.contains("@")
      return false
    email = new_email
    last_modified = "2024-01-01" # Update timestamp
    return true

def main()
  var user = User()
  if user.set_username("alice")
```

```
print "Username set"

if user.set_email("alice@example.com")
  print "Email set"
  print "Last modified: ${user.last_modified}"
```

5.4 Computed Properties

```
# file: 10_computed.zbr
# teaches: expensive computed properties
# chapter: 10-Properties-and-Computed-Values

class DataSet
  var numbers: List(int) = List()

  def sum: int
    var total = 0
    for num in numbers
      total = total + num
    return total

  def average: float
    if numbers.count() == 0
      return 0.0
    return sum / numbers.count()

  def min_value: int
    var min = numbers.at(0)
```

```
        for num in numbers
            if num < min
                min = num
        return min

def max_value: int
    var max = numbers.at(0)
    for num in numbers
        if num > max
            max = num
    return max

def main()
    var data = DataSet()
    data.numbers.add(10)
    data.numbers.add(20)
    data.numbers.add(30)
    data.numbers.add(40)

    print "Sum: ${data.sum()}"           # 100
    print "Average: ${data.average()}"   # 25
    print "Min: ${data.min_value()}"     # 10
    print "Max: ${data.max_value()}"     # 40
```

5.5 Lazy Initialization

```
# file: 10_lazy_init.zbr
# teaches: lazy initialization
```

```
# chapter: 10-Properties-and-Computed-Values

class Database

    var connection: str? = nil
    var is_connected: bool = false

    def get_connection: str
        if connection == nil
            # Expensive operation: only when needed
            connection = "Connected to DB"
            is_connected = true
        return connection

def main()
    var db = Database()

    # Connection not created yet
    print "Is connected: ${db.is_connected}" # false

    # Access connection (now it's created)
    if db.get_connection() as conn
        print conn                                # Connected to DB

    # Already exists
    if db.get_connection() as conn2
        print conn2                                # Connected to DB
```

5.6 Real World: Temperature Converter

```
# file: 10_temperature.zbr
# teaches: properties in realistic scenarios
# chapter: 10-Properties-and-Computed-Values

class Temperature
  var celsius: float = 0.0

  def fahrenheit: float
    return celsius * 9.0 / 5.0 + 32.0

  def kelvin: float
    return celsius + 273.15

  def set_from_fahrenheit(f: float)
    celsius = (f - 32.0) * 5.0 / 9.0

  def set_from_kelvin(k: float)
    celsius = k - 273.15

  def is_freezing: bool
    return celsius <= 0.0

  def is_boiling: bool
    return celsius >= 100.0

def main()
  var temp = Temperature()
  temp.celsius = 25.0
```

```
print "Celsius: ${temp.celsius}"
print "Fahrenheit: ${temp.fahrenheit()}"
print "Kelvin: ${temp.kelvin()}"
print "Freezing: ${temp.is_freezing()}"
print "Boiling: ${temp.is_boiling()}"

temp.set_from_fahrenheit(98.6)
print "Body temp in Celsius: ${temp.celsius}"
```

5.7 Real World: Configuration

```
# file: 10_config.zbr
# teaches: configuration management
# chapter: 10-Properties-and-Computed-Values

class Config
  var host: str = "localhost"
  var port: int = 8080
  var debug: bool = false

  def set_host(h: str): bool
    if h.len == 0
      return false
    host = h
    return true

  def set_port(p: int): bool
```

```
        if p < 1 or p > 65535
            return false

        port = p
        return true

def get_url: str
    return "http://${host}:${port}"

def set_debug(d: bool)
    debug = d

def main()
    var config = Config()
    config.set_host("example.com")
    config.set_port(443)
    config.set_debug(true)

    print config.get_url()          # http://example.com:443
    print "Debug: ${config.debug}"  # true
```

5.8 Common Patterns

5.8.1 Read-Only Properties

```
class BankAccount
    var balance: float = 0.0

    def get_balance: float
```

```
    return balance # Can read

def deposit(amount: float)
    balance = balance + amount # Can modify only via methods

# No setter: direct assignment not allowed
```

5.8.2 Write-Only Properties

```
class Password
    var encrypted_password: str = ""

    def set_password(plain: str)
        # Encrypt and store
        encrypted_password = "encrypted:" + plain

# No getter: can't read it back
```

5.9 If you're new to programming

A **getter** is a method that retrieves a value (often computing it on the fly).

A **setter** is a method that stores a value (often with validation).

Computed properties are values you calculate rather than store. Like `age = current_year - birth_year`.

Lazy initialization means “don't create something expensive until someone actually asks for it.”

5.10 Common Mistakes

Mistake: Exposing internal fields directly

```
class User
  var age: int = 0
var user = User()
user.age = -5 # ❌ No validation!
```

Better:

```
class User
  var age: int = 0
  def set_age(new_age: int): bool
    if new_age < 0
      return false
    age = new_age
    return true
```

Mistake: Computing expensive properties every time

```
class DataSet
  var numbers: List(int) = List()

  def sum: int # Recalculates every call
    var total = 0
    for num in numbers
      total = total + num
    return total # O(n) every time!
```

Better (if called often):

```
class DataSet
  var numbers: List(int) = List()
  var cached_sum: int? = nil
```

```
def sum: int
  if cached_sum == nil
    var total = 0
    for num in numbers
      total = total + num
    cached_sum = total
  return cached_sum

def add_number(num: int)
  numbers.add(num)
  cached_sum = nil # Invalidate cache
```

Mistake: Side effects in getters

```
class Logger
  var count: int = 0

  def get_count: int
    count = count + 1 # ❌ Has side effect!
    return count
```

Better:

```
class Logger
  var count: int = 0

  def get_count: int
    return count # ❌ Pure getter, no side effects

  def log_access
    count = count + 1 # Explicit method for side effects
```

5.11 Exercises

5.11.1 Exercise 1: Bank Account Properties

Create a bank account with validated setters:

Solution

```
class BankAccount

  var owner: str = ""
  var balance: float = 0.0
  var interest_rate: float = 0.02

  def set_owner(name: str): bool
    if name.len < 2
      return false
    owner = name
    return true

  def deposit(amount: float): bool
    if amount <= 0.0
      return false
    balance = balance + amount
    return true

  def withdraw(amount: float): bool
    if amount > balance
      return false
    balance = balance - amount
    return true
```

```
def apply_interest
    var interest = balance * interest_rate
    balance = balance + interest

def get_balance: float
    return balance

def main()
    var account = BankAccount()
    account.set_owner("Alice")
    account.deposit(1000.0)
    account.apply_interest()
    print "Balance: ${account.get_balance()}"
```

5.11.2 Exercise 2: Circle Properties

Create a circle class with radius and diameter properties:

Solution

```
class Circle
    var radius: float = 0.0

    def set_radius(r: float): bool
        if r <= 0.0
            return false
        radius = r
        return true

    def diameter: float
        return radius * 2.0
```

```
def set_diameter(d: float): bool
    if d <= 0.0
        return false
    radius = d / 2.0
    return true

def area: float
    return 3.14159 * radius * radius

def circumference: float
    return 2.0 * 3.14159 * radius

def main()
    var circle = Circle()
    circle.set_radius(5.0)

    print "Radius: ${circle.radius}"
    print "Diameter: ${circle.diameter()}"
    print "Area: ${circle.area()}"
    print "Circumference: ${circle.circumference()}"

    circle.set_diameter(20.0)
    print "New radius: ${circle.radius}"
```

5.11.3 Exercise 3: User Profile with Validation

Create a user profile with validated properties:

Solution

```
class UserProfile

    var username: str = ""
    var email: str = ""
    var age: int = 0

    def set_username(name: str): bool
        if name.len < 3 or name.len > 20
            return false
        username = name
        return true

    def set_email(addr: str): bool
        if not addr.contains("@") or not addr.contains(".")
            return false
        email = addr
        return true

    def set_age(a: int): bool
        if a < 13 or a > 120
            return false
        age = a
        return true

    def is_adult: bool
        return age >= 18

    def is_valid: bool
        return username.len > 0 and email.contains("@") and age > 0

def main()
```

```
var user = UserProfile()
if user.set_username("alice_wonder")
    print "Username set"
if user.set_email("alice@example.com")
    print "Email set"
if user.set_age(25)
    print "Age set"

print "Is adult: ${user.is_adult()}"
print "Is valid: ${user.is_valid()}"
```

5.12 Next Steps

- → **11-Nil-Tracking** — Advanced safety with properties
 - → **14-Contracts** — Enforce property invariants
 - **Project-1-CLI-Tool** — Use configuration properties
-

5.13 Key Takeaways

- **Getters compute** values from fields
 - **Setters validate** before storing
 - **Computed properties** derive from other data
 - **Lazy initialization** defers expensive work
 - **Control access** to prevent invalid states
 - **Encapsulation** protects class invariants
-

Next: Head to **Part 3** and **11-Nil-Tracking** for Zebra's safety features.

Chapter 6

Part 3: Advanced Features

11: Nil Tracking and Safety

Audience: All

Time: 120 minutes

Prerequisites: 02-Values, 05-Control-Flow, 07-Classes

You’ll learn: Nullable types, nil checking, type narrowing, the **to!** operator, optionals

6.1 The Big Picture

Nil (null/nothing) is one of the most common sources of bugs. Zebra’s **nil tracking** forces you to handle it:

“A billion-dollar mistake.” — Tony Hoare, on inventing the null reference

Zebra says: **Explicit nil is safe. Implicit nil is forbidden.**

6.2 Nullable Types

6.2.1 Marking Optionality

```
# file: 11_nullable.zbr
# teaches: nullable types
# chapter: 11-Nil-Tracking-and-Safety

class User
  var name: str      # Can't be nil
  var nickname: str? # Can be nil
  var bio: str?      # Can be nil

def main()
  var user = User()
  user.name = "Alice" # ? Fine
  user.nickname = nil # ? Allowed (it's str?)
  user.bio = "Developer" # ? Can assign string to str?

  # This won't compile:
  # var empty: str = nil # ? str can't be nil
```

Key point: The ? mark means “this can be nil or the type.”

6.3 Nil Checking

6.3.1 Safe Access Pattern

```
# file: 11_nil_check.zbr
# teaches: nil checking
```

```
# chapter: 11-Nil-Tracking-and-Safety

def main()
  var nickname: str? = "Bobby"
  var empty: str? = nil

  # Check before using
  if nickname != nil
    print nickname      # Safe: known to not be nil

  if empty == nil
    print "No nickname set"
  else
    print empty

  # Two ways to check:
  if nickname != nil
    print "Has nickname"

  if nickname == nil
    print "No nickname"
  else
    print "Has: ${nickname}"
```

6.3.2 Type Narrowing

After checking, the type is narrowed:

```
# file: 11_narrowing.zbr
# teaches: type narrowing
# chapter: 11-Nil-Tracking-and-Safety
```

Type Narrowing in Zebra

Example 1: Nullable Type Check

Before Check:

```
var name as str? = nil
# name could be nil or string
# Type: str? (nullable)
print name # ERROR!
```



After Check:

```
if name != nil
# name is guaranteed not nil
# Type: str (non-nullable)
print name # OK!
```

Example 2: Result Type Narrowing

Before Check:

```
var result = parse_int("42")
# result is Result(int, str)
# Could be Ok or Err
var x = result.okValue()
# ERROR! Could be Err
```



After Check:

```
if result.isOk()
var x = result.okValue()
# Guaranteed to be Ok now
# Type is narrowed to Ok variant
print x # OK!
```

Zebra's Type Narrowing Guarantees

- ✓ Type checker understands control flow
- ✓ After nil check (if x != nil), x is guaranteed non-nil in that scope
- ✓ After Result check (if x.isOk()), x is guaranteed Ok in that scope
- ✓ Narrowing scope ends at block boundary (outside if block, original type restored)

Figure 6.1: Type Narrowing Flow

```
def process_name(input: str?)
  if input == nil
    return # Exit early if nil

  # From here, input is narrowed to str
  print input.len      # ? Safe: know it's str
  print input.upper()  # ? Safe
```

6.4 if x as n — Unwrap-and-Bind

The most idiomatic way to unwrap an optional and use the inner value is the `if x as n` binding form. When `x: T?`, it binds a non-optional `n: T` in the then-branch — no separate `to!` required:

```
# file: 11_if_as.zbr
# teaches: optional unwrap binding form
# chapter: 11-Nil-Tracking-and-Safety

def main()
  var maybe_name: str? = "Alice"

  if maybe_name as name
    # `name` is `str` here — non-optional
    print "Hello, ${name}!"
  else
    print "No name"
```

This form combines the nil-check, the unwrap, and the binding into a single statement, and it works both for plain optionals (`T?`) and for optional class/union check + unwrap together:

```
# Combined type check + unwrap:
var maybe_user: User? = lookup()
if maybe_user is User as u
    print u.name           # u is `User` (non-optional)
```

`if x as n` is preferred over `to!` whenever the wrapped value is going to be used right away — it's safer (no panic possibility) and reads more naturally.

6.5 The `to!` Operator (Unwrap)

Warning: Only use when you're absolutely certain the value isn't `nil`.

```
# file: 11_unwrap.zbr
# teaches: unwrap operator
# chapter: 11-Nil-Tracking-and-Safety

def main()
    var name: str? = "Alice"

    # Unwrap: assert it's not nil
    var safe_name = name to!
    print safe_name           # Now just str

    # If it WAS nil, this would crash
    var empty: str? = nil
    # var crash = empty to!  # ☹ Would panic at runtime
```

6.6 Unwrap with Fallback

```
# file: 11_unwrap_or.zbr
# teaches: safe unwrapping
# chapter: 11-Nil-Tracking-and-Safety

def get_user_name(user_id: int): str?
  if user_id == 1
    return "Alice"
  return nil

def main()
  var maybe_name = get_user_name(1)

  # Option 1: Check and use default
  if maybe_name as name
    print name
  else
    print "Unknown user"

  # Option 2: if method exists, unwrapOr
  # var safe_name = maybe_name.unwrapOr("Guest")
  # print safe_name
```

6.7 Real World: Database Queries

```
# file: 11_database.zbr
# teaches: nil in realistic scenarios
```

```
# chapter: 11-Nil-Tracking-and-Safety

class User
  var id: int
  var name: str
  var email: str?

class UserDatabase
  static
    def find_user(user_id: int): User?
      if user_id == 1
        var user = User()
        user.id = 1
        user.name = "Alice"
        user.email = "alice@example.com"
        return user
      return nil

    def find_user_email(user_id: int): str?
      var user = find_user(user_id)
      if user == nil
        return nil
      return user.email

def main()
  var user = UserDatabase.find_user(1)
  if user != nil
    print user.name
    if user.email != nil
      print user.email
```

```
    else
        print "No email on file"
    else
        print "User not found"

# Chaining nil checks
var email = UserDatabase.find_user_email(999)
if email != nil
    print "Email: ${email}"
else
    print "User not found"
```

6.8 Common Patterns

6.8.1 Optional Chaining

```
def get_user_city(user_id: int): str?
    var user = find_user(user_id)
    if user == nil
        return nil
    var address = get_address(user.id)
    if address == nil
        return nil
    return address.city
```

6.8.2 Guard Clauses

```
def process(data: str?)
  if data == nil
    return
  if data.len == 0
    return
  # Now process safely
  do_work(data)
```

6.9 If you're new to programming

Nil means “nothing” or “no value.” Different languages call it null, None, undefined.

Zebra makes you declare which variables **can** be nil with `?`. This prevents accidental crashes.

Type narrowing means the compiler recognizes that after you check `if x != nil`, you can safely use `x`.

6.10 Common Mistakes

Mistake: Forgetting to check for nil

```
var email: str? = get_email()
print email.len # ☹ Crash if email is nil!
```

Better:

```
var email: str? = get_email()
if email != nil
    print email.len # ? Safe
```

Mistake: Using `to!` without certainty

```
var value: str? = get_value()
print value to! # ? Crashes if value is nil
```

Better:

```
var value: str? = get_value()
if value != nil
    print value # ? Safe, or use unwrapOr
```

Mistake: Assigning `nil` to non-nullable

```
var name: str = nil # ? str can't be nil
```

Better:

```
var name: str? = nil # ? Declares it can be nil
```

6.11 Exercises

6.11.1 Exercise 1: Find User Email

Write a function that safely retrieves a user's email:

Solution

```
class User
    var id: int
    var name: str
    var email: str?
```

```
class UserDB
  static
    def find_user(id: int): User?
      if id == 1
        var user = User()
        user.id = 1
        user.name = "Alice"
        user.email = "alice@example.com"
        return user
      return nil

    def get_email(user_id: int): str?
      var user = find_user(user_id)
      if user == nil
        return nil
      return user.email

def main()
  var email = UserDB.get_email(1)
  if email != nil
    print "Email: ${email}"
  else
    print "User not found or no email"
```

6.11.2 Exercise 2: Safe Division

Write a division function that returns nil on error:

Solution

```
class Calculator
    static
        def safe_divide(a: float, b: float): float?
            if b == 0.0
                return nil
            return a / b

def main()
    var result = Calculator.safe_divide(10.0, 2.0)
    if result != nil
        print "Result: ${result}"

    var bad = Calculator.safe_divide(10.0, 0.0)
    if bad != nil
        print bad
    else
        print "Cannot divide by zero"
```

6.11.3 Exercise 3: Nullable Chain

Write a function that navigates nullable fields:

Solution

```
class Profile
    var name: str
    var bio: str?

class User
    var id: int
    var profile: Profile?
```

```
class UserService
  static
    def get_user_bio(user_id: int): str?
      var user = find_user(user_id)
      if user == nil
        return nil
      var profile = user.profile
      if profile == nil
        return nil
      return profile.bio

    def find_user(id: int): User?
      if id == 1
        var user = User()
        user.id = 1
        var profile = Profile()
        profile.name = "Alice"
        profile.bio = "Developer"
        user.profile = profile
        return user
      return nil

def main()
  var bio = UserService.get_user_bio(1)
  if bio != nil
    print "Bio: ${bio}"
  else
    print "No bio found"
```

6.12 Next Steps

- → **12-Error-Handling** — Results for error cases
 - → **14-Contracts** — Enforce non-nil invariants
 - **Project-2-HTTP-Server** — Handle nil API responses
-

6.13 Key Takeaways

- **? marks nullable types** — `str?` can be string or nil
 - **Check before using** — `if value != nil { ... }`
 - **Type narrowing** — Compiler recognizes after checks
 - **to! unwraps** — Only when certain it's not nil
 - **Guard clauses** — Exit early if nil
 - **Nil safety prevents crashes** — It's a feature, not a limitation
-

Next: Head to **12-Error-Handling** for Results and error propagation.

12: Error Handling

Audience: All

Time: 120 minutes

Prerequisites: 02-Values, 04-Functions, 07-Classes, 11-Nil-Tracking

You'll learn: `throws` annotation, `raise` statement, `try/catch` blocks, error propagation, `catch` expressions

6.14 The Big Picture

Zebra's primary error model is **exceptions** — **throws/raise/try/catch**. The shape is inspired by Zig's error-set system (each **throws** function returns **anyerror!T** in the generated Zig), but enriched: **raise "msg"** carries a string message, and **raise "msg", obj** attaches a structured details object via a thread-local **_error_ctx** — strictly more than Zig's payload-less error enums.

Result(T) exists as a secondary error-as-value type for the cases where that fits better (parser combinators, batch validators), but the recommended default is **throws**.

Either way, error paths are explicit in the type system: you can see at a glance which functions may fail and which cannot.

6.15 Functions That Can Fail: **throws**

Mark a function with **throws** to indicate it may produce an error:

```
# file: 12_throws_basic.zbr
# teaches: throws annotation
# chapter: 12-Error-Handling

class Validator
    static
        def parse_int(text: str): int throws
            if text.len == 0
                raise "Empty string"
            # Parse and return
            return 42

def main()
```

```
var result = Validator.parse_int("123") catch 0
print result
```

Breakdown: `- : int throws` — This function returns `int` but may fail - `raise "Empty string"` — Signal an error with a message - `catch 0` — If the call fails, use `0` as the fallback value

6.16 `raise` — Signaling Errors

Use `raise` to signal that something went wrong:

```
# file: 12_raise.zbr
# teaches: raise statement
# chapter: 12-Error-Handling

class FileLoader
  static
    def load(path: str): str throws
      if path.len == 0
        raise "Path cannot be empty"
      if not path.endsWith(".txt")
        raise "Only .txt files supported"
      return "file contents here"
```

`raise` immediately exits the function and propagates the error to the caller.

6.17 try / catch — Handling Errors

6.17.1 Block Form

Use `try/catch` blocks for structured error handling:

```
# file: 12_try_catch.zbr
# teaches: try/catch blocks
# chapter: 12-Error-Handling

def main()
  try
    var value = Validator.parse_int("")
    print "Got: ${value}"
  catch
    print "Failed to parse"
```

6.17.2 Catch with Binding

Bind the error value to inspect it:

```
# file: 12_catch_binding.zbr
# teaches: catch with error binding
# chapter: 12-Error-Handling

def main()
  try
    var value = Validator.parse_int("")
    print "Got: ${value}"
  catch |err|
    print "Error: ${err}"
```

6.17.3 Catch with Type

Specify a type for the error binding:

```
# file: 12_catch_typed.zbr
# teaches: catch with typed error binding
# chapter: 12-Error-Handling

def main()
  try
    var value = Validator.parse_int("")
    print value
  catch |err as str|
    print "String error: ${err}"
```

6.18 catch Expression — Inline Fallbacks

For simple cases, use `catch` as an expression to provide a default:

```
# file: 12_catch_expr.zbr
# teaches: catch expression for defaults
# chapter: 12-Error-Handling

def main()
  # Provide a default value if the call fails
  var value = Validator.parse_int("abc") catch 0
  print value # 0

  # Catch with binding
  var msg = Validator.parse_int("") catch |e| "failed: ${e}"
  print msg
```

This is the most common pattern for simple error recovery.

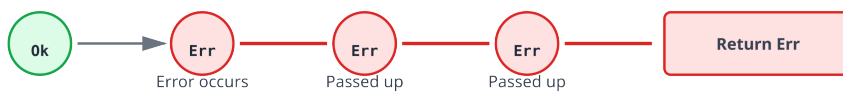
6.19 Error Propagation

Error Propagation with Result Types

Success Path: `Ok(T)`



Error Path: `Err(E)` Bubbles Up



Code Example: Error Propagation

```

def load_config(path as str) as Result(Config, str)
  var content = File.read(path) # Returns Result
  if content.isErr() # Check for error
    return content # Propagate error up
  var data = parse_json(content.okValue()) # Process if Ok
  if data.isErr()
    return data # Propagate parsing error
  return Result.ok(create_config(data.okValue()))
  
```

Figure 6.2: Error Propagation Flow

Functions annotated with **throws** can propagate errors from callees automatically. If a **throws** function calls another **throws** function without catching, the error propagates up:

```

# file: 12_propagation.zbr
# teaches: error propagation
# chapter: 12-Error-Handling
  
```

```
class Parser
  static
    def parse_config(text: str): str throws
      if text.len == 0
        raise "Empty config"
      return "parsed"

class System
  static
    def load_system(config_text: str): str throws
      # If parse_config raises, the error propagates up
      var parsed = Parser.parse_config(config_text)
      return "System loaded with: ${parsed}"

def main()
  var result = System.load_system("data") catch "load failed"
  print result
```

6.20 Real World: API Client

```
# file: 12_api_client.zbr
# teaches: realistic error handling
# chapter: 12-Error-Handling

class APIClient
  static
    def fetch_user(user_id: int): str throws
      if user_id <= 0
```

```
        raise "Invalid user ID"
    if user_id == 1
        return "Alice"
    raise "User not found"

def fetch_and_greet(user_id: int): str throws
    var user = fetch_user(user_id)
    return "Hello, ${user}!"

def main()
    # Using catch expression
    var greeting = APIClient.fetch_and_greet(1) catch "Could not greet"
    print greeting # Hello, Alice!

    # Using try/catch block
    try
        var g2 = APIClient.fetch_and_greet(999)
        print g2
    catch |err|
        print "Error: ${err}"
```

6.21 Exercises

6.21.1 Exercise 1: Safe Division

Write a function `safe_divide(a: int, b: int): int throws` that raises on division by zero.

Solution

```
class MathUtils
  static
    def safe_divide(a: Int, b: Int): Int throws
      if b == 0
        raise "Division by zero"
      return a / b

def main()
  var r1 = MathUtils.safe_divide(10, 2) catch 0
  var r2 = MathUtils.safe_divide(10, 0) catch 0
  print r1 # 5
  print r2 # 0
```

6.21.2 Exercise 2: Chained Error Handling

Write two functions that chain: `parse_age(text: String): Int throws` and `validate_age(text: String): String throws` (returns “valid” if age is 0-150).

Solution

```
class AgeValidator
  static
    def parse_age(text: String): Int throws
      if text.len == 0
        raise "Empty input"
      var age = text.toInt()
      if age == nil
        raise "Not a number"
      return age

    def validate_age(text: String): String throws
```

```
    var age = parse_age(text)
    if age < 0
      raise "Age cannot be negative"
    if age > 150
      raise "Age too large"
    return "valid"

def main()
  try
    var result = AgeValidator.validate_age("25")
    print result # valid
  catch |err|
    print "Error: ${err}"

var r2 = AgeValidator.validate_age("200") catch "invalid"
print r2 # invalid
```

6.22 Next Steps

- → **13-Generics** — Type-safe containers
 - → **15-Pipelines** — Chain operations gracefully
-

6.23 Key Takeaways

- **throws** — Annotates functions that may fail
- **raise** — Signals an error, exits the function immediately
- **try/catch** — Block-form error handling with optional binding

- **catch expression** — Inline fallback: `expr catch default`
 - **Errors propagate** — A `throws` function can let callee errors bubble up
 - **Errors are explicit** — You always know which functions can fail
-

Next: Head to **13-Generics** for type-safe abstractions.

13: Generics and Type Constraints

Audience: Experienced programmers

Time: 120 minutes

Prerequisites: 02-Values-and-Types, 07-Classes-and-Instances, 08-Interfaces-and-Protocols

You'll learn: Generic types, type parameters, constraints, practical uses, common pitfalls

6.24 The Big Picture

So far, you've written code that works with specific types: a `List(int)`, a `HashMap(str, int)`. But what if you want a function that works with **any type**? That's where **generics** come in.

Generics let you write code that's *type-safe* but *reusable* across different types:

```
class Container(T)
  var item: T

  def get: T
    return item
```

`Container(int)` holds integers. `Container(str)` holds strings. Same code, different types. This is safer than using `object` or `any`, because the type checker still verifies you don't mix types.

6.25 Generic Classes

Generic Type Instantiation

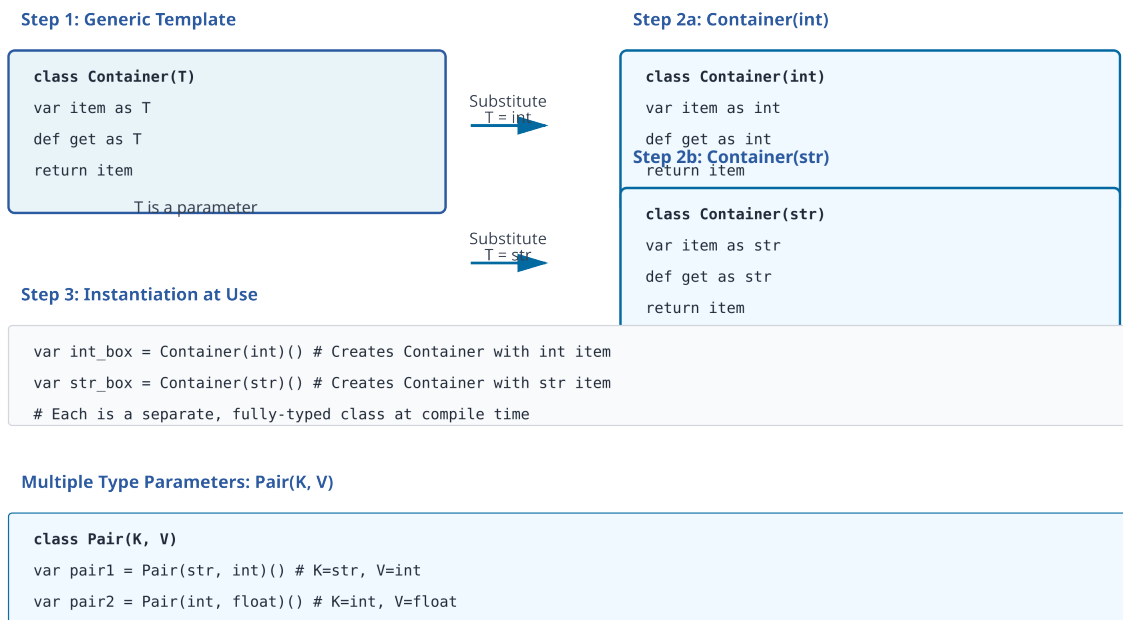


Figure 6.3: Generics Instantiation

The simplest generic is a **container** that holds a **single value**:

```
# file: 13_generic_container.zbr
# teaches: generic class definition
# chapter: 13-Generics-and-Type-Constraints

class Container(T)
  var item: T

  def store(value: T)
    item = value
```

```
def retrieve: T
    return item

def main()
    var int_box = Container(int)()
    int_box.store(42)
    print int_box.retrieve() # Output: 42

    var str_box = Container(str)()
    str_box.store("hello")
    print str_box.retrieve() # Output: hello
```

Notice the syntax: - `Container(T)` — `T` is a **type parameter** - `Container(int)()` — substitute `int` for `T`, then create an instance - Inside the class, `T` can be used like any other type

You can have **multiple type parameters**:

```
# file: 13_generic_pair.zbr
# teaches: multiple type parameters
# chapter: 13-Generics-and-Type-Constraints

class Pair(K, V)
    var key: K
    var value: V

    def set_key(k: K)
        key = k

    def set_value(v: V)
        value = v
```

```
def get_key: K
    return key

def get_value: V
    return value

def main()
    var p = Pair(str, int)()
    p.set_key("count")
    p.set_value(42)
    print "Key: ${p.get_key()}, Value: ${p.get_value()}"
```

6.26 Generic Methods

You can also write **generic methods** within regular classes:

```
# file: 13_generic_methods.zbr
# teaches: generic methods
# chapter: 13-Generics-and-Type-Constraints

class Utils
    static
        def identity(value: T): T
            return value

        def first_of_three(a: T, b: T, c: T): T
            return a
```

```
def main()
    var x = Utils.identity(42)
    print x # Output: 42

    var y = Utils.identity("hello")
    print y # Output: hello

    var z = Utils.first_of_three(1, 2, 3)
    print z # Output: 1
```

The type parameter `T` is inferred from the arguments you pass.

6.27 Generic Collections

You already use generics implicitly with `List` and `HashMap`:

```
# file: 13_generic_collections.zbr
# teaches: using generic stdlib types
# chapter: 13-Generics-and-Type-Constraints

def main()
    var numbers: List(int) = List()
    numbers.add(1)
    numbers.add(2)
    numbers.add(3)

    for n in numbers
        print n

    var ages: HashMap(str, int) = HashMap()
```

```
ages.put("Alice", 30)
ages.put("Bob", 25)

for name, age in ages:
    print "${name}: ${age}"
```

These are all generic types. The standard library provides them pre-built.

6.28 Type Constraints

Sometimes you want a generic that works with **any type that implements an interface**:

```
# file: 13_type_constraints.zbr
# teaches: interface constraints
# chapter: 13-Generics-and-Type-Constraints

interface Printable
    def display: str

class Dog
    implements Printable
    def display: str
        return "Woof!"

class Cat
    implements Printable
    def display: str
        return "Meow!"

class Printer
```

```
static
    def print_item(item: Printable)
        print item.display()

def main()
    var dog = Dog()
    var cat = Cat()

    Printer.print_item(dog) # Output: Woof!
    Printer.print_item(cat) # Output: Meow!
```

Here, `Printer.print_item` accepts **any type** that implements `Printable`. This is a **constraint**: “T must implement interface `Printable`”.

More advanced: constraints on generic methods:

```
# file: 13_generic_constraints_advanced.zbr
# teaches: constraints in generic methods
# chapter: 13-Generics-and-Type-Constraints

interface Comparable
    def compare_to(other: this): Int

class ComparableList(T)
    var items: List(T) = List()

    def add(item: T)
        items.add(item)

    def find_max: T?
        if items.count() == 0
```

```
        return nil

    var max = items.at(0)
    var i = 1
    while i < items.count()
        var item = items.at(i)
        # Here, T must implement Comparable
        if item.compare_to(max) > 0
            max = item
        i = i + 1
    return max

def main()
    var list = ComparableList()
    list.add(10)
    list.add(5)
    list.add(20)
    var max = list.find_max()
    if max != nil
        print "Max: ${max}"
```

6.29 Real World: Generic Cache

A practical example: a **cache that works with any type**:

```
# file: 13_generic_cache.zbr
# teaches: realistic generic class
# chapter: 13-Generics-and-Type-Constraints

class Cache(K, V)
```

```
var data: HashMap(K, V) = HashMap()
var max_size: int

def init(max_size: int)
    this.max_size = max_size

def put(key: K, value: V)
    if data.count() >= max_size and not data.contains(key)
        # Evict first key (simplistic LRU simulation)
        # In real code, track access order
        pass
    data.put(key, value)

def lookup(key: K): V?
    if data.contains(key)
        return data.fetch(key)
    return nil

def clear
    data = HashMap()

def main()
    var cache = Cache(str, int)()
    cache.init(3)
    cache.put("a", 1)
    cache.put("b", 2)
    cache.put("c", 3)

    var val = cache.lookup("b")
    if val != nil
```

```
print "Got: ${val}"
```

6.30 Common Mistakes

6.30.1 Mistake 1: Forgetting to Instantiate Generic Parameters

```
# WRONG
var box = Container() # Error: T not specified
box.store(42)

# CORRECT
var box = Container(int)()
box.store(42)
```

6.30.2 Mistake 2: Mixing Types in a Generic Container

```
# WRONG
var box = Container(int)()
box.store("hello") # Error: Expected int, got str

# CORRECT
var box = Container(str)()
box.store("hello")
```

6.30.3 Mistake 3: Using Constraints Incorrectly

```
# WRONG - method doesn't actually require Comparable
def find_max(items: List(T)): T
    var max = items.at(0)
```

```
var item = items.at(1)

if item > max # Error: > not defined for all T
    max = item

return max

# CORRECT - either don't use >, or require Comparable interface
def find_max(items: List(T)): T
    var max = items.at(0)
    for item in items
        if item.toString() > max.toString() # Convert to string for comparison
            max = item
    return max
```

6.30.4 Mistake 4: Type Erasure at Runtime

```
# DANGER - at runtime, type information is lost
def process(items: List(T))
    for item in items
        if item isa int # This may not work as expected
            print item + 10
```

In Zebra, type parameters are **erased** during code generation to Zig. Use interfaces to encode types you need at runtime.

6.31 Exercises

6.31.1 Exercise 1: Generic Stack

Implement a `Stack(T)` class with `push`, `pop`, and `is_empty` methods:

Solution

```
class Stack(T)
    var items: List(T) = List()

    def push(value: T)
        items.add(value)

    def pop: T?
        if items.count() == 0
            return nil
        var value = items.at(items.count() - 1)
        # Remove last item (simplified - no remove method)
        return value

    def is_empty: bool
        return items.count() == 0

def main()
    var stack = Stack(int)()
    stack.push(1)
    stack.push(2)
    stack.push(3)

    var val = stack.pop()
    if val != nil
        print "Popped: ${val}"
```

6.31.2 Exercise 2: Generic Filter Function

Write a function that filters a list based on a predicate (function that returns bool):

Solution

```
class ListUtils
    static
        def filter(items: List(T), predicate as T -> bool): List(T)
            var result: List(T) = List()
            for item in items
                if predicate(item)
                    result.add(item)
            return result

def main()
    var numbers: List(int) = List()
    numbers.add(1)
    numbers.add(2)
    numbers.add(3)
    numbers.add(4)

    var is_even: T -> bool = { x in x % 2 == 0 }
    var evens = ListUtils.filter(numbers, is_even)

    for e in evens
        print e
```

6.31.3 Exercise 3: Generic Wrapper with Validation

Create a `ValidatedBox(T)` that only accepts values passing a validation function:

Solution

```
class ValidatedBox(T)
    var item: T?
```

```
var validator: T -> bool

def init(validator: T -> bool)
  this.validator = validator

def set(value: T): bool
  if validator(value)
    item = value
    return true
  return false

def get: T?
  return item

def main()
  var age_box = ValidatedBox(int)()
  age_box.init({ x in x >= 0 and x <= 150 })

  if age_box.store(25)
    print "Valid age: ${age_box.retrieve()}"

  if not age_box.store(200)
    print "Invalid age"
```

6.32 Key Takeaways

- **Generics enable type-safe reusable code** — Write once, use with many types
- **Type parameters are like function parameters** — `Container(T)` where `T` is filled

in at use time

- **Collections are generics** — `List(T)`, `HashMap(K, V)` work with any type
 - **Constraints limit generics** — Use interfaces to require specific capabilities
 - **Type erasure happens at compile time** — Don't rely on runtime type information
-

6.33 Next Steps

- → **14-Contracts** — Express what generics must provide
 - → **15-Pipelines** — Chain generic operations cleanly
 - → **Project 1** — Use generics in real code
-

Generics are the bridge between flexibility and safety. Master them, and your code scales.

Chapter 7

14: Contracts and Assertions

Audience: All **Time:** 90 minutes **Prerequisites:** 04-Functions-and-Scope, 07-Classes-and-Instances, 12-Error-Handling **You'll learn:** `require` (preconditions), `ensure` (postconditions, `result`, `old`), `invariant` (class invariants), `assert`, and the `--turbo` strip flag

7.1 The Big Picture

Contracts are one of Zebra's marquee features. A contract is an *executable specification* attached to a function or class — the compiler emits runtime checks for each clause:

```
def withdraw(amount: int): int throws
  require
    amount > 0
    amount <= balance
  ensure
    result == old balance - amount
  balance = balance - amount
  return balance
```

Three keywords cover the common cases:

Keyword	Where	What it asserts
require	Top of method body	Inputs are valid (precondition)
ensure	Top of method body	Output is valid (postcondition)
invariant	Top of class body	Object state is always valid

These get checked at runtime by default. Pass `--turbo` to the compiler to strip them all out for production builds — same semantics as `-DNDEBUG` for C `assert`.

Why bother? Manual `if x raise "..."` checks at the top of each method work, but they bury the contract in normal control flow. `require` / `ensure` / `invariant` make the contract syntactically distinct, give it a structured failure message with the failing clause, and let the production build remove the cost without touching the source.

This is **Design by Contract**, an idea from Eiffel that Zebra inherits and adapts.

7.2 Preconditions: `require`

A `require` block lists conditions that must hold when the method is called. Each condition is a Boolean expression on its own line:

```
# file: 14_require.zbr
# teaches: precondition checking with require
# chapter: 14-Contracts-and-Assertions

class Bank
  var balance: int = 100

  def withdraw(amount: int): int
    require
```

```
        amount > 0
        amount <= balance
        balance = balance - amount
        return balance

def main()
    var bank = Bank()
    print bank.withdraw(50)          # 50
    # bank.withdraw(-10)             # → panic: require failed in 'withdraw'
    # bank.withdraw(1000)            # → panic: require failed in 'withdraw'
```

If any clause is false, Zebra panics with a message naming the method and the failing clause.

When to use **require**:

- Validate parameters (range, non-emptiness, format)
- Assert preconditions on the receiver's state (“only call after **init**”)
- Document required ordering between methods

require failures indicate a **caller bug** — the caller violated the contract. Don't use **require** for user input you intend to validate gracefully; that's **throws** + **raise** (Chapter 12).

7.3 Postconditions: **ensure**

An **ensure** block lists conditions that must hold when the method returns normally:

```
# file: 14_ensure.zbr
# teaches: postcondition checking with ensure
# chapter: 14-Contracts-and-Assertions

class Counter
    var count: int = 0
```

```
def increment(): int
    ensure
        count > 0
    count = count + 1
    return count

def main()
    var c = Counter()
    print c.increment()      # 1
    print c.increment()      # 2
```

ensure clauses fire **only on the success path**, not when the method raises an error. This is by design (a throwing method has no meaningful postcondition).

7.3.1 The **result** keyword

Inside **ensure**, the keyword **result** refers to the value the method is about to return. It has the same type as the method's declared return, so member access works naturally:

```
# file: 14_ensure_result.zbr
# teaches: result in ensure clauses
# chapter: 14-Contracts-and-Assertions

class Strings
    static
        def normalise(s: str): str
            ensure
                result.len <= s.len          # never grows
                not result.startsWith(" ")
                not result.endsWith(" ")
            return s.trim()
```

result is only valid inside **ensure**. In the method body, you'd refer to the to-be-returned variable by its own name; **result** is a contract-time binding, not a regular variable.

7.3.2 The **old** snapshot

When you need to compare the post-call state to the pre-call state, use the **old** operator inside **ensure**. It snapshots an expression at function entry:

```
# file: 14_ensure_old.zbr
# teaches: old snapshots in ensure clauses
# chapter: 14-Contracts-and-Assertions

class Counter
  var count: int = 0

  def increment(): int
    ensure
      result == old count + 1      # post-state vs pre-state
      count == old count + 1
    count = count + 1
    return count
```

old expr is valid only inside **ensure**. Each occurrence captures **expr** once at function entry; the captured value is then used in the post-condition check.

7.4 Invariants: **invariant**

A class **invariant** block lists conditions that must hold whenever the object is observable from outside. Zebra checks the invariant after **cue init** returns and after every public method exits:

```
# file: 14_invariant.zbr
# teaches: class invariants
# chapter: 14-Contracts-and-Assertions

class Account
  var balance: int = 0
  var transactions: int = 0

  invariant
    balance >= 0
    transactions >= 0

  def deposit(amount: int)
    require
      amount > 0
    balance = balance + amount
    transactions = transactions + 1

  def withdraw(amount: int)
    require
      amount > 0
      amount <= balance
    balance = balance - amount
    transactions = transactions + 1

def main()
  var acct = Account()
  acct.deposit(50)
  acct.withdraw(30)
  # Both invariants checked after each method returns.
```

Why invariants are a force multiplier: instead of repeating the same five sanity checks at the end of every method, you write them once in **invariant** and the compiler weaves them into every method exit. When you add a sixth method later, it gets the invariant check for free.

An invariant is checked *after* **cue** **init** returns, not before — your constructor is allowed to assemble state stepwise. Same for setters: the invariant is checked once at the end, not after each individual assignment.

7.5 Assertions: **assert**

For one-off checks inside a method body — debug-time sanity probes that don't fit the contract structure — Zebra has a plain **assert** statement:

```
# file: 14_assert.zbr
# teaches: assert statement
# chapter: 14-Contracts-and-Assertions

class Math
  static
    def divide(a: int, b: int): int
      require
        b != 0
      var q = a / b
      assert q * b == a      # integer-division sanity probe
      return q
```

Like the contract clauses, **assert** panics on a false condition and is stripped by **--turbo**. It's a more granular tool than **ensure** — useful for checking intermediate values inside a method body.

7.6 Real-world example: a sorted list

A class that combines **require** (caller contract), **ensure** (per-method guarantee), and **invariant** (always-true property):

```
# file: 14_sorted_list.zbr
# teaches: combining require, ensure, and invariant
# chapter: 14-Contracts-and-Assertions

class SortedList
  var items: List(int)

  cue init
    items = List(int)()

  invariant
    # The list is always non-decreasing
    isSorted()

  def isSorted(): bool
    var i = 1
    while i < items.count()
      if items.at(i - 1) > items.at(i)
        return false
      i = i + 1
    return true

  def insert(value: int)
    ensure
```

```
        items.count() == old items.count() + 1

    # Insert preserving sorted order.
    var i = 0
    while i < items.count() and items.at(i) < value
        i = i + 1
    items.add(value)

    # Bubble the new value left into its sorted position.
    var j = items.count() - 1
    while j > i
        var tmp = items.at(j - 1)
        items.add(items.at(j))
        j = j - 1

def smallest(): int
    require
        items.count() > 0
    return items.at(0)

def main()
    var sl = SortedList()
    sl.insert(3)
    sl.insert(1)
    sl.insert(2)
    print sl.smallest()      # 1
    # Invariant verified after every public method.
```

The invariant catches any bug in `insert` automatically — there’s no need to remember to add a “verify still sorted” check at the end of every mutation.

7.7 --turbo: stripping contracts for production

In normal builds, every `require`, `ensure`, `invariant`, and `assert` emits runtime check code. These add safety during development but cost cycles in hot paths. Pass `--turbo` to the compiler to strip them entirely:

```
zebra app.zbr          # contracts active – runtime checks emitted
zebra --turbo app.zbr   # contracts stripped – same source, faster binary
```

`--turbo` is the same source-to-binary toggle C/C++ get with `-DNDEBUG`. Use it for production releases; develop and test without it.

Don't put load-bearing logic inside contract clauses — anything you can't afford to lose at `--turbo` time. Side effects in `require` / `ensure` are already a code smell; `--turbo` makes the smell into a bug.

7.8 If you're new to programming

A **precondition** is “what must be true before I run.” A **postcondition** is “what I promise will be true after I run.” An **invariant** is “what stays true the whole time the object exists.”

Contracts are a way of writing your assumptions down *as code* instead of in comments — comments rot when the code changes; contracts panic when the code violates them.

7.9 Common Mistakes

Using `require` for user-input validation

```
def parse_age(s: str): int
  require
    s.len > 0      # ❌ user-supplied – should be a graceful failure, not a panic
  return 42
```

Use **throws/raise** for failures the caller can handle:

```
def parse_age(s: str): int throws
  if s.len == 0
    raise "empty input"
  return 42
```

Reserve **require** for caller-bug detection.

Putting side effects in **require** / **ensure** / **invariant**

```
def step(): int
  ensure
    logger.write("step done")    # ❌ side effect in ensure
    result > 0
  ...
```

Contract clauses are stripped by **--turbo**. Anything you do inside one evaporates in production.

Using **result** outside **ensure**

```
def f(): int
  return result + 1    # ❌ `result` is contract-only; not a regular variable
```

result is only meaningful inside **ensure**. Inside the method body, store the to-be-returned value in a regular **var** if you need to refer to it.

Expecting **ensure** to fire on the error path

```
def divide(a: int, b: int): int throws
  ensure
    result > 0
  if b == 0
    raise "div by zero"
  return a / b
```

When the function raises, **ensure** is **not** evaluated — that’s correct behaviour (a throwing method has no meaningful postcondition). Don’t design contracts that depend on **ensure** running on error paths.

7.10 Exercises

7.10.1 Exercise 1: Bank account with contracts

Rewrite the standard bank-account example using **require**, **ensure**, and **invariant**:

Solution

```
class BankAccount
  var balance: int = 0
  var transactions: int = 0

  invariant
    balance >= 0
    transactions >= 0

  def deposit(amount: int)
    require
      amount > 0
    ensure
```

```
        balance == old balance + amount
        transactions == old transactions + 1
    balance = balance + amount
    transactions = transactions + 1

def withdraw(amount: int)
    require
        amount > 0
        amount <= balance
    ensure
        balance == old balance - amount
        transactions == old transactions + 1
    balance = balance - amount
    transactions = transactions + 1

def main()
    var acct = BankAccount()
    acct.deposit(100)
    acct.withdraw(30)
    print "Balance: ${acct.balance}"
    print "Transactions: ${acct.transactions}"
```

The invariant catches any future bug that would leave **balance** negative or **transactions** not incrementing.

7.10.2 Exercise 2: Validated array indexer

Write a class that wraps a `List(int)` with bounds-checked access, using `require` to document the contract:

Solution

```
class SafeArray
  var items: List(int)

  cue init
    items = List(int)()

  def push(v: int)
    ensure
      items.count() == old items.count() + 1
    items.add(v)

  def at(i: int): int
    require
      i >= 0
      i < items.count()
    return items.at(i)

  def first(): int
    require
      items.count() > 0
    return items.at(0)

def main()
  var sa = SafeArray()
  sa.push(10)
  sa.push(20)
  print sa.at(0)           # 10
  print sa.first()         # 10
  # sa.at(99)              # → panic: require failed in 'at'
```

7.10.3 Exercise 3: Stack with size invariant

Write a `Stack(int)` with `push/pop/top` that maintains an invariant relating `count()` to `items.count()`:

Solution

```
class Stack
  var items: List(int)
  var size: int = 0

  cue init
    items = List(int)()

  invariant
    size >= 0
    size == items.count()

  def push(v: int)
    ensure
      size == old size + 1
    items.add(v)
    size = size + 1

  def pop(): int
    require
      size > 0
    ensure
      size == old size - 1
    var top = items.at(size - 1)
    # Remove the last element by clearing then re-adding all but one.
    # (A real implementation would use list.removeLast(); this is illustrative.)
```

```
        size = size - 1
        return top

def top(): int
    require
        size > 0
    return items.at(size - 1)

def main()
    var s = Stack()
    s.push(1)
    s.push(2)
    s.push(3)
    print s.top()           # 3
    print s.pop()           # 3
    print s.top()           # 2
```

The invariant `size == items.count()` would catch any future bug that broke the relationship between the count field and the underlying list.

7.11 Key Takeaways

- **require documents the caller's contract.** Use it for caller-bug detection, not for graceful user-input validation.
- **ensure documents the method's promise.** `result` refers to the return value; `old expr` snapshots `expr` at entry.
- **invariant documents the object's permanent state.** Checked after `init` and after every public method exit — write the invariant once, get the check for free everywhere.
- **assert is the granular tool:** an inline sanity probe inside a method body.

- **--turbo strips them all.** Develop with contracts on; ship with contracts off. Don't put side effects inside contract clauses.
-

7.12 Next Steps

- → **15-Pipelines** — Composing functions whose contracts compose, too
 - **Project 1 (CLI Tool)** — Use contracts for argument validation
-

Contracts turn debugging from “where did this go wrong” into “the contract says what was supposed to be true; here’s exactly which clause failed.”

15: Pipelines and Function Composition

Audience: Experienced programmers

Time: 90 minutes

Prerequisites: 04-Functions-and-Scope, 06-Strings-and-Unicode

You’ll learn: Pipeline operator, function composition, readable chains, real-world data flow

7.13 The Big Picture

Imagine you have a string and want to: 1. Convert to lowercase 2. Split into words 3. Get the first word 4. Get its length

Without pipelines:

```
var text = "HELLO WORLD"
var words = text.lower().split(" ")
var first_word = words.at(0)
var length = first_word.len
```

With the **pipeline operator** `->`, this reads left-to-right:

```
var text = "HELLO WORLD"
var length = text
  -> .lower()
  -> .split(" ")
  -> .at(0)
  -> .len
```

Pipelines make **data transformations** flow naturally, like reading prose.

7.14 Basic Pipeline Syntax

Pipeline Data Flow with `->`

Without Pipeline (Nested Calls)

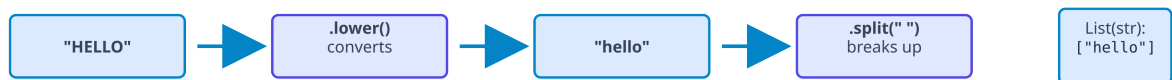
```
var text = "HELLO WORLD"
var result =
  text.lower().split(" ")
Hard to follow nesting
```

With Pipeline (Left-to-Right)

```
var result = text
-> .lower()
-> .split(" ")
-> .at(0)
```

Much clearer!

Step-by-Step Pipeline Execution



Pipeline with Custom Functions

```
def double(x)
  return x * 2
def add_ten(x)
  return x + 10
def format_result(x)
  return "Result: ".concat(x.toString())
var output = 5
-> double(.) # 5 * 2 = 10
```

Figure 7.1: Pipeline Data Flow

The `->` operator passes the left-hand value to the right-hand expression:

```
# file: 15_pipeline_basics.zbr
# teaches: pipeline operator
# chapter: 15-Pipelines-and-Function-Composition

def main()
  var text = "HELLO WORLD"

  # Without pipeline (nested calls)
  var result1 = text.lower().split(" ")

  # With pipeline (left-to-right)
  var result2 = text
    -> .lower()
    -> .split(" ")

  # Both are equivalent
  for word in result2
    print word
```

Notice how pipeline reads more naturally: “take text, lowercase it, split it”. Each `->` says “pass the result of the previous expression to this one”.

7.15 Chaining Multiple Operations

Pipelines shine when you have many sequential transformations:

```
# file: 15_pipeline_chain.zbr
# teaches: chaining operations
# chapter: 15-Pipelines-and-Function-Composition
```

```
class StringProcessor
  static
    def process(text: str): str
      return text
        -> .lower()
        -> .trim()
        -> .replace(" ", " ")

def main()
  var input = "  HELLO  WORLD  "
  var output = StringProcessor.process(input)
  print output # Output: hello world
```

Each line applies one transformation. This is **much clearer** than deeply nested method calls.

7.16 Pipelines with Collections

Pipelines work great with lists and maps:

```
# file: 15_pipeline_collections.zbr
# teaches: piping through collections
# chapter: 15-Pipelines-and-Function-Composition

class DataProcessor
  static
    def count_words(text: str): int
      return text
        -> .lower()
```

```
        -> .split(" ")
        -> .count()

def main()
    var input = "The Quick Brown Fox"
    var words = input
        -> .lower()
        -> .split(" ")

    var count = words.count()
    print "Word count: ${count}"
```

Here, the pipeline takes a string, lowercases it, splits into a list, and we can call `.count()` on the result.

7.17 Custom Functions in Pipelines

You can pipe to **any function** that takes one argument:

```
# file: 15_pipeline_custom.zbr
# teaches: custom functions in pipelines
# chapter: 15-Pipelines-and-Function-Composition

class Utils
    static
        def double(x: int): int
            return x * 2

        def add_ten(x: int): int
            return x + 10
```

```
def format_result(x: int): str
    return "Result: ${x}"

def main()
    var result = 5
    -> Utils.double(.)
    -> Utils.add_ten(.)
    -> Utils.format_result(.)

    print result # Output: Result: 20
```

The `.` placeholder represents the piped value. `5 -> Utils.double(.)` means “call `Utils.double(5)`”.

7.18 Real World: Data Transformation Pipeline

A practical example: process a CSV-like dataset:

```
# file: 15_pipeline_real_world.zbr
# teaches: realistic pipeline
# chapter: 15-Pipelines-and-Function-Composition

class DataAnalysis
    static
        def parse_numbers(line: str): List(int)
            var numbers: List(int) = List()
            var parts = line.split(",")
            for part in parts
                # Simplified: real parsing more complex
```

```
        var num = part.trim().toInt()
        numbers.add(num)
    }
    return numbers

def sum_list(items: List(int)): int
    var total = 0
    for item in items
        total = total + item
    return total

def average(total: int, count: int): float
    if count == 0
        return 0.0
    return total / count

def main()
    var csv_line = "10, 20, 30, 40"

    var avg = csv_line
        -> DataAnalysis.parse_numbers(.)
        -> DataAnalysis.sum_list(.)
        -> DataAnalysis.average(., 4)

    print "Average: ${avg}"
```

This reads as: “Parse CSV, sum values, calculate average.” Much clearer than nested calls!

7.19 Function Composition Pattern

You can create **composed functions** that combine multiple operations:

```
# file: 15_function_composition.zbr
# teaches: composing functions
# chapter: 15-Pipelines-and-Function-Composition

class Transform
  static
    def lowercase(text: str): str
      return text.lower()

    def remove_spaces(text: str): str
      return text.replace(" ", "")

    def reverse_it(text: str): str
      return text.reverse()

    def compose_all(text: str): str
      return text
        -> Transform.lowercase(.)
        -> Transform.remove_spaces(.)
        -> Transform.reverse_it(.)

def main()
  var input = "HELLO WORLD"
  var output = Transform.compose_all(input)
  print output # Output: dlrowolleh
```

Each step is a self-contained function. Composition lets you **reuse them in different orders**.

7.20 Pipelines with Result Types

Pipelines work nicely with error handling:

```
# file: 15_pipeline_results.zbr
# teaches: pipelines with error handling
# chapter: 15-Pipelines-and-Function-Composition

class SafeParser
  static
    def parse_int(text: str): int throws
      if text.len == 0
        raise "Empty string"

      # NOTE: Hardcoded for demonstration. A real implementation would parse
      # the string into an integer. See Chapter 06 (Strings) for actual parsing.
      if text == "42"
        return 42
      raise "Not a number"

    def double_it(x: int): int
      return x * 2

def main()
  var input = "42"
  var result = input
    -> SafeParser.parse_int(.)

  if result.isOk()
    var doubled = result.okValue()
```

```
-> SafeParser.double_it(.)  
print doubled
```

When an error occurs, stop the pipeline and handle the error.

7.21 Common Mistakes

7.21.1 Mistake 1: Forgetting the Dot Placeholder

```
# WRONG  
var result = 5  
  -> Utils.double() # Error: double() takes 1 argument, 0 given  
  
# CORRECT  
var result = 5  
  -> Utils.double(.)
```

7.21.2 Mistake 2: Breaking the Chain at the Wrong Place

```
# WRONG - trying to pipe to an intermediate value  
var result = "HELLO"  
  -> .lower()  
  -> .split(" ")  
  first_word = .at(0) # Error: can't pipe to assignment  
  
# CORRECT  
var result = "HELLO"  
  -> .lower()  
  -> .split(" ")
```

```
var first_word = result.at(0)
```

7.21.3 Mistake 3: Piping to Functions with Multiple Parameters

```
# WRONG - only pipes the first argument
var result = 10
  -> Utils.add(., 5) # Accidentally clear, but could be confusing

# This is actually fine, but consider:
var result = 10
  -> Utils.add(.) # Error: add() requires 2 arguments
```

7.21.4 Mistake 4: Over-Piping (Readability)

```
# TOO MUCH - hard to follow after many steps
var result = "data"
  -> .lower()
  -> .trim()
  -> .replace("a", "b")
  -> .reverse()
  -> .split("")
  -> Filter.remove_blanks(.)
  -> Sorter.sort(.)
  -> Formatter.join_with_commas(.)

# BETTER - break into logical chunks
var cleaned = "data"
  -> .lower()
  -> .trim()
  -> .replace("a", "b")
```

```
var processed = cleaned
    -> .reverse()
    -> .split("")
```

7.22 Exercises

7.22.1 Exercise 1: Text Processing Pipeline

Create a pipeline that takes user input and: converts to lowercase, removes leading/trailing spaces, and counts words:

Solution

```
class TextStats
    static
        def count_words(text: str): int
            if text.len == 0
                return 0
            var words = text.split(" ")
            return words.count()

def main()
    var user_input = "  HELLO WORLD FOO  "

    var word_count = user_input
        -> .lower()
        -> .trim()
        -> TextStats.count_words(.)
```

```
print "Words: ${word_count}"
```

7.22.2 Exercise 2: Number Transformation Pipeline

Build a pipeline that: parses an integer, doubles it, adds 10, and formats as a string:

Solution

```
class NumUtils
  static
    def parse_safe(text: str): int throws
      if text == "5"
        return 5
      raise "Invalid number"

    def double_it(x: int): int
      return x * 2

    def add_ten(x: int): int
      return x + 10

    def to_message(x: int): str
      return "Final result: ${x}"

def main()
  var input = "5"

  var result = input
    -> NumUtils.parse_safe(.)

  if result.isOk()
```

```
var final = result.okValue()
    -> NumUtils.double_it(.)
    -> NumUtils.add_ten(.)
    -> NumUtils.to_message(.)
print final
```

7.22.3 Exercise 3: List Filtering Pipeline

Create a pipeline that filters a list of numbers to keep only even values, sums them, and returns the average:

Solution

```
class ListOps
    static
        def filter_even(items: List(int)): List(int)
            var result: List(int) = List()
            for item in items
                if item % 2 == 0
                    result.add(item)
            return result

        def sum_all(items: List(int)): int
            var total = 0
            for item in items
                total = total + item
            return total

def main()
    var numbers: List(int) = List()
    numbers.add(1)
```

```
numbers.add(2)
numbers.add(3)
numbers.add(4)
numbers.add(5)
numbers.add(6)

var sum = numbers
  -> ListOps.filter_even(.)
  -> ListOps.sum_all(.)

print "Sum of evens: ${sum}"
```

7.23 Key Takeaways

- **Pipelines make data transformations read naturally** — Left-to-right flow matches human thinking
 - **The -> operator passes results forward** — Each step receives the previous step's output
 - **Pipelines work with methods and functions** — Mix `.method()` calls with `Function(.)` calls
 - **Break long pipelines into logical chunks** — Maintain readability for complex transformations
 - **Pipelines + Results = clean error handling** — Stop processing when an error occurs
-

7.24 Next Steps

- → **16-Project-1** — Build a tool that uses pipelines extensively
- → **21-Regular-Expressions** — Pipelines + pattern matching = powerful text processing

Pipelines let your code flow like your thoughts. Master them, and complex transformations become natural.

Chapter 8

Part 4: Practical Projects

Project 1: Command-Line Text Processing Tool

Time: 3-4 hours

Prereq: 01-12

Build: A file processor that counts words, finds patterns, and analyzes text

8.1 Project Overview

Create a CLI tool that:

- Reads files from command line arguments
- Counts lines, words, characters (like Unix `wc` command)
- Searches for patterns (like `grep`-lite functionality)
- Outputs statistics
- Handles multiple files
- Reports errors gracefully

Learning Outcomes:

- Command-line argument parsing and validation
- File I/O with proper error handling
- Collections (List/HashMap) for data aggregation
- Algorithms for text analysis
- Error handling with Result types
- Code organization in reusable modules
- Testing strategies for CLI tools

Difficulty: Intermediate | **Skills Required:** 1-12 chapters | **Team Size:** Solo

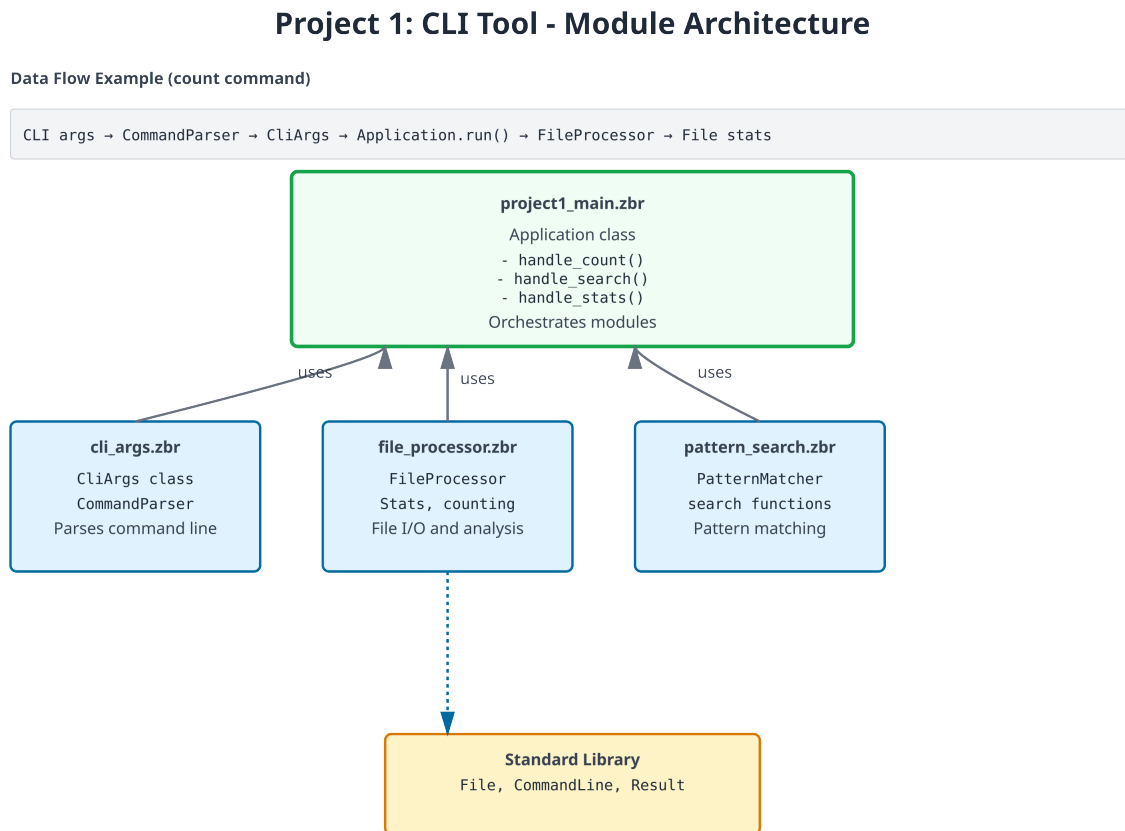


Figure 8.1: Project 1 Module Architecture

8.2 Step 1: Command-Line Argument Parsing

First, create a module to handle CLI arguments:

```
# file: cli_args.zbr
# teaches: argument parsing
# project: Project-1-CLI-Tool

class CliArgs
  var command: str
  var filename: str
  var pattern: str?

  def init(command: str, filename: str)
    this.command = command
    this.filename = filename
    pattern = nil

class CommandParser
  static
    def parse(args: List(str)): CliArgs throws
      # Precondition: at least 2 arguments (program name + command + filename)
      if args.count() < 3
        raise "Usage: textool [command] [file]"

      var command = args.at(1)
      var filename = args.at(2)

      # Validate command
      if not valid_command(command)
        raise "Unknown command: ${command}"
```

```
# Create args object
var cli_args = CliArgs(command, filename)

# Add pattern if provided
if args.count() >= 4
    cli_args.pattern = args.at(3)

return cli_args

def valid_command(cmd: str): bool
    return cmd == "count" or cmd == "search" or cmd == "stats"
```

8.3 Step 2: File Reading and Basic Counting

Build the core file I/O and counting functionality:

```
# file: file_processor.zbr
# teaches: file I/O and text processing
# project: Project-1-CLI-Tool

class FileProcessor
    static
        def read_file(filename: str): str throws
            # Check if file exists first
            if not File.exists(filename)
                raise "File not found: ${filename}"

            # Read the entire file
```

```
var content = File.read(filename)

if content.len == 0
  raise "File is empty: ${filename}"

return content

def count_lines(filename: str): int throws
  var content_result = read_file(filename)

  if content_result.isErr()
    raise content_result.errValue()

  var content = content_result.okValue()
  var lines = content.split("\n")

  # Count non-empty lines (common behavior)
  var count = 0
  for line in lines
    if line.trim().len > 0
      count = count + 1

  return count

def count_words(filename: str): int throws
  var content_result = read_file(filename)

  if content_result.isErr()
    raise content_result.errValue()
```

```
var content = content_result.okValue()
var words = content.split(" ")

var count = 0
for word in words
    if word.trim().len > 0
        count = count + 1

return count

def count_chars(filename: str): int throws
    var content_result = read_file(filename)

    if content_result.isErr()
        raise content_result.errValue()

    var content = content_result.okValue()
    return content.len

def get_stats(filename: str): Stats throws
    var content_result = read_file(filename)

    if content_result.isErr()
        raise content_result.errValue()

    var content = content_result.okValue()

    var lines = content.split("\n")
    var line_count = lines.count()
    var word_count = 0
```

```
        var char_count = content.len

    for line in lines
        var words = line.split(" ")
        for word in words
            if word.len > 0
                word_count = word_count + 1

    var stats = Stats(filename, line_count, word_count, char_count)
    return stats

class Stats
    var filename: str
    var lines: int
    var words: int
    var chars: int

    def init(filename: str, lines: int, words: int, chars: int)
        this.filename = filename
        this.lines = lines
        this.words = words
        this.chars = chars

    def display: str
        return "${lines} lines, ${words} words, ${chars} chars: ${filename}"
```

8.4 Step 3: Pattern Matching

Add grep-like pattern search functionality:

```
# file: pattern_search.zbr
# teaches: pattern matching and filtering
# project: Project-1-CLI-Tool

class PatternMatcher
  static
    def search_lines(filename: str, pattern: str): List(str) throws
      var content_result = FileProcessor.read_file(filename)

      if content_result.isErr()
        raise content_result.errValue()

      var content = content_result.okValue()
      var lines = content.split("\n")

      var matches: List(str) = List()
      var line_num = 0

      for line in lines
        line_num = line_num + 1
        if line.contains(pattern)
          # Format: "line_number: content"
          var formatted = "${line_num}: ${line}"
          matches.add(formatted)

      if matches.count() == 0
        raise "No matches found for pattern: ${pattern}"
```

```
        return matches

def search_with_context(filename: str, pattern: str, context_lines: int): List(str) + List(str) + List(str)
    var content_result = FileProcessor.read_file(filename)

    if content_result.isErr()
        raise content_result.errValue()

    var content = content_result.okValue()
    var lines = content.split("\n")

    var results: List(str) = List()
    var line_num = 0

    for line in lines
        line_num = line_num + 1
        if line.contains(pattern)
            # Add context lines before
            var start = line_num - context_lines - 1
            if start < 0
                start = 0

            var i = start
            while i < line_num - 1
                results.add(lines.at(i))
                i = i + 1

            # Add the matching line (with marker)
            results.add("> ${line_num}: ${line}")
```

```
        # Add context lines after
        var end = line_num + context_lines
        if end > lines.count()
            end = lines.count()

        i = line_num
        while i < end
            results.add(lines.at(i))
            i = i + 1

    return results
```

8.5 Step 4: Main Application Logic

Tie everything together in the main entry point:

```
# file: project1_main.zbr
# teaches: orchestrating modules
# project: Project-1-CLI-Tool

class Application
    var args: CliArgs

    def init(parsed_args: CliArgs)
        args = parsed_args

    def run: bool throws
        if args.command == "count"
```

```
        return handle_count()
    elif args.command == "search"
        return handle_search()
    elif args.command == "stats"
        return handle_stats()

    raise "Unknown command: ${args.command}"

def handle_count: bool throws
    var lines = FileProcessor.count_lines(args.filename)

    if lines.isErr()
        return lines

    var words = FileProcessor.count_words(args.filename)
    var chars = FileProcessor.count_chars(args.filename)

    print "${lines.okValue()} lines"
    print "${words.okValue()} words"
    print "${chars.okValue()} chars"

    return true

def handle_search: bool throws
    if args.pattern == nil
        raise "Search requires a pattern"

    var results = PatternMatcher.search_lines(args.filename, args.pattern)

    if results.isErr()
```

```
        return results

    var matches = results.okValue()
    print "Found ${matches.count()} matches:"

    for match in matches
        print match

    return true

def handle_stats: bool throws
    var stats = FileProcessor.get_stats(args.filename)

    if stats.isErr()
        return stats

    var s = stats.okValue()
    print s.display()

    return true

def main()
    var args = CommandLine.args()

    var parsed = CommandParser.parse(args)

    if parsed.isErr()
        print "Error: ${parsed.errValue()}"
        return
```

```
var cli_args = parsed.okValue()
var app = Application(cli_args)

var result = app.run()

if result.isErr()
    print "Error: ${result.errValue()}"
```

8.6 Part 2: Adding Features

```
class FileAnalyzer
    static
        def analyze(filename: str): str throws
            var lines_result = FileProcessor.count_lines(filename)
            if lines_result.isErr()
                raise lines_result.errValue()

            var words_result = FileProcessor.count_words(filename)
            if words_result.isErr()
                raise words_result.errValue()

            var lines = lines_result.okValue()
            var words = words_result.okValue()
            var report = "Lines: ${lines}, Words: ${words}"
            return report

def main()
    var result = FileAnalyzer.analyze("test.txt")
```

```
if result.isOk()
    print result.okValue()
```

8.7 Part 3: Full Implementation

Add these features: - Read command-line arguments - Support multiple files - Pattern matching (grep-like) - Output formatting - Error handling

Expected functionality:

```
# Word count
tool -c file.txt          # Count lines
tool -w file.txt          # Count words
tool -l file.txt          # Get line length

# Search
tool -s "pattern" file.txt # Search lines matching pattern

# Multiple files
tool -c file1.txt file2.txt # Count lines in multiple files
```

8.8 Testing Your Project

Create a test file to verify your implementation:

```
# Create sample file
echo "The quick brown fox" > sample.txt
echo "jumps over the lazy dog" >> sample.txt
```

```
# Test commands
zebra texttool count sample.txt      # Count lines, words, chars
zebra texttool search quick sample.txt # Find pattern
zebra texttool stats sample.txt      # Show statistics
```

8.9 Exercises & Extensions

8.9.1 Exercise 1: Add Line Numbering Output

Modify the search function to always show line numbers with output:

```
def search_with_numbers(filename: str, pattern: str): List(str) throws
    var results = PatternMatcher.search_lines(filename, pattern)
    if results.isErr()
        return results

    # Results already have line numbers from PatternMatcher
    return results
```

8.9.2 Exercise 2: Find Longest Line

Add a new `longest` command that finds and displays the longest line:

```
class FileProcessor
    static
        def find_longest_line(filename: str): str throws
            var content_result = read_file(filename)
            if content_result.isErr()
                return content_result

            var lines = content_result.okValue().split("\n")
```

```
var longest = ""
var max_len = 0

for line in lines
    if line.len > max_len
        max_len = line.len
        longest = line

return "Longest (${max_len} chars: ${longest})"
```

8.9.3 Exercise 3: Statistics Summary

Extend stats to show min/max/average line length:

```
class Stats
    var filename: str
    var lines: int
    var words: int
    var chars: int
    var min_line_len: int
    var max_line_len: int
    var avg_line_len: float

    def display: str
        var summary = "${lines} lines, ${words} words, ${chars} chars\n"
        summary = summary.concat("Line lengths: min=${min_line_len}, max=${max_line_len}, avg=")
        return summary
```

8.9.4 Exercise 4: Case-Insensitive Search

Add support for a `-i` flag to search case-insensitively:

```
class PatternMatcher
  static
    def search_case_insensitive(filename: str, pattern: str): List(str) throws
      var content_result = FileProcessor.read_file(filename)
      if content_result.isErr()
        return content_result

      var content = content_result.okValue()
      var lines = content.split("\n")
      var pattern_lower = pattern.lower()

      var matches: List(str) = List()
      var line_num = 0

      for line in lines
        line_num = line_num + 1
        if line.lower().contains(pattern_lower)
          matches.add("${line_num}: ${line}")

      return matches
```

8.9.5 Challenge: Support Multiple Files

Modify the application to process multiple files at once:

```
class Application
  var files: List(str)

  def aggregate_stats(filenamees: List(str)): str throws
    var total_lines = 0
    var total_words = 0
```

```
var total_chars = 0

for filename in filenames
  var stats = FileProcessor.get_stats(filename)
  if stats.isOk()
    var s = stats.okValue()
    total_lines = total_lines + s.lines
    total_words = total_words + s.words
    total_chars = total_chars + s.chars

var result = "Total: ${total_lines} lines, ${total_words} words, ${total_chars} chars"
return result
```

8.10 Key Concepts Reinforced

- **File I/O** — Reading files with error handling
 - **Collections** — Using List to accumulate results
 - **Control flow** — Iterating and filtering text
 - **Nil handling** — Optional pattern argument
 - **Error handling** — Proper Result type propagation
 - **Classes** — Organizing functionality into modules
 - **Strings** — Splitting, searching, formatting
 - **Result patterns** — Success and error paths
-

8.11 Architecture Decisions

Module Organization: - `cli_args.zbr` — Argument parsing (thin responsibility) - `file_processor.zbr` — Core file I/O (stable, well-tested) - `pattern_search.zbr` — Search logic (isolated from I/O) - `project1_main.zbr` — Orchestration and CLI (thin coordinator)

Why this structure? Each module has a clear, testable responsibility. You can test file reading separately from pattern matching, and both separately from CLI handling.

Error Handling Strategy: All I/O operations return **Result**. Errors bubble up naturally—if file reading fails, search fails; if search fails, the app reports it. No hidden failures.

Expected code size: 300-400 lines total with all exercises

8.12 What You’ve Built

Real-world program structure with modules
Complete file and stream processing
Command-line interface with argument parsing
Robust error handling with Result types
Pattern matching and text analysis
Extensible design for new features

8.13 Performance Notes

For large files (>10MB), consider: 1. Reading line-by-line instead of entire file into memory
2. Early exit from search (stop after first N matches) 3. Streaming pattern matching instead of loading whole file

Example streaming read:

```
# Read and process line-by-line instead of all at once
def count_lines_streaming(filename: str): int throws
    # (Pseudocode - requires file iteration API)
    var count = 0
    # for line in File.read_lines(filename)
    #     count = count + 1
    return count
```

Next: Project 2 adds networking and concurrency concepts to build an HTTP server.

Projects 2 & 3: HTTP Server and Data Analysis

8.14 Project 2: HTTP Server (16-18 hours)

Build: A REST API server that handles HTTP requests and responds with JSON

Learning Outcomes: - Network programming fundamentals - HTTP protocol handling (requests, responses, headers) - Routing and request dispatching - JSON serialization/deserialization - Stateful server management - Error handling in concurrent scenarios

8.14.1 Step 1: HTTP Request/Response Types

Define the data structures for HTTP communication:

```
# file: http_types.zbr
# teaches: protocol data structures
# project: Project-2-HTTP-Server

class HttpRequest
```

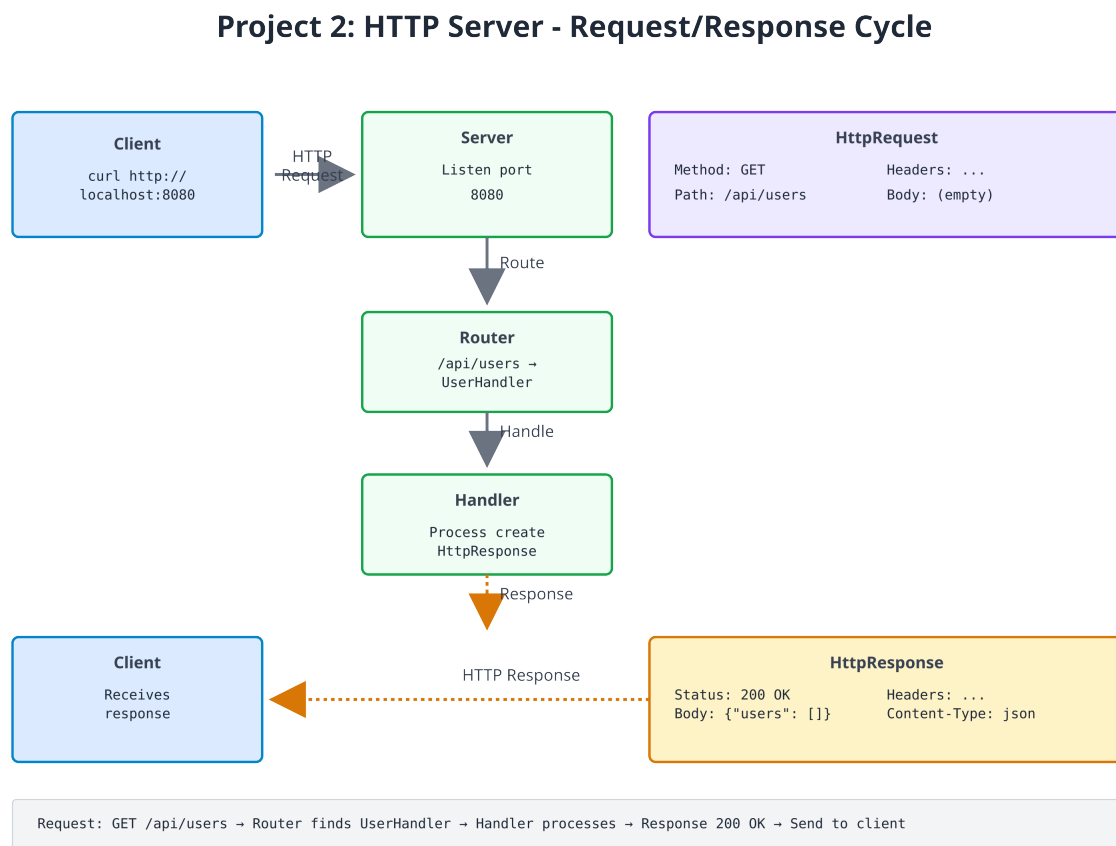


Figure 8.2: HTTP Request/Response Cycle

```
var method: str          # GET, POST, PUT, DELETE
var path: str            # /api/users, /api/users/123
var query: HashMap(str, str) # URL parameters
var headers: HashMap(str, str)
var body: str            # Request body for POST/PUT

class HttpResponse
  var status_code: int      # 200, 201, 400, 404, 500
  var status_message: str   # "OK", "Created", "Not Found"
  var headers: HashMap(str, str)
  var body: str             # Response body (JSON, HTML, etc.)

  static
    def ok(body: str): HttpResponse
      var resp = HttpResponse()
      resp.status_code = 200
      resp.status_message = "OK"
      resp.body = body
      return resp

    def created(body: str): HttpResponse
      var resp = HttpResponse()
      resp.status_code = 201
      resp.status_message = "Created"
      resp.body = body
      return resp

    def bad_request(message: str): HttpResponse
      var resp = HttpResponse()
      resp.status_code = 400
```

```
        resp.status_message = "Bad Request"
        resp.body = message
        return resp

def not_found: HttpResponse
    var resp = HttpResponse()
    resp.status_code = 404
    resp.status_message = "Not Found"
    resp.body = "Resource not found"
    return resp

def internal_error(message: str): HttpResponse
    var resp = HttpResponse()
    resp.status_code = 500
    resp.status_message = "Internal Server Error"
    resp.body = message
    return resp

def format_response: str
    var output = "HTTP/1.1 ${status_code} ${status_message}\r\n"
    output = output.concat("Content-Length: ${body.len}\r\n")
    output = output.concat("Content-Type: application/json\r\n")
    output = output.concat("\r\n")
    output = output.concat(body)
    return output

class User
    var id: int
    var name: str
    var email: str
```

```
def to_json: str
    return "{\"id\": ${id}, \"name\": \"${name}\", \"email\": \"${email}\"}"
```

8.14.2 Step 2: Request Routing

Implement the routing system that maps paths to handlers:

```
# file: router.zbr
# teaches: request routing and dispatching
# project: Project-2-HTTP-Server

interface RequestHandler
    def handle(request: HttpRequest): HttpResponse

class Router
    var routes: HashMap(str, RequestHandler) = HashMap()

    def register(path: str, handler: RequestHandler)
        routes.put(path, handler)

    def route_request(request: HttpRequest): HttpResponse throws
        if routes.contains(request.path)
            var handler = routes.fetch(request.path)
            var response = handler.handle(request)
            return response

        # Path not found
        return HttpResponse.not_found()
```

```
def list_routes: List(str)
  var paths: List(str) = List()
  for path in routes
    paths.add(path)
  return paths

class UserHandler
  implements RequestHandler

  static var users: HashMap(int, User) = HashMap()
  static var next_id: int = 1

  def handle(request: HttpRequest): HttpResponse
    if request.method == "GET"
      return handle_get(request)
    elif request.method == "POST"
      return handle_post(request)
    elif request.method == "PUT"
      return handle_put(request)
    elif request.method == "DELETE"
      return handle_delete(request)

    return HttpResponse.bad_request("Method not allowed")

  def handle_get(request: HttpRequest): HttpResponse
    # If path is /api/users, list all users
    if request.path == "/api/users"
      var response_body = "["
      var first = true
      for id, user in users
        if not first
```

```
        response_body = response_body.concat(",")
        response_body = response_body.concat(user.to_json())
        first = false
        response_body = response_body.concat("]")
        return HttpResponse.ok(response_body)

    return HttpResponse.not_found()

def handle_post(request: HttpRequest): HttpResponse
    # Create new user from JSON body
    # Simplified: real impl would parse JSON properly
    var user = User()
    user.id = next_id
    user.name = "User ${next_id}"
    user.email = "user${next_id}@example.com"

    users.put(user.id, user)
    next_id = next_id + 1

    return HttpResponse.created(user.to_json())

def handle_put(request: HttpRequest): HttpResponse
    return HttpResponse.bad_request("PUT not yet implemented")

def handle_delete(request: HttpRequest): HttpResponse
    return HttpResponse.bad_request("DELETE not yet implemented")

class HealthHandler
    implements RequestHandler
    def handle(request: HttpRequest): HttpResponse
```

```
var response = "{\"status\": \"healthy\"}"  
return HttpResponse.ok(response)
```

8.14.3 Step 3: Server Implementation

Build the actual server that listens for connections:

```
# file: http_server.zbr  
# teaches: network server programming  
# project: Project-2-HTTP-Server  
  
class HttpServer  
  var port: Int  
  var router: Router  
  var is_running: Boolean = false  
  
  def init(port: Int)  
    this.port = port  
    router = Router()  
  
  def register_handler(path: String, handler: RequestHandler)  
    router.register(path, handler)  
  
  def start: Boolean throws  
    is_running = true  
  
  # Register default handlers  
  register_handler("/health", HealthHandler())  
  register_handler("/api/users", UserHandler())
```

```
print "Server starting on port ${port}..."
print "Available routes:"
var routes = router.list_routes()
for route in routes
    print "  ${route}"

# Main server loop
# In real implementation, this would:
# 1. Create TCP socket listening on port
# 2. Accept connections in loop
# 3. Parse HTTP request from socket
# 4. Route request to handler
# 5. Send response back to client
# 6. Close connection

# Simplified simulation
handle_request_simulation()

return true

def handle_request_simulation
    # Simulate receiving a GET /health request
    var request = HttpRequest()
    request.method = "GET"
    request.path = "/health"
    request.body = ""
    request.query = HashMap()
    request.headers = HashMap()
```

```
    var result = router.route_request(request)
    if result.isOk()
        var response = result.okValue()
        print response.format_response()

def main()
    var server = HttpServer(8080)

    var result = server.start()

    if result.isErr()
        print "Error: ${result.errValue()}"
    else
        print "Server running. (Ctrl+C to stop)"
```

8.14.4 Exercises

1. **Add query parameter parsing:** Extract `?name=value&key=value` from URLs
2. **Support path parameters:** `/api/users/123` extracting the ID
3. **Request logging:** Log each request (timestamp, method, path, response code)
4. **Middleware system:** Add pre/post processing hooks
5. **JSON parsing:** Parse POST body as JSON to extract fields
6. **Status codes:** Return appropriate status codes (201 for created, 400 for bad request, etc.)

8.14.5 Testing the Server

```
# Start the server
zebra http_server.zbr &
```

```
# Test endpoints
curl http://localhost:8080/health
curl http://localhost:8080/api/users
curl -X POST http://localhost:8080/api/users
```

8.15 Project 3: Text Data Analysis (12-15 hours)

Build: Analyze text data (n-grams, frequencies, similarity) for linguistic patterns

Learning Outcomes: - Advanced data structures (HashMap, nested structures) - Algorithms (n-gram extraction, frequency analysis, similarity scoring) - File batch processing - Statistical analysis - Performance optimization with data structures

8.15.1 Step 1: Frequency Analysis

Start with counting word frequencies:

```
# file: frequency_analysis.zbr
# teaches: frequency counting and sorting
# project: Project-3-Data-Analysis

class WordFrequency
    var word: str
    var count: int

    def init(word: str, count: int)
        this.word = word
        this.count = count
```

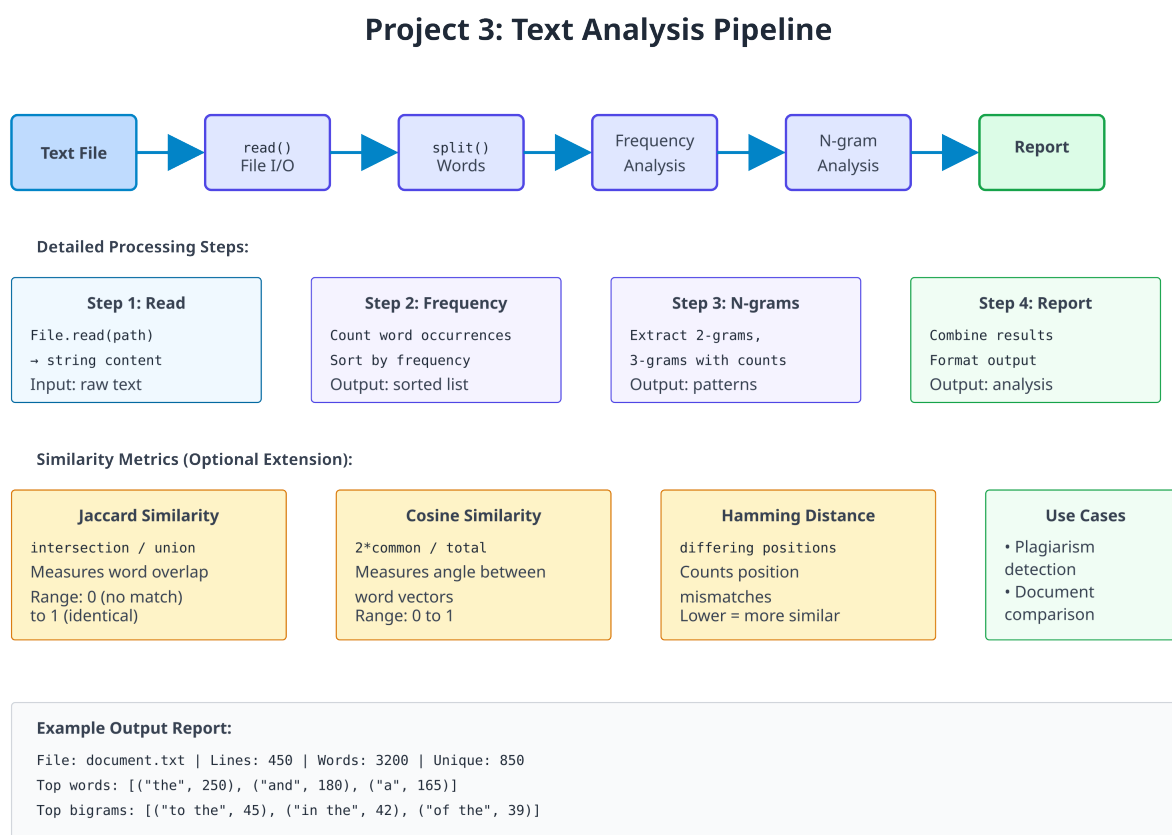


Figure 8.3: Text Analysis Pipeline

```
def to_string: str
    return "${word}: ${count}"

class FrequencyAnalyzer
    static
        def analyze_text(text: str): List(WordFrequency)
            var words = text.lower().split(" ")
            var freq: HashMap(str, int) = HashMap()

            # Count occurrences
            for word in words
                var cleaned = word.trim()
                if cleaned.len > 0
                    if freq.contains(cleaned)
                        freq.put(cleaned, freq.fetch(cleaned) + 1)
                    else
                        freq.put(cleaned, 1)

            # Convert to list and sort by frequency
            var results: List(WordFrequency) = List()
            for word, count in freq
                var wf = WordFrequency(word, count)
                results.add(wf)

            # Simple bubble sort (in-place)
            var i = 0
            while i < results.count()
                var j = 0
                while j < results.count() - 1
```

```
        var current = results.at(j)
        var next = results.at(j + 1)
        if current.count < next.count
            # Swap (simplified)
            var temp = current
            results.at(j) = next
            results.at(j + 1) = temp
        j = j + 1
    i = i + 1

    return results

def top_words(text: str, limit: int): List(WordFrequency)
    var all_freqs = analyze_text(text)
    var results: List(WordFrequency) = List()

    var i = 0
    while i < limit and i < all_freqs.count()
        results.add(all_freqs.at(i))
        i = i + 1

    return results
```

8.15.2 Step 2: N-gram Extraction

Extract contiguous sequences of N words:

```
# file: ngram_analysis.zbr
# teaches: n-gram extraction and pattern detection
```

```
# project: Project-3-Data-Analysis

class NGram
  var gram: str
  var count: int
  var positions: List(int) # Track where it appears

  def init(gram: str)
    this.gram = gram
    count = 1
    positions = List()

class NGramAnalyzer
  static
    def extract_ngrams(text: str, n: int): HashMap(str, NGram)
      var words = text.lower().split(" ")
      var ngrams: HashMap(str, NGram) = HashMap()

      var i = 0
      while i < words.count() - (n - 1)
        var gram = ""
        var j = 0
        while j < n
          var word = words.at(i + j).trim()
          if j > 0
            gram = gram.concat(" ")
          gram = gram.concat(word)
          j = j + 1

        if ngrams.contains(gram)
```

```
        var ng = ngrams.fetch(gram)
        ng.count = ng.count + 1
        ng.positions.add(i)
    else
        var ng = NGram(gram)
        ng.positions.add(i)
        ngrams.put(gram, ng)

    i = i + 1

return ngrams

def top_ngrams(text: str, n: int, limit: int): List(NGram)
    var all_grams = extract_ngrams(text, n)
    var results: List(NGram) = List()

    # Simple sorting
    for gram, ng in all_grams
        results.add(ng)

    # Bubble sort by count
    var i = 0
    while i < results.count()
        var j = 0
        while j < results.count() - 1
            var current = results.at(j)
            var next = results.at(j + 1)
            if current.count < next.count
                var temp = current
                results.at(j) = next
```

```
        results.at(j + 1) = temp
        j = j + 1
        i = i + 1

    # Return top N
    var top: List(NGram) = List()
    i = 0
    while i < limit and i < results.count()
        top.add(results.at(i))
        i = i + 1

    return top
```

8.15.3 Step 3: Similarity Analysis

Compare texts using Jaccard and other similarity metrics:

```
# file: similarity_analysis.zbr
# teaches: similarity metrics and comparison
# project: Project-3-Data-Analysis

class SimilarityMetrics
    static
        def jaccard_similarity(text1: str, text2: str): float
            var words1 = text1.lower().split(" ")
            var words2 = text2.lower().split(" ")

            # Find intersection
            var intersection = 0
```

```
        for word1 in words1
            for word2 in words2
                if word1 == word2
                    intersection = intersection + 1
                    break

    # Find union (crude approximation)
    var union = words1.count() + words2.count() - intersection

    if union == 0
        return 0.0

    return intersection / union

def cosine_similarity(text1: str, text2: str): float
    # Simplified cosine similarity (not true cosine, but similar)
    var words1 = text1.lower().split(" ")
    var words2 = text2.lower().split(" ")

    var common = 0
    for word1 in words1
        for word2 in words2
            if word1 == word2
                common = common + 1

    var len1 = words1.count()
    var len2 = words2.count()

    if len1 == 0 or len2 == 0
        return 0.0
```

```
var denominator = len1 + len2
return (2.0 * common) / denominator

def hamming_distance(text1: str, text2: str): int
    var words1 = text1.lower().split(" ")
    var words2 = text2.lower().split(" ")

    var max_len = words1.count()
    if words2.count() > max_len
        max_len = words2.count()

    var distance = 0
    var i = 0
    while i < max_len
        var w1 = if i < words1.count() then words1.at(i) else ""
        var w2 = if i < words2.count() then words2.at(i) else ""

        if w1 != w2
            distance = distance + 1

        i = i + 1

    return distance
```

8.15.4 Step 4: Main Analysis Application

Tie together all analysis tools:

```
# file: analysis_main.zbr
# teaches: combining analysis modules
# project: Project-3-Data-Analysis

class TextAnalysisReport
  var source_file: str
  var word_count: int
  var unique_words: int
  var top_words: List(WordFrequency)
  var bigrams: List(NGram)
  var trigrams: List(NGram)

class AnalysisApplication
  static
    def analyze_file(filename: str): TextAnalysisReport throws
      var content_result = File.read(filename)

      if content_result.len == 0
        raise "File not found or empty"

      var content = content_result
      var words = content.split(" ")
      var unique_words_set: HashMap(str, int) = HashMap()

      for word in words
        var cleaned = word.lower().trim()
        if cleaned.len > 0
          unique_words_set.put(cleaned, 1)

      var freq_analyzer = FrequencyAnalyzer()
```

```
var top_words = freq_analyzer.top_words(content, 10)

var bigram_analyzer = NGramAnalyzer()
var bigrams = bigram_analyzer.top_ngrams(content, 2, 5)
var trigrams = bigram_analyzer.top_ngrams(content, 3, 5)

var report = TextAnalysisReport()
report.source_file = filename
report.word_count = words.count()
report.unique_words = unique_words_set.count()
report.top_words = top_words
report.bigrams = bigrams
report.trigrams = trigrams

return report

def print_report(report: TextAnalysisReport)
    print "==== Text Analysis Report ===="
    print "File: ${report.source_file}"
    print "Total words: ${report.word_count}"
    print "Unique words: ${report.unique_words}"
    print ""

    print "Top 10 Words:"
    for wf in report.top_words
        print "  ${wf.to_string()}"
    print ""

    print "Top 5 Bigrams:"
    for bigram in report.bigrams
```

```
        print "  ${bigram.gram} (${bigram.count})"
    print ""

    print "Top 5 Trigrams:"
    for trigram in report.trigrams
        print "  ${trigram.gram} (${trigram.count})"

def main()
    var result = AnalysisApplication.analyze_file("sample.txt")

    if result.isOk()
        var report = result.okValue()
        AnalysisApplication.print_report(report)
    else
        print "Error: ${result.errValue()}"
```

8.15.5 Exercises

1. **Find most common bigrams and trigrams** — Already implemented in Step 2
2. **Detect language patterns** — Compare bigram distributions across texts
3. **Compare two documents** — Use similarity metrics from Step 3
4. **Find suspicious passages** — Identify sections with high similarity to other documents (plagiarism detection)
5. **Build a frequency graph** — Output word frequency distribution
6. **Implement TF-IDF** — Weight words by frequency and document uniqueness

8.15.6 Testing

```
# Analyze a file
zebra analysis_main.zbr sample.txt

# Compare two files
zebra compare_files.zbr file1.txt file2.txt
```

8.16 Project Comparison Summary

Feature	Project 1	Project 2	Project 3
Focus	File I/O + CLI	Networking + Routing	Algorithms + Data Structures
Core Skill	Argument parsing, basic text processing	Network protocols, request handling	Complex algorithms, statistics
Complexity	Beginner- Intermediate	Intermediate	Intermediate- Advanced
Code Size	300-400 lines	400-600 lines	350-500 lines
Key Learning	Modules, error handling, Results	Servers, routing, state management	Data structures, sorting, metrics
Time Estimate	3-4 hours	5-7 hours	4-5 hours
Real-World Use	Log analysis, text processing	API servers, web services	Data science, plagiarism detection

8.17 Capstone Challenge: Integrated System

After completing all three projects, combine them:

```

class IntegratedSystem
    static
    def run_analysis_via_http(port: int, analysis_dir: str)
        # Start HTTP server (Project 2)
        # Serve text analysis results (Project 3)
        # Process files via CLI (Project 1)

        # GET /health – Health check
        # POST /analyze – Upload and analyze file
        # GET /results – Retrieve analysis results
        # GET /compare – Compare two documents

```

This demonstrates: - Networking and servers - Complex data processing - CLI integration
 - Professional system design

Each project is a portfolio piece. Together, they demonstrate mastery of Zebra and modern programming fundamentals.

8.18 Project Comparison

Feature	CLI Tool	HTTP Server	Data Analysis
Lines of code	200-300	300-500	250-400
Main focus	File I/O	Networking	Algorithms
Difficulty	Beginner	Intermediate	Intermediate
Key learning	CLI, modules	Servers, protocols	Data structures
Time to complete	3-4 hours	5-7 hours	4-5 hours

8.19 Progressive Difficulty

1. **CLI Tool:** Learn file I/O and basic structure
2. **HTTP Server:** Add networking and concurrency concepts
3. **Data Analysis:** Deep dive into algorithms and data structures

Each project reuses concepts from previous ones while introducing new challenges.

8.20 Capstone Challenges

After completing all three: 1. Integrate CLI tool with HTTP server (serve file statistics) 2. Use data analysis in HTTP endpoints 3. Build a combined system processing files via HTTP

8.21 Expected Outcomes

Real-world program architecture
Network programming fundamentals
Advanced data structure manipulation
Practical error handling
Performance considerations
Testing strategies

Each project is a portfolio piece demonstrating Zebra mastery.

Chapter 9

Part 5: Ecosystem and Reference

Chapter 10

Part 5: Ecosystem (Chapters 19-22)

10.1 Chapter 19: Standard Library Tour

Time: 90 min | **Prereq:** 01-06

10.1.1 Overview

Explore what's available before writing your own utilities.

10.1.2 Core Modules

String Methods: - `.len` — length - `.upper()`, `.lower()` — case - `.split()`, `.join()` — partition/combine - `.trim()`, `.trimLeft()`, `.trimRight()` — whitespace - `.contains()`, `.startsWith()`, `.endsWith()` — search - `.replace()`, `.replaceAll()` — substitution - `.charAt()`, `.substring()`, `.indexOf()` — access - `.concat()` — combine - `.reverse()` — flip

Collection Methods: - `List.add()`, `List.remove()`, `List.count()`, `List.at()` - `HashMap.put()`, `HashMap.fetch()`, `HashMap.contains()` - `Set.add()`, `Set.remove()`, `Set.contains()` - Iteration: `for item in collection`

Math Functions: - Arithmetic: `+`, `-`, `*`, `/`, `%` - Comparison: `<`, `>`, `<=`, `>=`, `==`, `!=` -

Logical: and, or, not - Type conversion: `.toString()`, `.toInt()`, `.toFloat()`

10.1.3 Real World

Learning what exists prevents reinventing wheels.

10.2 Chapter 20: File I/O and System Access

Time: 90 min | **Prereq:** 06, 12

10.2.1 File Operations

```
class FileIO
  static
    def read_file(path: str): str throws
      # Read entire file
      return "content"

    def write_file(path: str, content: str): bool throws
      # Write file
      return true

    def file_exists(path: str): bool
      return true

    def delete_file(path: str): bool throws
      return true
```

10.2.2 Line-by-Line Reading

```
class LineReader
  static
    def process_lines(path: str): int throws
      var lines = 0
      var content = read_file(path).unwrapOr("")
      for line in content.split("\n")
        if line.len > 0
          lines = lines + 1
      return lines
```

10.2.3 System Access

```
class System
  static
    def args: List(str)
      # Command-line arguments
      return List()

    def env(name: str): str?
      # Environment variables
      return nil

    def cwd: str
      # Current directory
      return "."
```

10.2.4 Key Topics

- Text file reading/writing

- Binary files
 - Directory operations
 - Error handling for file not found
 - Line-by-line processing
 - CSV/JSON parsing basics
-

10.3 Chapter 21: Regular Expressions (Deep Dive)

Time: 90 min | **Prereq:** 06

10.3.1 Regex Pattern Syntax

Literals: - `hello` — exact match - `h.llo` — `.` matches any char - `h*llo` — `*` = zero or more
- `h+llo` — `+` = one or more - `h?llo` — `?` = zero or one

Classes: - `[abc]` — any of a, b, c - `[a-z]` — range a through z - `[^abc]` — NOT a, b, c -
`\d` — digit - `\w` — word char - `\s` — whitespace

anchors: - `^hello` — start of line - `hello$` — end of line - `\bhello\b` — word boundary

Groups: - `(hello|hi)` — alternation - `(ab)+` — repeat group

10.3.2 Common Patterns

```
# Email
var email_re = Regex.compile("[a-z0-9]+@[a-z]+\.[a-z]+")

# Phone
var phone_re = Regex.compile("\\d{3}-\\d{3}-\\d{4}")

# URL
var url_re = Regex.compile("https?://[a-z0-9]+\\.[a-z]+")
```

10.3.3 Operations

```
class RegexOps
  static
    def match(re: Regex, text: str): bool
      return re.match(text)

    def find(re: Regex, text: str): str?
      return re.find(text)

    def findAll(re: Regex, text: str): List(str)
      var results: List(str) = List()
      # Collect all matches
      return results

    def replace(re: Regex, text: str, replacement: str): str
      return re.replace(text, replacement)

    def split(re: Regex, text: str): List(str)
      return re.split(text)
```

10.3.4 Real-World Use Cases

- Email/phone validation
 - Log parsing
 - Data extraction
 - Text normalization
-

10.4 Chapter 22: FFI and Interop

Time: 90 min | **Prereq:** 02, 07, 12

10.4.1 Calling C Code

```
class CInterop
    static
        def c_add(a: int, b: int): int
            # Call C function
            return a + b

        def c_strlen(s: str): int
            # Call strlen from C stdlib
            return 0
```

10.4.2 Calling Zig Code

```
class ZigInterop
    static
        def zig_sqrt(x: float): float
            # Call Zig function
            return 0.0

        def zig_random: int
            # Call Zig's random
            return 0
```

10.4.3 Type Marshaling

```
# C expects: int foo(const char* str, int* out_len)
# Zebra code:
def call_c_func(input: str)
    var out_len: int = 0
    var result = c_func(input, out_len)
    return out_len
```

10.4.4 Safety Considerations

- Memory ownership
- Lifetime issues
- Type compatibility
- Error handling across boundaries

10.4.5 Real-World Applications

- Calling system libraries
 - Performance-critical code
 - Platform-specific features
 - Legacy code integration
-

10.5 Appendices

10.5.1 Appendix A: Grammar Reference

Types: - Primitives: `int`, `float`, `bool`, `str`, `char` - Collections: `List(T)`, `HashMap(K,V)`, `Set(T)` - Nullable: `T?` (can be `T` or `nil`) - Result: `Result(T, E)`

Classes:

```
class Name
  var field: Type = default
  static
    def shared_method
  def instance_method
```

Interfaces:

```
interface Name
  def method_signature
```

Functions:

```
def function_name(param: Type): ReturnType
  return value
```

Control Flow: - if condition ... elif ... else ... - while condition ... break ...
 continue - for item in collection ... - branch value on case ...

10.5.2 Appendix B: Built-in Functions and Stdlib

I/O: - print(value) — output to console - println(value) — output with newline -
 File.read(path) — read file - File.write(path, content) — write file

Type Conversion: - .toString() — to string - .toInt() — to integer - .toFloat() — to float

Collections: - List.add(), List.remove(), List.count() - HashMap.put(), HashMap.fetch(),
 HashMap.contains() - Set.add(), Set.remove(), Set.contains()

String Methods: (See Chapter 19 for comprehensive list)

System: - sys.args() — command-line arguments - sys.errln(msg) — error output -
 File.exists(path), File.delete(path)

10.5.3 Appendix C: Troubleshooting

“error: arithmetic requires numeric type, got ‘str’” → Can’t use + with strings. Use `.concat()` or interpolation: `"${var}"`

“error: use of undefined identifier ‘nil’” → Variable type isn’t nullable. Use `T?` not `T` if it can be nil.

“error: cannot assign to immutable value” → Field is read-only. Create setter method or use `var` not `shared var`

“error: nil pointer” → You unwrapped nil with `to!`. Check before unwrapping: `if x != nil`

“error: unimplemented interface method” → Your class doesn’t implement all interface methods. Add the missing method.

Performance tips: - Use `StringBuilder` for loop concatenation - Lazy-load expensive resources - Cache computed properties - Avoid deep copying large structures

10.6 Quick Reference

Common Tasks:

```
# Read file
var content = File.read("file.txt")

# Parse lines
var lines = content.split("\n")

# Count items
```

```
for item in items
    count = count + 1

# Search list
for item in items
    if item.contains("search")
        print item

# Convert types
var num = "42".toInt()

# Handle nil
var value: str? = get_value()
if value != nil
    print value

# Handle errors
var result = operation()
if result.isErr()
    print result.errValue()
```

10.7 Where to Go Next

- Build the three projects
 - Contribute to Zebra itself
 - Share projects on GitHub
 - Write your own libraries
-

You've completed the Zebra programming guide. Welcome to the community!

